From Prompt to Response

An End-to-End Journey Inside LLM and RAG Systems

Table of contents

Welcome **1**

I The Art of the True Name 3

1 the intro **5**

II The Runes Beneath Language 7

2 APIs and Model Serving **9**
2.1 The gap between training and deployment . . . 9
2.2 The anatomy of an LLM API . . . 10
2.3 From checkpoint to serving: the loading pipeline . . . 11
2.4 The serving architecture . . . 12
2.5 Batching: the key to serving efficiently . . . 13
2.6 KV cache management . . . 14
2.7 Latency and throughput tradeoffs . . . 15
2.8 Key takeaways . . . 15
2.9 Further reading . . . 17

3 Tokenization **19**
3.1 Two tokenization chapters, two different problems . . . 19
3.2 The encoding pipeline . . . 19
3.3 Token continuation and context window management . . . 21
3.4 The decoding pipeline: from numbers back to text . . . 22
3.5 Logprobs and their uses at inference time . . . 23
3.6 Common failure modes . . . 24
3.7 Key takeaways . . . 24
3.8 Further reading . . . 25

4 Embeddings and Meaning Representation **27**
4.1 The problem with integers . . . 27
4.2 The embedding table . . . 28
4.3 How embeddings are learned . . . 28
4.4 The geometry of embedding space . . . 29
4.5 Contextual vs. static embeddings . . . 30
4.6 Embedding models vs. generation models . . . 30
4.7 Similarity metrics . . . 32
4.8 Embedding dimensions and matryoshka representations . . . 33
4.9 Embeddings in the generative inference pipeline . . . 33
4.10 Practical embedding operations . . . 34

4.11 Key takeaways . . . 35
4.12 Further reading . . . 37

III The Labyrinth of Attention 39

5 Transformer Architecture 41
5.1 Why the transformer won . . . 41
5.2 The intuition before the math . . . 42
5.3 The high-level forward pass . . . 42
5.4 Token embeddings and positional encoding . . . 43
5.5 Self-attention: the core mechanism . . . 45
5.6 Multi-head attention . . . 47
5.7 The feed-forward network . . . 48
5.8 Residual connections and layer normalization . . . 48
5.9 Stacking transformer blocks . . . 50
5.10 Architecture variants . . . 50
5.11 The output layer . . . 53
5.12 What the model actually learns . . . 54
5.13 Key takeaways . . . 54
5.14 Further reading . . . 54

6 Attention Computation 57
6.1 The quadratic problem . . . 57
6.2 What the GPU actually does during attention . . . 58
6.3 Standard attention: the naive computation . . . 58
6.4 FlashAttention . . . 59
6.5 Multi-head attention at inference . . . 60
6.6 The KV cache at inference . . . 61
6.7 Sparse and linear attention . . . 61
6.8 Ring attention and sequence parallelism . . . 63
6.9 Attention in the decode loop: step by step . . . 64
6.10 Key takeaways . . . 65
6.11 Further reading . . . 65

7 KV Cache 67
7.1 The problem that KV cache solves . . . 67
7.2 What is stored . . . 68
7.3 Attention with and without a KV cache . . . 68
7.4 Memory arithmetic . . . 69
7.5 The lifecycle of a KV cache entry . . . 70
7.6 Naive allocation and its problems . . . 71
7.7 PagedAttention . . . 71
7.8 KV cache quantization . . . 73
7.9 KV cache eviction . . . 73
7.10 Key takeaways . . . 74
7.11 Further reading . . . 74

8 GPU Execution 77
8.1 Why GPU execution deserves its own chapter . . . 77
8.2 GPU architecture fundamentals . . . 78
8.3 The roofline model . . . 79
8.4 Prefill vs. decode: two different regimes . . . 80
8.5 Kernel fusion . . . 81
8.6 Precision and quantization at the kernel level . . . 82

8.7 Tensor parallelism at the kernel level . . . 83
8.8 CUDA graph capture . . . 84
8.9 Key takeaways . . . 84
8.10 Further reading . . . 85

9 Decoding and Sampling 87
9.1 The decoding problem . . . 87
9.2 Greedy decoding . . . 88
9.3 Beam search . . . 88
9.4 Temperature sampling . . . 89
9.5 Top-k sampling . . . 89
9.6 Nucleus (top-p) sampling . . . 90
9.7 Min-p sampling . . . 91
9.8 Repetition penalties . . . 91
9.9 Structured decoding and constrained generation . . . 92
9.10 Best-of-n sampling . . . 93
9.11 Speculative sampling . . . 93
9.12 Chain-of-thought and reasoning tokens . . . 95
9.13 Decoding strategy selection guide . . . 95
9.14 Key takeaways . . . 96
9.15 Further reading . . . 98

IV The Forbidden Archive 99

10 Parametric Memory vs External Memory 101
10.1 Two different ways of knowing . . . 101
10.2 What parametric memory is . . . 102
10.3 What external memory is . . . 102
10.4 The complementary structure . . . 103
10.5 When to rely on parametric memory . . . 103
10.6 When to rely on external memory . . . 104
10.7 The interaction between the two . . . 104
10.8 Forms of external memory beyond RAG . . . 105
10.9 Designing the memory split . . . 106
10.10 The emerging continuum . . . 106
10.11 Key takeaways . . . 107
10.12 Further reading . . . 107

11 Document Ingestion 109
11.1 The ingestion problem . . . 109
11.2 Document formats and their challenges . . . 110
11.3 Text cleaning . . . 111
11.4 Document metadata extraction . . . 112
11.5 The ingestion pipeline assembled . . . 113
11.6 Ingestion at scale . . . 115
11.7 Key takeaways . . . 115
11.8 Further reading . . . 116

12 Chunking Strategies 117
12.1 Why chunking exists and why it is hard . . . 117
12.2 Fixed-size chunking . . . 118
12.3 Sentence-based chunking . . . 119
12.4 Recursive character text splitting . . . 119
12.5 Structure-aware chunking . . . 120

12.6 Semantic chunking . . . 122
12.7 Chunk enrichment . . . 124
12.8 Chunk size selection . . . 126
12.9 Multi-granularity retrieval . . . 126
12.10Key takeaways . . . 127
12.11Further reading . . . 128

13 Embeddings for Search **131**
13.1 This chapter vs. chapter 06 . . . 131
13.2 The retrieval embedding task . . . 132
13.3 Choosing an embedding model . . . 132
13.4 Asymmetric encoding and instruction prefixes . . . 133
13.5 Batching and throughput for corpus embedding . . . 134
13.6 Domain adaptation . . . 136
13.7 Embedding stability and versioning . . . 137
13.8 Embedding quality signals . . . 138
13.9 Late interaction models: ColBERT . . . 140
13.10Key takeaways . . . 140
13.11Further reading . . . 141

14 Vector Databases **143**
14.1 What a vector database does . . . 143
14.2 How ANN Search Works . . . 144
14.3 Vector database landscape . . . 147
14.4 Metadata filtering . . . 151
14.5 Key takeaways . . . 152
14.6 Further reading . . . 153

15 Retrieval Algorithms **155**
15.1 The retrieval problem . . . 155
15.2 Dense retrieval: vector similarity search . . . 156
15.3 Sparse retrieval: BM25 . . . 156
15.4 Hybrid retrieval . . . 157
15.5 Query expansion and reformulation . . . 158
15.6 Contextual and conversational retrieval . . . 158
15.7 Knowledge graph retrieval . . . 159
15.8 Retrieval evaluation . . . 160
15.9 The full retrieval pipeline . . . 161
15.10Key takeaways . . . 162
15.11Further reading . . . 162

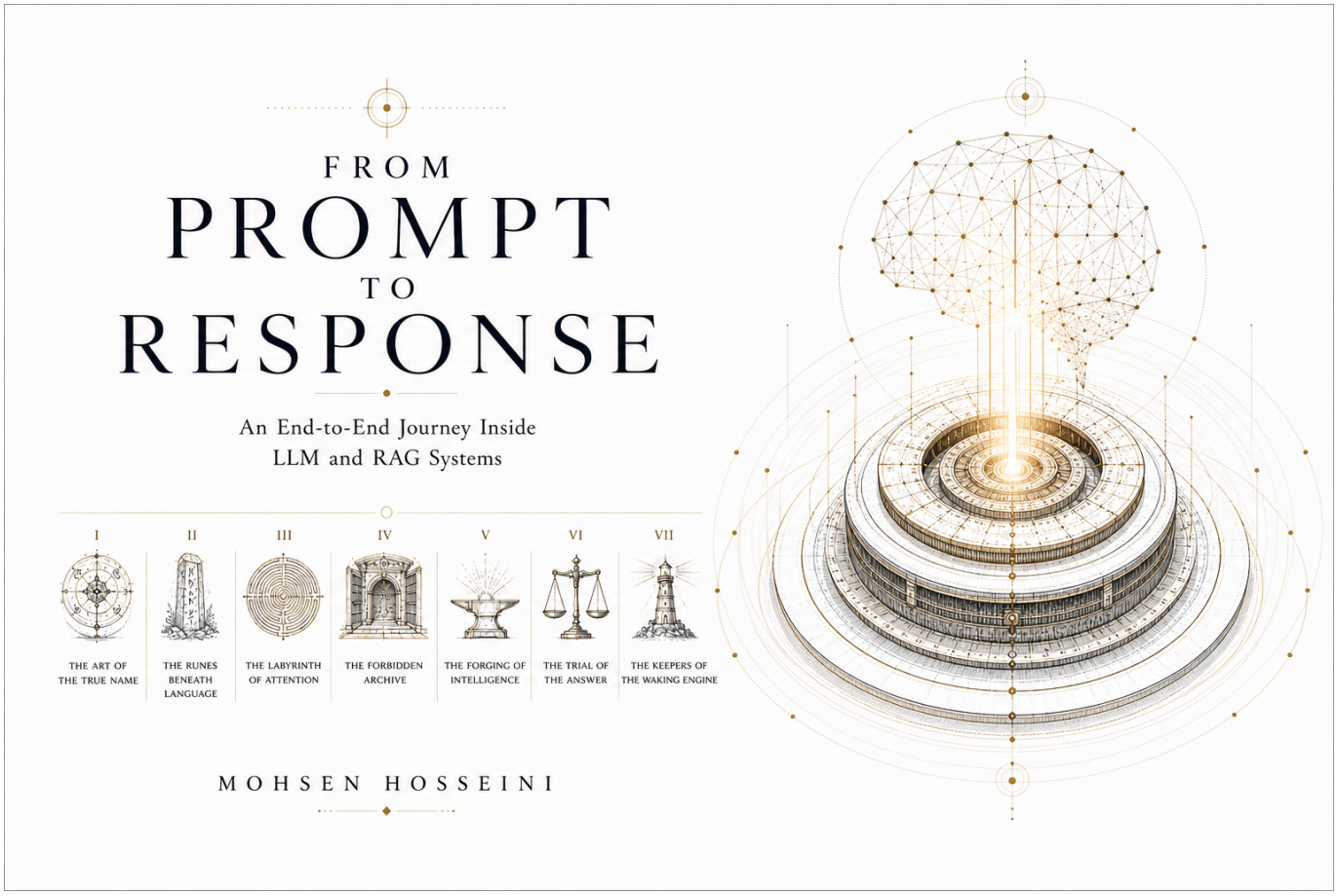
16 Reranking **165**
16.1 Why reranking exists as a separate stage . . . 165
16.2 The spectrum of reranking approaches . . . 166
16.3 Score-based reranking . . . 166
16.4 Cross-encoder reranking . . . 167
16.5 LLM-based reranking . . . 169
16.6 Diversity-aware reranking . . . 171
16.7 Contextual compression . . . 173
16.8 The full reranking pipeline . . . 174
16.9 Training custom rerankers . . . 175
16.10Measuring reranking quality . . . 175
16.11Key takeaways . . . 176
16.12Further reading . . . 178

V The Forging of Intelligence 179

VI The Trial of the Answer 181

VII The Keepers of the Waking Engine 183

Welcome



A technical deep-dive into the full LLM stack, from training and alignment through serving, retrieval, evaluation, and production operation. Every chapter is built around a single question that unlocks a layer of the system.

Designed for engineers and researchers who work with LLM systems and want to understand not just how to use them, but how they behave end-to-end under real constraints.

Part I

The Art of the True Name

Chapter 1

the intro

Part II

The Runes Beneath Language

Chapter 2

APIs and Model Serving

The canonical question for this chapter: *A trained model is a file on a disk. How does it become something millions of people can query simultaneously, with low latency, high reliability, and predictable cost?*

Figure 2.1: The journey through the Model Mind.

The prompt has been written and structured. Now it needs to travel somewhere. This chapter covers everything between the user pressing Enter and the model receiving the request: the wire format, the serving infrastructure, and the engineering decisions that make large-scale LLM deployment economical.

2.1 The gap between training and deployment

Training produces a checkpoint: a collection of weight tensors saved to disk. At this point, the model can do nothing on its own. It has no interface. It cannot receive requests. It does not know what a user is.

Turning that checkpoint into a production service requires solving a completely different set of engineering problems from those that were involved in training. During training, we try to optimize for producing as many tokens as possible per second. On the other hand, during serving, the optimization focuses on latency, concurrency, reliability, and cost, which means we want to respond to requests quickly, serve many users simultaneously, remain available under failure, and do all of this economically.

These requirements require a different approach. The same GPU cluster that runs training efficiently is not automatically an efficient serving infrastructure. Serving is its own engineering discipline, with its own abstractions, its own failure modes, and its own optimization techniques.

This chapter covers the full stack from trained checkpoint to the API endpoint a user or application calls.

2.2 The anatomy of an LLM API

The standard interface for production LLM services is an HTTP API, almost universally following the conventions established by OpenAI's API.

2.2.1 The chat completions endpoint

The primary endpoint for most production use:

`POST /v1/chat/completions`

Request body:

```
{
  "model": "gpt-4o",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What is the capital of France?"}
  ],
  "max_tokens": 100,
  "temperature": 0.7,
  "stream": false
}
```

Response:

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1699000000,
  "model": "gpt-4o",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "The capital of France is Paris."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 26,
    "completion_tokens": 9,
    "total_tokens": 35
  }
}
```

Several design decisions in this interface iluminates the realities of LLM serving:

Messages array instead of a single string. Conversations have a clear structure consisting of a system prompt, user turns, and assistant turns. The API explicitly defines the structure, instead of relying on concatenated strings and expecting the model to interpret the formatting correctly.

Token counts in the response. Because LLM cost and latency are proportional to token count, the API reports exactly how many tokens were consumed allowing costs to be monitored and controlled programmatically.

`finish_reason`. Why did the model stop generating? `stop` means it produced an end-of-sequence token. `length`

means it reached the `max_tokens`. `content_filter` means the response was filtered. These signals help to determine whether the response is complete

`stream` parameter. Whether to stream tokens as they are generated or return the complete response when done. Streaming is architecturally significant and covered in detail in chapter "REFF".

2.2.2 The completions endpoint (legacy)

An older interface that takes the raw string prompt rather than the structured messages array:

```
{
  "model": "gpt-3.5-turbo-instruct",
  "prompt": "The capital of France is",
  "max_tokens": 10
}
```

This endpoint represents an earlier era of LLM deployment, when models were not chat tuned and the interface was closer to a text continuation service. It is still useful for non-chat use cases but has been superseded by chat completions for most applications.

2.2.3 Key parameters

`temperature` controls the randomness of sampling. At temperature 0, the model always selects the most likely next token (greedy decoding). At temperature 1, it samples from a full distribution. Above 1, the distribution flattens, producing more random and unstable outputs. Temperature interacts with the decoding strategy covered in chapter "REFF".

`top_p` (nucleus sampling) this is an alternative to temperature for controlling randomness. The model samples from the smallest set of tokens in which the cumulative probability exceeds `top_p`. Setting `top_p = 0.9` restricts sampling to tokens that together account for 90% of the probability mass that means the long tail of unlikely tokens is ignored.

`max_tokens` sets the maximum number of tokens to generate. It functions as a cost and latency cap. If the model reaches this limit before producing a stop token, `finish_reason` is `length` and the response may be end mid-sentence.

Stop sequences are one or more strings that cause halt in generation immediately if the model produces them. They are useful for structured generation where you know the output should end at a specific delimiter.

`frequency_penalty` and `presence_penalty` discourage repetition. `frequency_penalty` applies a penalty proportional to how many time a token has already appeared. `presence_penalty` applies a flat penalty to any token that has appeared at all. Both help the model to prevent from getting stuck in repetitive loops.

`n` specifies how many independent completions to generate for a single prompt. Useful for best-of-n sampling where you generate multiple responses and select the best one.

`logprobs` returns the log probabilities of the selected tokens and also the top-k most likely tokens at each position. It is used for calibration, analysis, and classification tasks where you care more about the model's confidence, not just its output.

2.3 From checkpoint to serving: the loading pipeline

Before a model can serve requests, it must be loaded. For a model with hundreds of billions of parameters, this is a nontrivial task.

2.3.1 Checkpoint format and sharding

Training checkpoints are often sharded due to the fact that the full model does not fit in the memory of a single storage node or loading process. A 70B parameter model in BF16 requires approximately 140 GB of storage and Loading it

into a GPU memory requires careful orchestration.

Common checkpoint formats:

- **PyTorch `.pt` / `.bin`**: the standard PyTorch format which can be slow to load due to pickle deserialization
- **SafeTensors**: a faster and safer alternative which can be loaded significantly faster via memory-mapped files
- **GGUF**: a format used by llama.cpp, optimized for inference on consumer hardware and it is able to quantize weights natively

2.3.2 Quantization for serving

Training typically utilizes BF16 weights (2 bytes per parameter). For serving, quantization reduces this further:

INT8 quantization stores weights as 8-bit integers (1 byte per parameter). This achieves a 2× memory reduction which causes a small quality loss that is mostly negligible for generation tasks.

INT4 quantization uses 4-bit integers (0.5 bytes per parameter) for a 4× memory reduction. Causes more quality loss and needs careful calibration. Enables running 70B models on consumer GPUs that could not otherwise fit them.

GPTQ is a post-training quantization technique that calibrates 4-bit quantization using a small dataset, minimizing quality loss from reduced precision.

AWQ (Activation-aware Weight Quantization) quantizes weights while accounting for the distribution of activations, producing better results than naive quantization at the same bit width.

The tradeoff is consistent: the quantization reduces memory requirements while increase throughput, at the cost of some output quality. For production serving where cost matters, INT8 is nearly universal. INT4 is used when memory is severely constrained or when we want to serve on consumer hardware.

2.3.3 Tensor parallelism for large models

A 405B parameter model in BF16 requires approximately 810 GB of GPU memory that is far beyond a single GPU's capacity. In these scenarios, loading requires distributing the model across multiple GPUs using tensor parallelism (splitting individual weight matrices across GPUs) or pipeline parallelism (assigning different layers to different GPUs).

The same parallelism strategies used in training apply to serving, but with different constraints: training prioritizes throughput (large batches, high utilization), while serving prioritizes latency (fast first-token generation, smaller batches).

2.4 The serving architecture

2.4.1 The inference server

The core component: a process that holds the model weights in GPU memory and handles requests. The major implementations:

vLLM is the dominant open-source inference server. It introduces PagedAttention (covered in detail in chapter "REFF") for efficient KV cache management, and supports tensor parallelism, continuous batching, and most major model architectures.

TensorRT-LLM (NVIDIA) is NVIDIA's inference optimization library. It applies kernel fusion, INT8/FP8 quantization, and hardware-specific optimizations. Typically the fastest option on NVIDIA hardware but requires compilation for specific model shapes which is a trade between flexibility and performance.

llama.cpp is a pure C++ implementation of LLaMA-family models, optimized for CPU and consumer GPU inference. Not competitive with vLLM for high-throughput serving, but it is unmatched for accessibility and local deployment.

2.4.2 The API gateway

In front of the inference server sits the API layer:

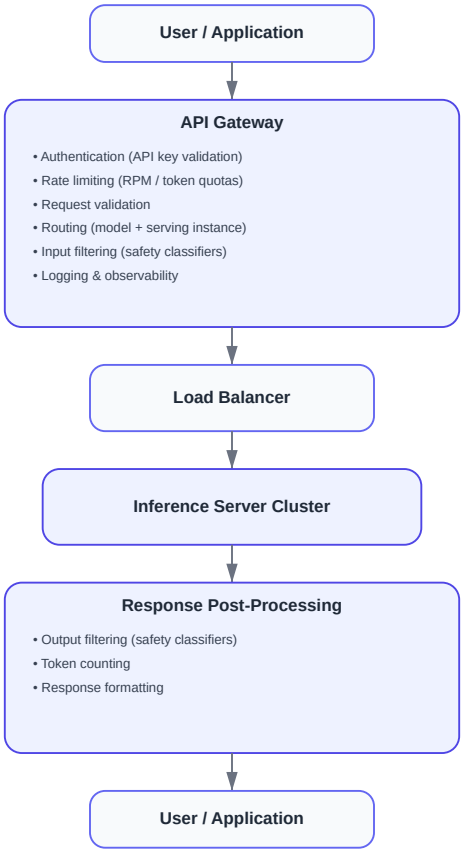


Figure 2.2: The API gateway.

The API gateway is not a performance bottleneck what it does is to process the metadata. It is critical for security (authentication prevents unauthorized access), cost control (rate limiting prevents abuse), reliability (routing distributes traffic and handles failures), and observability (logging every request enables debugging and billing).

2.5 Batching: the key to serving efficiently

A single inference server instance can handle many requests simultaneously. How it batches those requests together affects throughput and GPU utilization.

2.5.1 Naive static batching

The simplest approach is to wait until you have `batch_size` requests, process them together, return all results.

The problem is that LLM generation is variable length. On request might generate 10 tokens while the other might generate 500. In a static batch, all requests must wait for the longest one to finish. GPU utilization drops as shorter requests complete and their slots sit idle, waiting.

2.5.2 Continuous batching

The key innovation for efficient LLM serving, introduced by [1] and popularized by vLLM.

Instead of batching at the request level, continuous batching operates at the iteration level meaning it happens at every token generation step. After each token is generated:

1. Check which sequences in the current batch have finished
2. Remove finished sequences from the batch
3. Add new incoming requests to fill the freed slots
4. Run the next generation step on the updated batch

```
Step 1: [A , B , C ]         ← three requests, first token
Step 2: [A , B , C ]
Step 3: [A , B , C_stop]     ← C finishes after 3 tokens
Step 3: [A , B , D ]         ← D immediately fills C's slot
Step 4: [A , B , D ]
Step 5: [A_stop, B , D ]     ← A finishes
Step 5: [E , B , D ]         ← E fills A's slot
```

The GPU is always processing a full batch. None of the slots sit idle waiting for the longest sequence to finish. On real workloads, this approach can improve throughput by 2 to 3 times compared to a naive static batching. Continuous batching requires careful KV cache management which explains why PagedAttention exists.

2.5.3 Prefill and decode phases

LLM generation has two distinct phases with different compute profiles:

Prefill: processing the input prompt. All prompt tokens are processed in parallel. This phase is compute bound and the bottleneck is GPU arithmetic throughput.

Decode: generating output tokens one at a time. Each step processes one single new token, attending to all previous tokens via the KV cache. This phase is memory-bandwidth-bound and the bottleneck is reading the model weights and KV cache from GPU memory.

Because prefill and decode have different bottlenecks, mixing them in the same batch is not optimal. **Chunked prefill** and **disaggregated serving** are emerging techniques that separate prefill and decode across different hardware or batch slots to maximize utilization of each phase independently.

2.6 KV cache management

The KV cache stores the key and value tensors computed for each previous token, avoiding recomputation on every generation step. It is the central memory management challenge in serving, and will be explained in chapter “REFF”.

A brief introduction here, because it directly constrains serving architecture:

For a model with `L` layers, `h` KV heads, `d_k` key/value dimension, and a sequence of length `n`:

```
KV cache size = 2 × L × h × d_k × n × bytes_per_element
```

For LLaMA 2 70B (L=80, h=8 GQA heads, d_k=128) in BF16:

```
Per token: 2 × 80 × 8 × 128 × 2 = 327,680 bytes  320 KB
4,096 tokens: 320 KB × 4,096  1.3 GB
32,768 tokens: 320 KB × 32,768  10.5 GB
```

At high concurrency with long contexts, KV cache competes directly with model weights for GPU memory.

2.6.1 PagedAttention

The key innovation in vLLM, inspired by virtual memory paging in operating systems.

Traditional KV cache reserves a contiguous memory block for each sequence, sized to accommodate the maximum possible length. This causes internal fragmentation (reserved memory that goes unused when sequences are shorter than expected) and external fragmentation (free memory that cannot be used because it is not contiguous).

PagedAttention allocates KV cache in fixed-size pages of tokens, with a page table mapping logical sequence positions to physical memory locations. Memory is allocated and freed one page at a time as sequences grow and complete:

```
Sequence A: [Page 3, Page 7, Page 12]     ← pages need not be contiguous
Sequence B: [Page 1, Page 9]
Sequence C: [Page 2, Page 4, Page 5, Page 11]
Free pages: [Page 6, Page 8, Page 10, ...]
```

This will eliminates fragmentation and enables an almost perfect memory utilization. It also enables prefix sharing which means if two requests share a common system prompt, the KV cache for that prefix can be shared across both sequences without duplication.

2.7 Latency and throughput tradeoffs

Serving performance is characterized by two metrics that pull against each other:

Time to first token (TTFT) is how long the user waits before seeing any output. This is determined primarily by prefill time and since longer prompts take longer to prefill it is the metric users perceive most directly.

Time between tokens (TBT) is how fast tokens arrive after the first one. This is determined by decode speed. For a 70B model on a single A100, 20 to 60 tokens per second is typically expected.

Throughput is total tokens processed per second across all concurrent requests. Maximized by large batch sizes, which directly conflict with low TTFT.

Production systems manage this tradeoff through maximum batch size limits (cap batch size to maintain acceptable latency), priority queuing (prioritize interactive requests over batch workloads), and SLA-based scheduling (different latency targets for different user tiers).

2.8 Key takeaways

- A trained model checkpoint becomes a production service through a pipeline involving quantization, multi-GPU loading, an inference server, and an API gateway, since training and serving are distinct engineering disciplines
- The chat completions API format, including the messages array, key parameters, and token usage reporting, reflects the practical realities of LLM serving rather than arbitrary design choices
- Continuous batching operates at the token level rather than the request level, keeping GPUs fully utilized despite variable-length generation and serving as the main efficiency improvement for high-throughput systems
- KV cache memory is the central serving constraint because it grows with sequence length and batch size and competes directly with model weights for GPU memory, while PagedAttention manages it using virtual memory paging
- Prefill is compute bound and decode is memory bandwidth bound, so they have fundamentally different performance profiles and can be optimized independently
- TTFT and throughput trade off against each other, and the appropriate balance depends on whether the system serves interactive applications or batch workloads

---

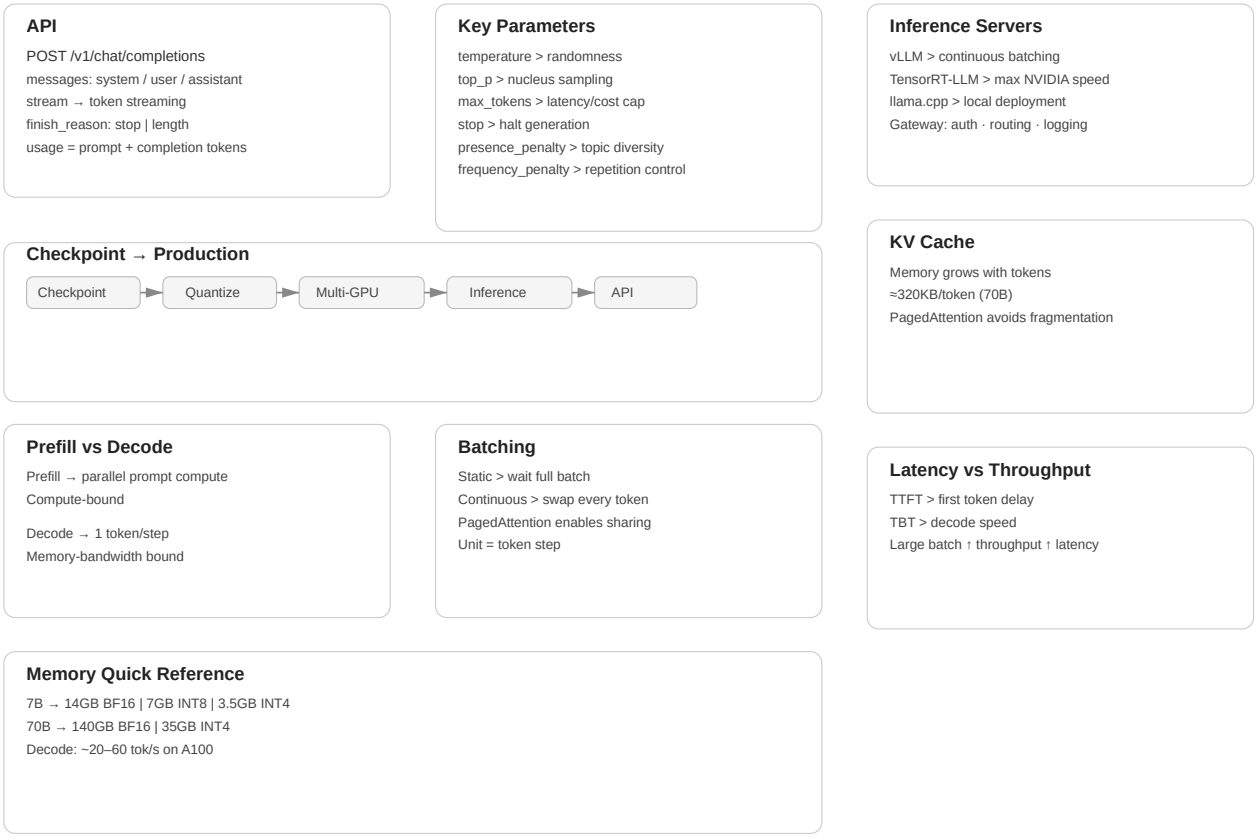


Figure 2.3: Cheat sheet.

2.9 Further reading

- Yu et al. (2022). *Orca: A Distributed Serving System for Transformer-Based Generative Models.*: Introduces continuous batching.
- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.*: The vLLM paper; foundational for modern serving.

Chapter 3

Tokenization

The canonical question for this chapter: *A user sends a message while a model receive and returns numbers. How does text become numbers on the way in, and how do numbers become text on the way out?*

Figure 3.1: The journey through the Model Mind.

The request has arrived at the inference server. Before the model sees a single character, the text must be converted to numbers. This chapter covers that conversion in both directions and describes everything that can go wrong in between.

3.1 Two tokenization chapters, two different problems

If you read the training section first, you have already seen tokenization as in a batch preprocessing environment which means a corpus is tokenized once, offline, before training begins. The vocabulary is fixed. The inputs are known and errors are caught early.

Tokenization at inference is a different story. It runs on arbitrary user input, in real time, and on every request. It supposed to handle every string a user might type which could be multilingual text, source code, emoji, adversarial inputs, or a combination of all four. The encoded output feeds directly into a running model. The decoded output appears on a user's screen, sometimes token by token while they watch.

The concerns here are mostly operational, not theoretical. Understanding them is the difference between shipping reliable LLM applications or debugging corruption failures at 2am.

3.2 The encoding pipeline

When a request arrives, the first operation is encoding: converting the messages array into a flat sequence of token IDs ready for the model. This happens in two steps that are easy to confuse but are conceptually different.

3.2.1 Step 1: chat template application

The messages array has structure which includes roles, turns, and system prompts. The model has no concept of a dictionary. Before tokenization, the structured input must be flattened into a single string using a chat template.

Each model family has its own template. For LLaMA 3:

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>

You are a helpful assistant.<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is the capital of France?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

For Mistral:

```
<s>[INST] What is the capital of France? [/INST]
```

For ChatML (OpenAI and many others):

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
What is the capital of France?<|im_end|>
<|im_start|>assistant
```

The special tokens (`<|begin_of_text|>`, `<|eot_id|>`, `<|im_start|>`) are part of the model's vocabulary and were added during fine-tuning. The model learns to associate these markers with role transitions. Feeding the wrong template to the model causes it to sees an unexpected token sequence which in turn may cause the model to ignore the system prompt, and fail to complete the turn correctly, or produces outputs that look almost right but are systematically wrong in ways that can be hard to diagnose. This is one of the most common sources of bugs: no error is thrown yet the model just quietly misbehaves.

3.2.2 Step 2: token encoding

Once the flat string is constructed, the tokenizer converts it to integer IDs. This is a deterministic step in which the same string always produces the same IDs given the same tokenizer.

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user",   "content": "What is the capital of France?"}
]

# Step 1: flatten with chat template
flat_string = apply_chat_template(messages, tokenizer)
# → "<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n..."

# Step 2: convert to integers
token_ids = tokenizer.encode(flat_string)
# → [151644, 8948, 198, 2610, 525, 264, 10950, 17847, 13, ...]

# Step 3: pass to model
logits = model.forward(token_ids)
```

The tokenizer runs on CPU and for typical prompt lengths it is fast enough to be negligible relative to model inference time. On the other hand, for very long prompts (100k+ tokens) it can become measurable and there are some inference frameworks that cache tokenized system prompts to avoid repeating this work on every request.

3.2.3 Special tokens that require explicit handling

Several special tokens need attention beyond the template:

Beginning of sequence (BOS). Many models expect a BOS token at position 0 and the tokenizer typically adds it automatically while some serving configurations require manual insertion. A missing BOS token causes silent quality degradation which means the model's input distribution does not match training, and early tokens may be erratic.

End of turn markers. The template includes tokens marking the end of each turn and it must appear in the correct positions or the model misidentifies role boundaries.

Tool call tokens. Models fine-tuned for function calling have additional special tokens called tool call syntax. These must be in the tokenizer vocabulary and correctly placed in the template.

The generation prompt. The template typically ends with the opening of the assistant turn (e.g., `<|im_start|>assistant\n`) without closing it. This will signal the model that it should continue from here and omitting it causes the model to generate an incorrect turn structure.

3.3 Token continuation and context window management

The context window is the maximum sequence length the model can process in a single forward pass which is typically a range between 8k to 128k tokens for current models. Managing what fits in this window is a critical operational concern.

3.3.1 Count with the tokenizer, not your intuition

Token counts are not characters divided by four, or words divided by 0.75. They are the actual output of the tokenizer on the specific input string. Heuristics are inaccurate enough to cause real failures:

- Code tokenizes very differently from prose which means identifiers, operators, and whitespace have their own patterns
- Non-English text is often less efficient than English for most tokenizers
- Special tokens (role markers, tool tokens) add counts that character estimates miss entirely

Every production system that needs to stay within a context window must count tokens using the actual tokenizer, not an approximation:

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4o")

def count_tokens(messages: list[dict]) -> int:
    num_tokens = 0
    for message in messages:
        num_tokens += 4  # role, content, separators overhead
        for value in message.values():
            num_tokens += len(enc.encode(value))
    num_tokens += 2  # reply priming tokens
    return num_tokens
```

The exact overhead is varied by model and template. Always validate against the API's reported token counts on a representative sample.

3.3.2 The context window budget

In a typical production request, the context window is shared between several competing claims:

```
Total context window (e.g., 128k tokens)
   System prompt              (fixed, e.g., 500-2,000 tokens)
   Conversation history       (grows with each turn)
   Retrieved context (RAG)    (variable, e.g., 2,000-10,000 tokens)
   Current user message       (variable)
   Reserved for completion    (max_tokens parameter)
```

The sum must not exceed the window and if it does, the request will fail or the serving layer silently truncates something which is usually the earliest conversation history, that could be the exact context the user needs the most. Applications must actively manage this budget rather than hoping it fits.

Rolling window keeps only the most recent N tokens of history. While it is simple, it loses early context which can be fine for short tasks and problematic for long conversations.

Summarization uses the model to compress earlier turns into a compact representation when history approaches the limit. With this approach it preserves semantic content at the cost of an additional model call.

Hierarchical retrieval stores full history externally and only retrieve the most relevant portions for the current turn. This approach requires a retrieval component but it can scale to arbitrarily long conversations.

3.3.3 Prompt caching and its token implications

Some APIs (Anthropic's Claude, Google's Gemini) offer explicit prompt caching: mark a portion of the prompt as cacheable, and subsequent requests that share the same prefix reuse the KV cache from the first request, charged at a reduced rate.

For cost management, structure prompts with the stable content (system prompt, long documents) first and variable content (user messages, dynamic context) in the end. Token counting for cached prompts will distinguishs between newly processed tokens and cache-read tokens, as the API reports these separately and they have completely different cost implications.

3.4 The decoding pipeline: from numbers back to text

After the model selects the next token ID (the selection process is covered in chapter "REFF"), the serving layer converts that ID back to text. This is less straightforward than it appears.

3.4.1 Detokenization

The inverse operation: token IDs to string.

```
token_ids = [15223, 374, 279, 6864, 315, 9822, 13]
text = tokenizer.decode(token_ids)
# → "Paris is the capital of France."
```

While it seems that detokenization simply involves looking up each token's string and concatenating them together, several complications exist:

Byte-level BPE and UTF-8 reconstruction. In byte-level tokenizers, tokens represent raw bytes rather than complete characters. Because UTF-8 characters are able to span multiple bytes, a single character (many Chinese characters or emojis) may be splited across several tokens. Decoding each token independently can therefore produce invalid text or replacement symbols. The correct procedure is to concatenate all token bytes first and then perform a single UTF-8 decode, ensuring the original characters are correctly reconstructed.

Special tokens in output. If the model produces a special token (end of text, role marker, tool call token), detokenization must strip it, convert it to a meaningful representation, or treat it as a halt signal. Which action is correct totally depends on context and must be handled explicitly.

3.4.2 Streaming detokenization

In streaming mode, tokens arrive one at a time and must be decoded incrementally which creates a specific problem: a byte sequence that spans multiple tokens cannot be decoded until all of its bytes have arrived.

Consider a Chinese character requiring 3 UTF-8 bytes, split across three tokens of 1 byte each:

- After token 1: 1 byte received and it is not a valid UTF-8 codepoint yet and cannot emit
- After token 2: 2 bytes received and it is still incomplete
- After token 3: 3 bytes received finally it is completed, decode and emit the character

A streaming decoder must buffer incomplete byte sequences and emit characters only once a complete Unicode codepoint is available. A naive implementations that pass `errors="ignore"` or `errors="replace"` will silently corrupt non-ASCII output while the HuggingFace `TextStreamer` and similar utilities handle this correctly.

3.5 Logprobs and their uses at inference time

Most APIs can optionally return the log probabilities of generated tokens which is more useful than it might initially appear.

3.5.1 What logprobs tell you

For each generated token, logprobs report the log probability the model assigned to the chosen token, and optionally the top-k alternatives with their probabilities:

```
{
  "token": "Paris",
  "logprob": -0.0023,
  "top_logprobs": [
    {"token": "Paris", "logprob": -0.0023},
    {"token": "Lyon",  "logprob": -6.12},
    {"token": "Nice",  "logprob": -7.88}
  ]
}
```

A logprob of -0.0023 means the model assigned probability `exp(-0.0023)` `0.998` to this token which means the model is essentially certain. A logprob of -6.12 means `exp(-6.12)` `0.002` and represent a distant second choice.

3.5.2 Classification without generation

For binary or multiple-choice tasks, logprobs let you skip generating a full response and instead read the probabilities of specific tokens directly which is faster and cheaper:

```
response = client.completions.create(
    model="gpt-4o",
    prompt=f"{text}\n\nIs this sentiment positive? Answer with yes or no.",
    max_tokens=1,
    logprobs=True,
    top_logprobs=5
)

top = response.choices[0].logprobs.top_logprobs[0]
yes_logprob = next(t.logprob for t in top if t.token.strip().lower() == "yes")
no_logprob  = next(t.logprob for t in top if t.token.strip().lower() == "no")
```

```
is_positive = yes_logprob > no_logprob
```

3.5.3 Calibration and uncertainty estimation

The perplexity of a generated response which is represented by the average negative log probability per token is a rough proxy for the model's confidence:

```
import numpy as np

logprobs = [token.logprob for token in response.choices[0].logprobs.content]
perplexity = np.exp(-np.mean(logprobs))
# High perplexity → model was uncertain about this response
```

This is not a hallucination detector which means the model can be confidently wrong, and often is. But it is a useful routing signal: responses with high perplexity can be flagged for human review or follow-up verification without running a separate evaluation model.

3.6 Common failure modes

Tokenization at inference produces specific, reproducible failure patterns. Knowing them saves significant debugging time.

Wrong tokenizer for the model. Using the GPT-2 tokenizer with a LLaMA model produces completely wrong token ID sequences. The model interprets unrelated IDs as completely different tokens which in turn leads to a garbage output.

Wrong chat template. Applying the wrong template produces subtly wrong token sequences. The model may ignore the system prompt, continue the user turn instead of starting an assistant turn, or generate role labels as content. This also fails silently and is especially confusing because outputs look almost right but are systematically off.

BOS token doubling. Some serving frameworks add BOS automatically while some expect the caller to add it. If BOS appears twice, the model sees an unexpected sequence at position 0 and may behave erratically.

Context overflow without truncation. If the total token count exceeds the context window and the serving layer does not truncate explicitly, some APIs return an error while some silently truncate from the beginning.

Encoding/decoding asymmetry. `decode(encode(s))` may not equal `s` for strings containing Unicode normalization differences, special tokens with non-standard decoding, or whitespace that the tokenizer normalizes.

3.7 Key takeaways

- Chat template application: flattening the messages array into a string with model-specific role delimiters it happens before tokenization and is model-specific; the wrong template produces silent quality degradation
- Token counting must use the actual tokenizer, not character or word heuristics; accurate counting is essential for context window management and cost control
- The context window budget must be actively managed across system prompt, conversation history, retrieved context, and completion reservation
- Streaming detokenization must buffer incomplete UTF-8 byte sequences and emit characters only once a complete codepoint is available; naive implementations corrupt non-ASCII output
- Stop sequence detection operates on decoded strings, not token IDs, and must handle sequences that span token boundaries and appear at buffer edges

- Token healing solves the boundary problem for prompts ending at arbitrary character positions, improving completion quality for structured generation
- Logprobs enable efficient classification, uncertainty estimation, and confidence- based routing without generating full responses

---

3.8 Further reading

- HuggingFace Tokenizers documentation: Fast tokenizers and chat templates.
- OpenAI Cookbook: How to count tokens with tiktoken.
- HuggingFace (2023): Chat Templating Guide.

Tokenization at Inference, Cheat Sheet

Encoding Pipeline

1. Apply chat template → flatten structured messages

`<system> ... <user> ... <assistant>`

2. Tokenizer converts text → token IDs

`"Paris is capital" → [1512, 374, 279]`

Critical Special Tokens

- BOS token must exist at position 0
- End-of-turn markers define roles
- Tool tokens required for function calling
- Assistant prompt signals generation start

Context Window Budget

System prompt
Conversation history
RAG context
User message + completion tokens
Always count tokens using tokenizer, never estimate.

History Management Strategies

Rolling window → keep recent tokens only
Summarization → compress earlier turns
Hierarchical retrieval → fetch relevant past context

Decoding Pipeline

Model outputs token IDs → detokenizer reconstructs UTF-8 text

`[15223, 374, 279] → "Paris is the capital"`

Byte-level tokenizers require buffering before emitting characters.

Streaming Decoding

- Tokens arrive one at a time
- Partial UTF-8 bytes must be buffered
- Emit text only when character completes

Logprobs Uses

- Confidence estimation
- Classification without generation
- Perplexity ≈ uncertainty signal

Common Failure Modes

- Wrong tokenizer for model
- Wrong chat template → silent quality loss
- Double BOS token
- Context overflow / silent truncation
- encode(decode(x)) != x due to normalization

Figure 3.2: Cheat sheet.

Chapter 4

Embeddings and Meaning Representation

The canonical question for this chapter: *How does a model convert a token ID into something that has meaning, and how does that representation evolve as it passes through the network?*

Figure 4.1: The journey through the Model Mind.

Tokenization converts the prompt text into a sequence of integer IDs. These integers has no meaning a neural network on their own. This chapter covers the step that makes them meaningful and it introduces the geometric structure of meaning that the rest of the model operates on.

4.1 The problem with integers

A token ID is an arbitrary index. The token ID for `"Paris"` might be 15342 while the token ID for `"London"` is 9562. These numbers do not carry any information about the relationship between the two cities. They are just indices in a lookup table, nothing more.

An embedding is the thing that you get after that lookup: a dense vector of floating point numbers with hundreds or thousands of dimensions. The embedding for `"Paris"` might be something like `[0.23, -0.87, 0.12, ...]` with 4096 values and the embedding for `"London"` is a completely different vector nevertheless in a well-trained model, it will be close to Paris's vector in this high-dimensional space, because the two tokens appear in similar contexts throughout the training corpus.

This is the core insight: geometric proximity in embedding space convey the semantic relatedness. Tokens that appear in similar contexts have similar embeddings. The model does not learn this rule explicitly it caused by gradient descent on the next-token prediction objective. Training discovers that representing semantically related tokens as nearby vectors produces better predictions, and adjusts accordingly over trillions of examples.

Embeddings are the bridge between the discrete world of tokens and the continuous world of neural network computation. Everything the transformer does which includes every attention operation, and every feed-forward pass operates on these continuous vectors and the model never touches the original token IDs again after the embedding lookup.

4.2 The embedding table

The embedding table is a matrix of shape `[vocab_size, d_model]`. Each row is the learned embedding for one token.

```
Embedding table (simplified, d_model = 4):

Token ID  Token     Embedding vector
0         <pad>     [ 0.00,  0.00,  0.00,  0.00]
1         <eos>     [ 0.12, -0.34,  0.87,  0.23]
2         "the"     [-0.45,  0.67, -0.12,  0.89]
3         "Paris"   [ 0.91,  0.23, -0.67,  0.45]
4         "London"  [ 0.88,  0.19, -0.71,  0.43]
5         "cat"     [-0.23,  0.89,  0.34, -0.67]
```

The lookup is a simple indexed memory access: `embedding = table[token_id]`. Implemented as a matrix multiplication with a one-hot vector in theory; in practice it is O(1) and takes microseconds.

For GPT-3 with `vocab_size = 50,257` and `d_model = 12,288`:

```
Embedding table size = 50,257 × 12,288 × 2 bytes (BF16)
                     1.24 billion parameters
                     2.47 GB
```

This is a significant fraction of total model size and it is also the most memory efficient part of the model relative to its influence: every token the model processes uses exactly one row of this table.

4.2.1 Weight tying

The same embedding table is used twice: once at the input (token IDs → embeddings) and once at the output (final hidden state → logits over vocabulary). This is called weight tying and is standard in most transformer language models.

At the output, the hidden state `h` of shape `[d_model]` is multiplied by the transposed embedding table `E^T` of shape `[d_model, vocab_size]` to produce logits of shape `[vocab_size]`:

```
logits = h @ E^T
```

Weight tying reduces the parameter count by `vocab_size × d_model` and, more importantly it enforces a geometric consistency: the direction that represents a token at the input is the same direction the model uses to predict that token at the output. This constraint improves training efficiency and generalization and it is one of those architectural decisions that looks obvious in hindsight yet it was not obvious at all beforehand.

4.3 How embeddings are learned

At the start of training, the embedding table is initialized from a small random distribution and it does not carry any semantic information they are just noise.

Through training, gradient descent updates each embedding row whenever its token appears in the training data. The update moves the embedding in the direction that reduces prediction loss. Over millions of update steps on millions of tokens, the embeddings converge to representations that capture the statistical structure of language.

The mechanism works by giving tokens that appear in similar contexts, similar gradient updates. If `"Paris"` and `"London"` both appear after `"capital of"` and before `"is beautiful"`, their embeddings are repeatedly pushed in similar directions. Over enough training, they converge to nearby vectors.

This is the distributional hypothesis, operationalized by gradient descent: *you shall know a word by the company it keeps.*

4.3.1 The rare token problem

Embedding parameters receive less frequent updates than other model parameters. A token that appears once in a billion token batch updates once while a common token might get updates in every batch. This creates an implicit learning rate inequality: frequent tokens have well trained embeddings but rare tokens may be poorly trained even in large models.

This is one reason why multilingual models struggle with low-resource languages, and why domain specific jargon may get handled poorly even in a large general model due to this simple fact that the embeddings for rare tokens have not seen enough gradient signal to settle into a useful position.

4.4 The geometry of embedding space

Trained embeddings organize into a rich geometric structure. Understanding this structure is practically useful for interpreting the model behavior and for building a retrieval system.

4.4.1 Semantic neighborhoods

Semantically related tokens cluster together. In embedding space, countries cluster near each other, verbs in the same semantic field cluster together, synonyms are close, and antonyms are often directionally opposite. This is not an artifact of any particular architecture it emerges from the distributional statistics of the language and it is similar across all model families.

4.4.2 Linear structure

The most famous property of word embeddings is the fact that analogical relationships correspond to linear operations.

```
embedding("king") - embedding("man") + embedding("woman")   embedding("queen")
embedding("Paris") - embedding("France") + embedding("Germany")   embedding("Berlin")
embedding("walked") - embedding("walk") + embedding("run")   embedding("ran")
```

Directions in embedding space encode semantic dimensions. The vector from `"man"` to `"woman"` is a gender direction. The vector from `"France"` to `"Paris"` is a capital of direction. Addition and subtraction of embedding vectors correspond to semantic composition and transformation.

This property approximately exist in static embeddings (Word2Vec, GloVe) and it continues to presist in the token embeddings of large transformers, though it might not be as clean as static embedding. The transformer layers above the embedding table further transform these representations in ways that create richer structure yet it makes it harder to extract clean linear relationships.

4.4.3 Isotropy and representation collapse

Embeddings occupy only a narrow cone of the available space, with most directions unused which makes it one of the failure modes in embedding training which is called anisotropy. When embeddings are anisotropic, cosine similarity between random embeddings is high even for semantically unrelated tokens and causes the embedding based retrieval unreliable.

Modern large models are less prone to this compared older smaller models, yet it remains a concern in fine-tuning models where the embedding distribution can shift significantly during training.

4.5 Contextual vs. static embeddings

The embedding table produces a single fixed vector for each token, regardless of context. The token `"bank"` gets the same embedding whether the surrounding context is about finance or about rivers.

This is the limitation of static embeddings an the transformer layers above the embedding table are able to solve it. After the initial embedding lookup, each transformer block modifies the token representations through attention and feed-forward operations and when it gets to the final layer, the representation of `"bank"` in a financial context is completley different from `"bank"` in a geological context. This happens not because the embedding table has changed, but because the transformer layers incorporated contextual information from surrounding tokens.

These contextual representations are called "contextual embeddings" to distinguish them from static lookup embeddings at the first layer. They are what BERT family models were designed to expose, and what RAG retrieval systems use when they need semantically rich representations.

4.5.1 The residual stream

This is a useful model for understanding how representations evolve through the network:

```
x_0 = token_embedding + positional_encoding

x_1 = x_0 + Attention_1(LayerNorm(x_0))
x_1 = x_1 + FFN_1(LayerNorm(x_1))

x_2 = x_1 + Attention_2(LayerNorm(x_1))
x_2 = x_2 + FFN_2(LayerNorm(x_2))

...

x_L = final hidden state → logits
```

Each layer adds a refinement to the running representation while the early layers tend to encode syntactic features, middle layers encode semantic relationships and the final layers encode task specific features that is used for prediction.

The residual structure means the original token embedding is always present in the stream and it gets added to every layer and never replaced. This is why the embedding table remains relevant even in very deep models: its contribution persists all the way through computation, like a base note that harmonics are built on top of.

4.5.2 Which layer to extract from

For tasks that use hidden states as embeddings, such as classification, retrieval, or similarity, a practical question that could arise is which layer should be used?

The last layer contains the most task specific information but it may over optimize for next token prediction at the expense of general semantic content. The second to last layer is a common heuristic that often outperforms the last layer for semantic similarity tasks. An average across all layers captures information at all levels of abstraction can be considered robust yet expensive. Middle layers often encode the richest structural information for syntactic tasks.

For dedicated embedding models (text-embedding-3-large, E5, BGE), this question is moot, these models are specifically trained to produce useful representations at the last layer using contrastive objectives rather than next token prediction. For retrieval tasks these models should be used instead of the generative models.

4.6 Embedding models vs. generation models

Generative models are trained to predict the next token although their internal hidden states can be extracted and used for retrieval, this is not what they are optimized for. On the other hand dedicated embedding models form a separate

class. They are trained with different objectives and are designed specifically to produce useful vector representations of text.

4.6.1 Contrastive training

Contrastive training is the dominant training approach for embedding models. The model is trained in a way that semantically similar texts have nearby embeddings and semantically different texts have distant embeddings.

The InfoNCE loss, used by models like E5, GTE, and many others:

$$\mathcal{L} = -\log\left(\frac{\exp\left(\text{sim}(q, k^+)/\tau\right)}{\sum_i \exp\left(\text{sim}(q, k_i)/\tau\right)}\right)$$

Where q is the query embedding, k^+ is the positive (semantically similar) embedding, k_i are all keys including negatives, sim is cosine similarity, and τ is a temperature parameter.

The model learns to pull positive pairs together and push negative pairs apart. Training data consists of (query, positive document) pairs which is derived from naturally occurring sources: question-answer pairs, search query logs, natural language inference datasets.

4.6.2 Hard negatives

The quality of negative examples is critical. Random negatives, which mostly means arbitrary documents from the corpus, are too easy. The model learns to distinguish clearly unrelated texts without developing fine-grained discrimination.

Hard negatives are documents that are topically related but semantically different from the positive:

```
Query:         "What is the capital of France?"
Positive:      "Paris is the capital and most populous city of France."
Easy negative: "The mitochondria is the powerhouse of the cell."
Hard negative: "France is a country in Western Europe with Paris as a major city."
               (related topic, but does not directly answer the question)
```

Mining hard negatives is itself a retrieval problem. Modern embedding model training pipelines use a previous model version or a BM25 index to find hard negatives and train a better model on those negatives, and iterate.

4.6.3 Asymmetric encoding

Many retrieval tasks are asymmetric which means that the query and the document are different in length, style, and information density. A question is short and underspecified yet the document is long and detailed.

Some embedding models handle this by encoding queries and documents with different instruction prefixes:

```
Query encoding:    embed("Represent this query for retrieval: " + query)
Document encoding: embed("Represent this document for retrieval: " + document)
```

The prefix signals to the model what role the text plays, allowing it to produce better representations for each. Models like E5 and Instructor were designed around this approach.

4.6.4 Embedding model families

Model	Dimensions	Context	Notes
text-embedding-3-small	1536	8,191 tokens	OpenAI; fast and cheap
text-embedding-3-large	3072	8,191 tokens	OpenAI; best quality in family

Model	Dimensions	Context	Notes
text-embedding-ada-002	1536	8,191 tokens	OpenAI legacy; widely deployed
E5-large-v2	1024	512 tokens	Open-source; strong performance
BGE-large-en-v1.5	1024	512 tokens	BAAI; strong on MTEB
Nomic-embed-text-v1	768	8,192 tokens	Open-source; long context
voyage-large-2	1536	16,000 tokens	Voyage AI; strong retrieval

The MTEB (Massive Text Embedding Benchmark) leaderboard is the standard reference for comparing embedding models across retrieval, classification, clustering, and semantic similarity tasks.

4.7 Similarity metrics

Embeddings are only useful if you can compare them so the choice of metric matters.

4.7.1 Cosine similarity

The standard metric for text embedding comparison:

`cosine_sim(a, b) = (a · b) / (||a|| × ||b||)`

Cosine similarity measures the angle between two vectors, ignoring magnitude. Values range from -1 (opposite directions) to 1 (identical direction). The appeal for this metric is the invariance to magnitude. A document and its summary should be similar even if the document produces a larger magnitude vector due to greater information density.

For normalized embeddings (unit vectors), cosine similarity equals the dot product: `cosine_sim(a, b) = a · b` when `||a|| = ||b|| = 1`. Most embedding APIs return normalized vectors, making the dot product the efficient choice at scale.

4.7.2 Dot product

Without normalization, the dot product combines angle and magnitude. For some retrieval tasks, magnitude carries useful signal which means a more specific or central document might have higher magnitude and it makes the raw dot product a better metric than cosine similarity.

Modern embedding models and vector databases support both; check the model documentation for which metric the model was trained to optimize.

4.7.3 Euclidean distance (L2)

This is a less common metric for text embeddings andor normalized vectors, L2 distance and cosine similarity are monotonically related:

$$\|a - b\|_2^2 = 2 - 2\cos(a, b)$$

L2 is mostly specific for applications (k-means clustering, certain index structures) where the metric properties are desirable.

4.8 Embedding dimensions and matryoshka representations

The dimensionality of an embedding determines its representational capacity and its cost. While a 3072-dimension embedding can represent finer grained semantic distinctions, it requires 12 KB per vector in float32 and more compute for similarity search. On the other hand a 768-dimension embedding has less capacity but 4× lower storage and faster search. For retrieval over millions to billions of vectors, the difference is significant.

4.8.1 Matryoshka Representation Learning (MRL)

This is a training technique that allows a single model to produce useful embeddings at multiple dimensions from the same forward pass. Instead of training separate models for different embedding sizes, MRL organizes the embedding space in a way that **useful representations exist at progressively larger prefixes of the same vector**.

MRL trains the model so that the first `d` dimensions of the full embedding are as useful as a `d`-dimensional embedding trained from scratch and simultaneously, for multiple values of `d`:

4.8.1.1 Core Idea

Let the model output a full embedding:

$$\text{embed} \in \mathbb{R}^{1536}$$

MRL trains the network such that **every prefix of the embedding** is independently meaningful:

```
- `embed[:64]` behaves like a strong 64-dim embedding
- `embed[:128]` behaves like a strong 128-dim embedding
- `embed[:256]` behaves like a strong 256-dim embedding
- ...
- `embed[:1536]` remains the highest-quality representation
```

This is analogous to **Russian matryoshka dolls**, where smaller representations are nested inside larger ones.

4.8.1.2 Training Objective

During training, the same embedding is evaluated at multiple truncation levels:

```
L = L(embed[:64]) + L(embed[:128]) + L(embed[:256]) + L(embed[:512]) + L(embed[:1536])
```

Each term computes the task loss (e.g., contrastive similarity loss) using only the first `d` dimensions.

The result is that you can use the full 1536-dimension embedding for high-accuracy search, or truncate to 256 dimensions for a 6× speedup at a small quality cost, using the same model. Dimensionality becomes a deployment parameter rather than a model choice.

OpenAI's text-embedding-3 series uses MRL, allowing callers to specify output dimensions at request time.

4.9 Embeddings in the generative inference pipeline

Within a generative LLM's inference pipeline, embeddings participate at two specific points.

4.9.1 Input: the embedding lookup

After tokenization, token IDs are converted to embeddings via the table lookup. Positional encodings are added (either as additive vectors or via RoPE modifications to attention), and result enters the first transformer block. This step is computationally trivial and takes microseconds even for long sequences.

4.9.2 Output: projection back to vocabulary

The final hidden states are projected to logit space via the transposed embedding table. This is a matrix multiplication of shape `[seq_len, d_model] @ [d_model, vocab_size]` which is the largest matrix multiply in a single forward pass for models with large vocabularies.

For a model with `d_model = 4096` and `vocab_size = 128,000`:

```
Output projection: [seq_len, 4,096] @ [4,096, 128,000]
                 = seq_len × 524,288,000 multiplications
```

For a sequence of 1,000 tokens, this is over 500 billion operations it is significant enough that some models apply the output projection only to the final token position during generation rather than to all positions.

4.10 Practical embedding operations

4.10.1 Embedding a batch of texts

```python
from openai import OpenAI

client = OpenAI()

texts = [
    "The capital of France is Paris.",
    "Paris is a city in Europe.",
    "The mitochondria is the powerhouse of the cell."
]

response = client.embeddings.create(
    input=texts,
    model="text-embedding-3-large",
    dimensions=1024  # MRL truncation
)

embeddings = [item.embedding for item in response.data]
# embeddings[0] and embeddings[1] will be much closer than embeddings[0] and [2]
```

4.10.2 Computing similarity

```python
import numpy as np

def cosine_similarity(a: list[float], b: list[float]) -> float:
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

sim_01 = cosine_similarity(embeddings[0], embeddings[1])
sim_02 = cosine_similarity(embeddings[0], embeddings[2])

print(f"Paris sentences:       {sim_01:.3f}")
print(f"Paris vs mitochondria: {sim_02:.3f}")
```

4.10.3 Semantic search over a corpus

```python
import numpy as np

corpus = ["doc 1 text...", "doc 2 text...", ...]
corpus_embeddings = np.array([
    client.embeddings.create(
        input=doc, model="text-embedding-3-large"
    ).data[0].embedding
    for doc in corpus
])  # shape: [n_docs, 1024]

query = "What is the capital of France?"
query_embedding = np.array(
    client.embeddings.create(
        input=query, model="text-embedding-3-large"
    ).data[0].embedding
)  # shape: [1024]

# Cosine similarity (assuming normalized embeddings from the API)
similarities = corpus_embeddings @ query_embedding  # shape: [n_docs]
top_k_indices = np.argsort(similarities)[::-1][:5]

for idx in top_k_indices:
    print(f"Score: {similarities[idx]:.3f} | {corpus[idx][:80]}")
```

For production retrieval over large corpora, replace the numpy dot product with a vector database (Pinecone, Weaviate, Qdrant, pgvector) that handles indexing and approximate nearest neighbor search at scale. The vector database is the subject of chapter “REFF”.

4.11 Key takeaways

- An embedding converts a discrete token ID into a continuous vector; geometric proximity in embedding space encodes semantic relatedness, emerging from the distributional statistics of training rather than any explicit rule
- The embedding table is a `[vocab_size, d_model]` matrix shared between input lookup and output projection (weight tying), enforcing geometric consistency between how tokens are represented and how they are predicted
- Static embeddings give every token a fixed vector regardless of context; contextual embeddings (hidden states at later layers) incorporate surrounding context through attention, the residual stream shows how representations accumulate refinements from syntactic to semantic to task-specific
- Dedicated embedding models are trained with contrastive objectives (InfoNCE loss on positive/negative pairs) rather than next-token prediction, producing representations optimized for similarity search
- Hard negatives, topically related but semantically wrong examples, are essential for training models that make fine-grained distinctions; mining them is itself a retrieval problem
- Cosine similarity is the standard metric for comparing embeddings; always use the metric the model was trained to optimize, using the wrong one degrades retrieval results in ways that are hard to diagnose
- Matryoshka Representation Learning allows a single model to produce useful embeddings at multiple dimensions, making dimensionality a deployment parameter
- The output projection (final hidden state to logits) is the most compute- intensive embedding operation in the generative forward pass; the input lookup is negligible by comparison

---

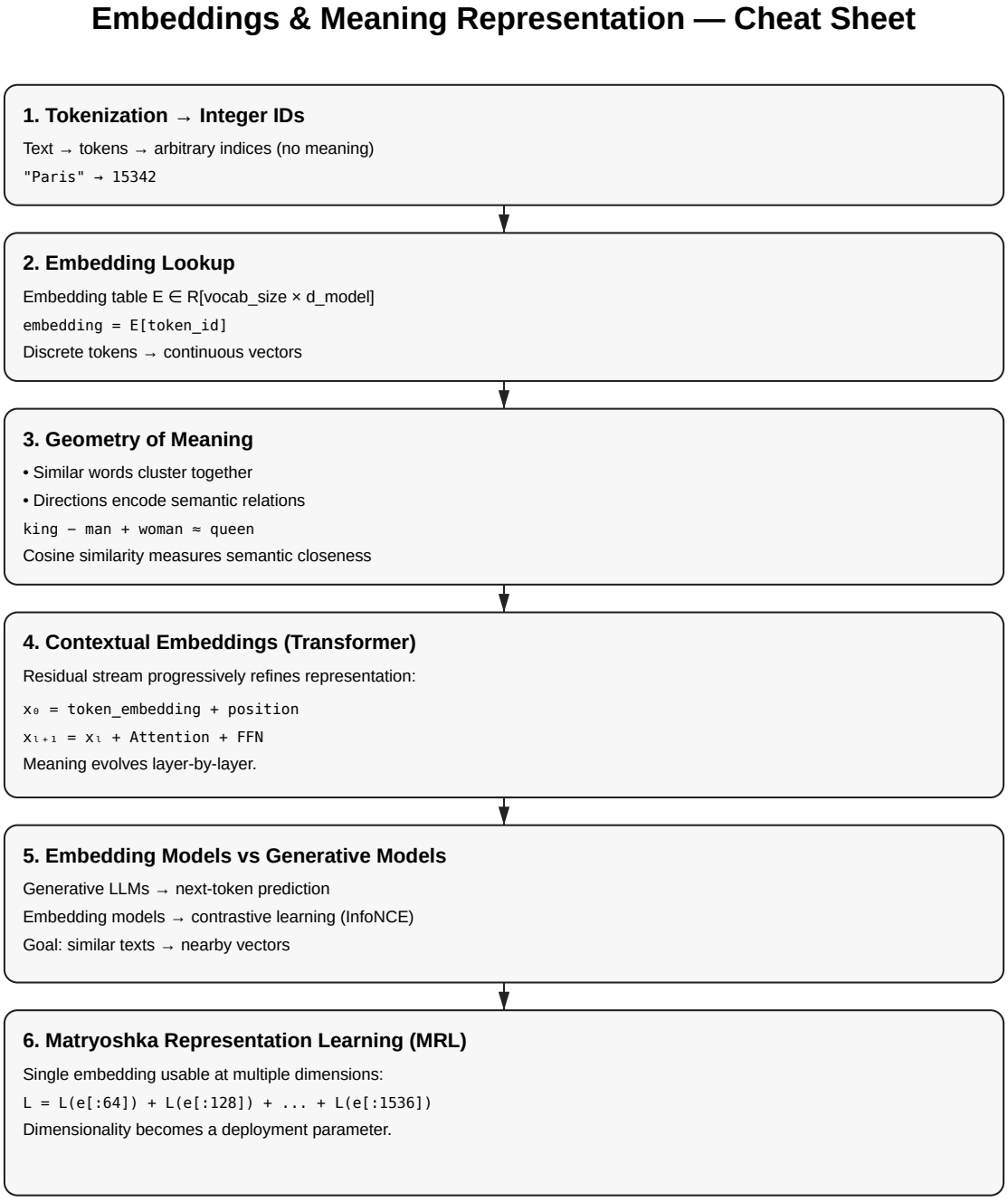


Figure 4.2: Cheat sheet.

4.12 Further reading

- Elhage et al. (2021). *A Mathematical Framework for Transformer Circuits.*: The residual stream model and how information flows through transformer layers.
- Wang et al. (2022). *Text Embeddings by Weakly-Supervised Contrastive Pre-training.*: The E5 paper; representative of modern embedding model training with hard negatives.
- Kusupati et al. (2022). *Matryoshka Representation Learning.*: MRL for flexible-dimension embeddings from a single model.
- Muennighoff et al. (2023). *MTEB: Massive Text Embedding Benchmark.*: The standard benchmark for comparing embedding models across tasks and domains.

Part III

The Labyrinth of Attention

Chapter 5

Transformer Architecture

The canonical question for this chapter: *Given a sequence of token embeddings, how does the transformer convert them into a prediction about what comes next and why is this architecture the one that won?*

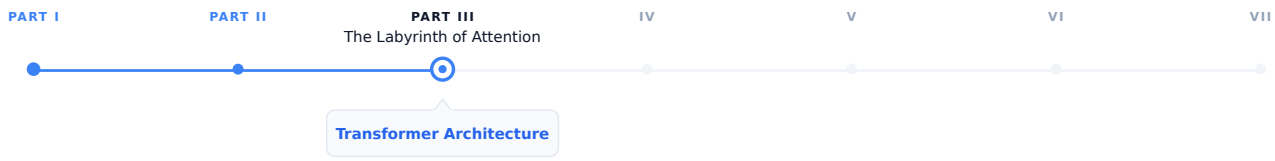


Figure 5.1: The journey through the Model Mind.

The prompt has been tokenized, the tokens have been converted to embedding vectors, and the request has arrived at the inference server. Now the model begins to think. This chapter covers the architecture that does the thinking (what it is, why it is built this way, and what each component actually does.)

5.1 Why the transformer won

Before 2017, sequence modeling was dominated by recurrent neural networks (LSTMs and GRUs). These architectures processed text one token at a time, left to right, maintaining a hidden state that carried information forward through the whole sequence. They worked, but they had two fundamental problems.

The first was the bottleneck problem, all the information about the entire input had to be compressed into a single fixed-size hidden state vector before being used to generate output. For the long sequences, early information was inevitably lost or distorted and the model had to decide what to remember and what to forget at every step.

The second was parallelization. Because each step depended on the previous step's hidden state, RNNs could not be parallelized across the sequence dimension. While the model GPU hardware are heavily tuned for parallelisim training these models were fundamentally bottlenecked by sequential computation.

The transformer, introduced in Vaswani et al. (2017), eliminated both problems with a single architectural decision: replace recurrence with attention. Every token attends directly to every other token in a single step, resulting in neither sequential dependency nor an information bottleneck, and the training can be fully parallelized across the sequence.

This is not a minor optimization but a different computational paradigm that happens to be perfectly matched to the hardware of the last decade.

5.2 The intuition before the math

Before working through the mechanics, it helps to have a mental model of what the transformer is doing at a high level.

Think of each token as a person sitting in a room with everyone else who has ever appeared in the same sequence. At each layer, every person can ask a question of every other person, receive weighted answers based on relevance, and update their understanding accordingly. After many rounds of this, each person's understanding has been shaped by the entire room.

The questions are learned queries, the relevance weighting is attention, the updating occurs in the residual stream, and the many rounds correspond to the layers.

5.3 The high-level forward pass

A transformer language model takes a sequence of token IDs as input and produces a probability distribution over the vocabulary as output in other word a prediction of what token comes next.

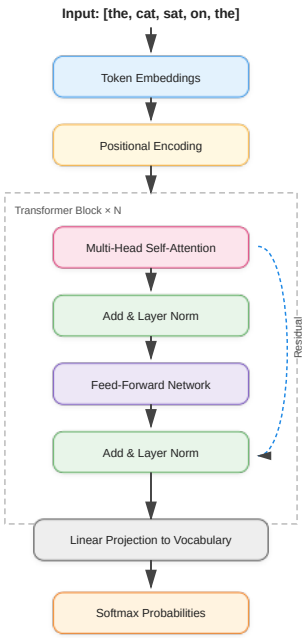


Figure 5.2: Transformer block.

Each component serves a specific function. In the following sections we work through them in order.

5.4 Token embeddings and positional encoding

5.4.1 Token embeddings

The first operation maps integer token IDs to continuous vectors using an embedding table, which is a matrix with shape `[vocab_size, d_model]` and Each row is a learned vector for that token. Looking up token ID 42 retrieves row 42 of the embedding table.

For GPT-3, `d_model = 12,288` and the vocabulary has roughly 50,000 tokens. The embedding table therefore has about 600 million parameters which is a significant fraction of the total model. These embeddings are learned end-to-end during training; the model discovers what representation of each token is most useful for predicting the next one.

The embedding table is shared with the output layer. The same matrix used to look up input embeddings is used (transposed) to project the final hidden state back to vocabulary space. This weight tying reduces parameters and improves performance by enforcing geometric consistency between how tokens are represented and how they are predicted.

5.4.2 Positional encoding

Attention, as we will see, is permutation-invariant which means if you shuffle the tokens in the input, attention produces the same output for each token regardless of order. This is a problem: `"the cat sat"` and `"sat cat the"` would be processed identically. Positional encodings inject order information into the representations by adding a position-dependent signal to each token embedding.

5.4.3 Sinusoidal encoding (the original transformer)

It uses a deterministic functions:

```
PE(pos, 2i)   = sin(pos / 10000^(2i / d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))
```

Different dimensions oscillate at different frequencies that produces a unique fingerprint for each position. These encodings are fixed and generalizes to sequence lengths not seen during training.

5.4.4 Learned positional embeddings**

they replace the formula with a second learned matrix of shape `[max_sequence_length, d_model]`, each position gets its own trained vector, updated by gradient descent.

5.4.5 Rotary Positional Embeddings (RoPE)

They are widely used in modern transformer architectures such as LLaMA, PaLM, and many recent open-weight large language models. Unlike absolute positional encodings, which are added directly to token embeddings before the attention mechanism, RoPE injects positional information directly into the self-attention computation itself.

The core idea behind RoPE is to encode position by applying a rotation to the query and key vectors in each attention head. For each token position, the representation is split into pairs of dimensions, and each pair is rotated by an angle that depends on the token's absolute position. This rotation is structured so that when computing the dot product between a query at position `m` and a key at position `n`, the result depends on the relative offset `(m - n)` rather than their absolute positions.

More formally, RoPE treats each pair of dimensions in a query or key vector as a 2D plane and applies a rotation matrix whose angle increases linearly with position. For a given position `p`, each 2D subspace is rotated by:

$$\theta_p = p \cdot \omega_i$$

where ω_i is a frequency term that depends on the embedding dimension index. This design closely relates to sinusoidal encodings, but differs fundamentally in that the positional information is applied as a transformation of the query and key vectors rather than an additive signal.

In practice, each pair of dimensions is rotated using sine and cosine functions:

$$\begin{pmatrix} x_1' \\ x_2' \end{pmatrix} = \begin{pmatrix} \cos(\theta_p) & -\sin(\theta_p) \\ \sin(\theta_p) & \cos(\theta_p) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

This rotation preserves vector magnitude while changing orientation, embedding positional structure geometrically into the representation space.

A key consequence of this formulation is that the dot product between a query at position `m` and a key at position `n` naturally depends on the relative distance `(m - n)`. This emerges from trigonometric identities that cause the absolute positional components to cancel out during the inner product. As a result, RoPE encodes relative position information implicitly without explicitly computing pairwise distances.

This property provides several important benefits:

- Firstly, it captures relative position information directly, which is often more meaningful for language modeling than absolute position. Language understanding typically depends on relationships between tokens rather than their absolute locations in a sequence.
- Secondly, RoPE improves extrapolation to longer sequences. Because the rotation is a smooth, continuous function of position, it generalizes beyond the training context window more gracefully than learned absolute embeddings. However, performance can still degrade for very long contexts if not trained appropriately.
- Thirdly, RoPE introduces no additional learned parameters for positional encoding. The positional structure is fully deterministic, making it parameter-efficient and reducing the risk of overfitting positional patterns.

In multi-head attention, each head applies the same rotational mechanism but operates in different learned subspaces. This allows different heads to specialize in different positional behaviors. Some heads naturally focus on local dependencies, while others capture long-range relationships due to accumulated phase differences.

5.4.6 ALiBi (Attention with Linear Biases)

It takes a very different approach to positional information. Instead of explicitly encoding positions into token representations, ALiBi modifies the attention mechanism itself by adding a fixed bias to attention scores based on the distance between tokens.

In ALiBi, the attention score between a query at position `m` and a key at position `n` is penalized linearly according to their distance `|m - n|`. This means that tokens that are farther apart receive a progressively stronger negative bias, encouraging the model to focus more on nearby context while still allowing access to long-range dependencies when needed.

A key advantage of ALiBi is that it requires no positional embeddings at all—neither learned nor fixed sinusoidal ones. Instead, positional information is entirely encoded through the attention bias structure. This simplicity makes ALiBi highly efficient and robust, particularly for long-context extrapolation. It has been used in models such as MPT and has shown strong performance in settings where generalization to longer sequences is important.

For a query at position m and a key at position n, the attention score is adjusted as:

$$\text{score}(m, n) = q_m \cdot k_n - \alpha_h \cdot |m - n|$$

Here: - $q_m \cdot k_n$ is the standard dot-product attention score - $| m - n |$ is the absolute distance between tokens - α_h is a head-specific slope (each attention head gets a different decay rate)

One of the key design choices in ALiBi is that different attention heads use different slopes. Some heads have a steep penalty (they focus very locally), while others have a shallow penalty (they can attend further away). This creates a

natural diversity of attention ranges without explicitly designing multiple mechanisms. The slopes are not learned, Instead, they are fixed using a deterministic scheme that assigns geometrically spaced values. With this method it avoids additional parameters and ensures predictable extrapolation behavior.

5.5 Self-attention: the core mechanism

Self-attention is an operation that allows every token to gather information from every other token in the sequence. It is the defining innovation of the transformer.

5.5.1 The intuition

Consider: `"The cat slept on the bed because it was dark."`

What does `"it"` refer to? For a human, it's clear that `"it"` refers to the environment, not `"the cat"` or `"the bed"`. A model needs to make this connection to process the sentence correctly. Self-attention allows the token `"it"` to look at every other token, assign an attention weight to each one (higher weight to the context suggesting darkness, lower weight to unrelated tokens like `"bed"`), and blend their representations into its own. After attention, the representation of `"it"` carries information about the surrounding environment being dark.

5.5.2 Queries, keys, and values

Self-attention uses three learned linear projections of each token's embedding: Query (Q), Key (K), and Value (V).

For a sequence of `n` tokens with embedding dimension `d_model`:

$$
\begin{aligned}
Q &= XW_Q \qquad shape : [n, d_k] \\
K &= XW_K \qquad shape : [n, d_k] \\
V &= XW_V \qquad shape : [n, d_v]
\end{aligned}
$$

Where `X` is the matrix of token embeddings and W_Q, W_K, W_V are learned weight matrices.

The attention scores are computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Breaking this down:

1. QK^T is dot product between every query and every key. The shape `[n, n]` and an entry `[i, j]` measures how much token `i` should attend to token `j`.
2. $\sqrt{d_k}$ scales the dot products by the square root of the key dimension. Without this, dot products in high dimensions become very large and pushes the softmax into regions with near-zero gradients and destabilizing training.
3. $softmax()$ converts scores to probabilities. Each row sums to 1. This is the attention weight matrix.
4. $\times V$ is the weighted sum of value vectors. Each token's output is a blend of all value vectors, weighted by how much it attends to each position.

The result is a new representation for each token that has been informed by the entire sequence. The attention chapter REFF goes deeper into the computational mechanics and what attention heads actually compute.

5.5.3 The causal mask

For language model training, the model must not attend to future tokens. When predicting the token at position `i`, only positions `0` through `i-1` should be visible, otherwise the model is simply reading the answer.

This is enforced by a causal mask: before softmax, a large negative value (effectively `-∞`) is added to attention scores where `j > i`. After softmax, these positions have attention weight 0.

```
Tokens:
Position:   0     1     2     3
          the   cat   sat    on

Causal Attention Matrix (upper triangle masked)

                Key tokens →
              the   cat   sat    on
Query ↓
the           1.0   0.0   0.0   0.0
cat           0.3   0.7   0.0   0.0
sat           0.1   0.5   0.4   0.0
on            0.2   0.1   0.3   0.4
```

Each token attends to itself and all previous tokens and it will never see the future tokens. This masking enables a decoder-only Transformer to function as an autoregressive language model. It allows the model to be trained on the full sequence in a single forward pass, while ensuring that each position can attend only to preceding tokens.

5.5.3.1 Bidirectional attention (encoder models)

Used in encoder-only models such as BERT, RoBERTa, and most embedding models.

Here, **no causal mask is applied**:

- each token can attend to all other tokens
- both left and right context are visible

This enables richer representations because every token is contextualized by the full sequence.

However, it breaks autoregressive generation:

> if a model can see future tokens, it cannot learn to predict them

Instead, encoder models are trained with masking objectives (e.g., masked language modeling), where some tokens are hidden and must be reconstructed.

5.5.3.2 Prefix masking (prefix-LM / partial bidirectionality)

A hybrid scheme used in some long-context and instruction models.

The sequence is split into two parts:

- **prefix (context)**: fully visible to itself and often to the entire sequence
- **generation region (suffix)**: causal masking applies

This allows:

- bidirectional understanding of the prompt/context
- autoregressive generation for outputs

It is particularly useful for: - long document conditioning - retrieval-augmented generation - structured prompt formats

---

5.5.3.3 Encoder–decoder (cross-attention masking)

The encoder–decoder transformer is designed for tasks where an input sequence must be *understood* before a new sequence is generated. Models such as **T5** and **BART** follow this structure.

The architecture separates the model into two cooperating systems:

- The **encoder** reads the entire input at once using bidirectional self-attention. Every token can attend to every other token, allowing the model to construct a contextual representation which is often described as a *memory* of the input.
- The **decoder** then generates the output sequence step by step. Unlike the encoder, its self-attention is **causally masked**, meaning each token can only attend to previously generated tokens. This preserves autoregressive generation.

  A second attention mechanism (**cross-attention**) connects the two halves of the model. Here, the decoder forms queries while the encoder provides keys and values, allowing generation to remain grounded in the encoded input representation.

This separation of *understanding* and *generation* makes the encoder–decoder architecture especially effective for transformation tasks such as translation, summarization, and structured rewriting. This creates a structured information flow:

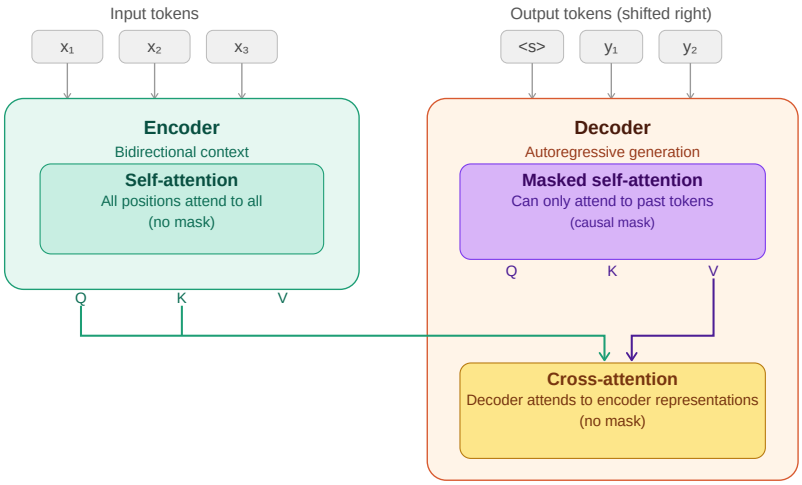


Figure 5.3: encoder-decoder information flow

5.6 Multi-head attention

A single attention operation gives each token one way of relating to every other token nevertheless tokens simultaneously relate to each other in multiple ways.

In `"John gave Mary the book"`, `John` relates to `gave` syntactically (as subject), to `Mary` semantically (as giver to recipient), and to `the book` as the transferred object. A single attention head cannot capture all of these simultaneously and it results in production of a single weighted blend over the sequence.

Multi-head attention runs multiple attention operations in parallel, each with its own learned Q, K, V projections. With this format each head can specialize in a different type of relationship. Finally, the outputs of all heads are concatenated and projected through a learned output matrix W_O back to d_{model} dimensions.

For GPT-3: `d_model = 12,288`, `h = 96` heads, `d_k = 128` per head. The 96 heads run in parallel, each attending to the sequence from a different perspective.

5.7 The feed-forward network

After the attention sublayer, each token passes through a position-wise feed-forward network (FFN) independently:

$$FFN(x) = GeLU(xW_1 + b_1)W_2 + b_2$$

The intermediate dimension is typically 4× the model dimension: for `d_model = 1,024`, the FFN expands to 4,096 dimensions, applies the activation, then projects back to 1,024.

Crucially, the FFN operates on each token independently and identically. The attention layer mixes information across tokens; the FFN processes each token in isolation. Together they form a complementary pair: attention gathers the context and FFN processes it.

5.8 Residual connections and layer normalization

5.8.1 Residual connections

```
x = x + Attention(LayerNorm(x))
x = x + FFN(LayerNorm(x))
```

The output of each sublayer is added back to its input which gives the gradients a direct path backward through the network, enabling training of very deep models. Without residual connections, training a 96-layer network is extremely difficult due to gradients vanishing or exploding before reaching the earlier layers.

Residual connections make the default behavior of each sublayer simply pass its input through unchanged. As a result, sublayers only need to learn small refinements on top of the identity function, which is a much easier optimization problem than learning the full transformation from scratch.

5.8.2 Layer normalization

Layer normalization is used to stabilize the activations of a transformer by normalizing each token's representation across its feature dimensions. Unlike normalization methods that have been used in convolutional networks, it operates independently on each token rather than across the batch or sequence.

Formally, for a hidden state vector $x \in \mathbb{R}^d$:

$$\text{LayerNorm}(x) = \frac{x - \text{mean}(x)}{\sqrt{\text{var}(x) + \varepsilon}} \cdot \gamma + \beta$$

where: - `mean(x)` and `var(x)` are computed over the feature dimension - ε is a small constant for numerical stability - γ and β are learned scale and shift parameters and allow the model to restore or reshape the normalized representation if needed.

This normalization keeps activations in a controlled range, which prevents instability as signals propagate through many stacked layers. Without it, deep transformers become difficult to train due to exploding or vanishing activations in the residual stream.

5.8.2.1 Why not Batch Normalization?

Batch Normalization is rarely used in transformer language models for a fundamental reason: it depends on batch-level statistics.

In BatchNorm, normalization is computed across examples in a minibatch. This creates several problems for language modeling:

- **Inference mismatch**: at inference time, batch sizes are often 1 (especially in autoregressive decoding), making batch statistics unreliable or unavailable
- **Sequence variability**: token sequences have variable lengths, making batch-wise statistics inconsistent across steps
- **Autoregressive constraint**: generation happens one token at a time, so there is no meaningful "batch" context during decoding

Because of these issues, BatchNorm introduces instability and is incompatible with the sequential nature of language model inference. LayerNorm avoids this entirely by normalizing within each token independently.

5.8.2.2 Pre-LN vs Post-LN

The original transformer used **Post-LN**, where normalization is applied after the residual connection:

```
x = x + Sublayer(x)
x = LayerNorm(x)
```

Modern large language models instead use **Pre-LN**, where normalization happens before the sublayer:

```
x = x + Sublayer(LayerNorm(x))
```

Pre-LN is now the standard architecture because it significantly improves optimization stability in deep networks. While it can sometimes slightly reduce raw performance compared to Post-LN in certain setups, it provides much more reliable training at scale. It ensures that each sublayer receives well-conditioned inputs while maintaining a clean gradient flow through the residual stream. This stability benefit becomes increasingly important as models grow to hundreds of layers, where training dynamics would become difficult to control.

5.8.2.3 RMSNorm (modern simplification)

Some recent architectures replace LayerNorm with **RMSNorm (Root Mean Square Normalization)**, used in models such as LLaMA.

RMSNorm simplifies LayerNorm by removing the mean-centering step:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}(x^2) + \varepsilon}} \times \gamma$$

Key differences: - no subtraction of the mean - normalization is based only on vector magnitude - slightly cheaper computationally - empirically comparable or better performance in large-scale models

Despite its simplicity, RMSNorm preserves the key property needed for deep transformers: controlling the scale of activations in the residual stream.

5.9 Stacking transformer blocks

A single transformer block gives the model one round of attention and one round of FFN computation while modern LLMs stack many of them:

Model	Layers	d_model	Heads	Parameters
GPT-2 Small	12	768	12	117M
GPT-2 XL	48	1,600	25	1.5B
GPT-3	96	12,288	96	175B
LLaMA 2 70B	80	8,192	64	70B
LLaMA 3 405B	126	16,384	128	405B

Each layer refines the representation produced by the previous layer. Early layers tend to encode syntactic features while middle layers encode semantic relationships and final layers encode task-specific features relevant to prediction. This hierarchy

5.10 Architecture variants

The standard transformer has been modified in various ways by recent models.

5.10.1 Grouped Query Attention (GQA)

Standard multi-head attention uses separate Q, K, and V projections for each head. In contrast, Multi-Query Attention (MQA) shares a single set of K and V projections across all heads, while keeping separate Q projections per head. Grouped Query Attention (GQA) works in a middle ground: K and V are shared across groups of heads, while each head (or each group of heads) retains its own Q projections.

LLaMA 2 70B, LLaMA 3, and Mistral use GQA. The motivation is inference efficiency: KV cache memory (chapter REFF) scales with the number of K, V heads and fewer KV heads is equal to smaller cache, resulting in longer sequences and larger batches at the same memory budget.

5.10.2 Sliding window attention

Used by Mistral 7B. Instead of attending to all previous tokens, each token attends only to a local window of the most recent `w` tokens. For long sequences this reduces attention complexity from $O(n^2)$ to $O(n \cdot w)$. Information from beyond the window propagates through the layered structure of the network and multiple layers of local attention can capture long-range dependencies indirectly.

5.10.3 Activation functions (ReLU → Tanh → Sigmoid → GELU → Swish → GLU → SwiGLU)

The feed-forward network (FFN) in transformers is where most of the non-linear feature transformation happens and over time, activation functions have evolved from simple thresholding operations to smooth functions and finally to gated multiplicative mechanisms.

5.10.3.1 ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

ReLU is the simplest activation function, zeroing out all negative values while leaving positive values unchanged, which makes it computationally efficient and encourages sparse activations. However, this abrupt cutoff at zero can discard useful information, making it a useful starting point but not ideal for training deep transformer models.

5.10.3.2 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The sigmoid function was one of the earliest nonlinearities used in neural networks, mapping inputs into the range ((0, 1)) in a smooth and differentiable way that can be interpreted as a probability or gating mechanism; intuitively, it acts like a soft switch that controls how much of a signal passes through, from fully blocked at 0 to fully allowed at 1.

5.10.3.3 Tanh (Hyperbolic Tangent)

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Tanh was widely used in early neural networks and recurrent models before transformers, producing outputs in the range ([-1, 1]) with a smooth, zero-centered, symmetric shape that preserves both positive and negative signals; unlike ReLU, it does not discard negative information but instead compresses inputs into a bounded range, allowing both excitatory and inhibitory effects to be represented.

5.10.3.4 Why sigmoid and tanh are rarely used in transformers

Both sigmoid and tanh share a key limitation in that they saturate for large absolute values of (x), causing gradients to vanish in those regions and making optimization difficult in deep architectures such as transformers. Despite this, sigmoid remains important in practice, especially in gating mechanisms like GLU and LSTM-style components, as well as in attention-related soft selection functions where smooth probabilistic control is useful.

5.10.3.5 GELU (Gaussian Error Linear Unit)

$$\text{GELU}(x) = x \cdot \Phi(x)$$

GELU replaces hard thresholding with a smooth probabilistic gating function.

Intuition: Instead of “drop or keep”, it asks: > “how likely is this feature to be useful?”

This smoothness improves optimization stability and is used in BERT and early GPT-style models.

5.10.3.6 Swish

Swish is a simpler, learned-smooth alternative to GELU:

$$\text{Swish}(x) = x \cdot \sigma(x)$$

where ((x)) is the sigmoid function.

Swish is a smooth, fully differentiable activation similar in spirit to GELU, but simpler in form and notable for being non-monotonic, meaning it can both slightly suppress or enhance features depending on the input value. Intuitively,

rather than using hard gating like ReLU or probabilistic gating like GELU, Swish applies a self-gated mechanism where each feature determines how much of itself to pass forward, which is why it often matches or slightly improves GELU's performance while remaining conceptually simpler.

5.10.3.7 GLU (Gated Linear Unit)

instead of simply modifying the activation function, GLU introduces explicit gating between two learned projections, where the input is split into a content stream and a gate stream:

$$\mathrm{GLU}(x) = (xW_a) \odot \sigma(xW_b)$$

This design separates "what to pass" from "how much to pass," enabling multiplicative feature control that is substantially more expressive than standard pointwise activations.

5.10.3.8 SwiGLU (Swish-Gated Linear Unit)

Modern LLMs (LLaMA, PaLM, Mistral) use SwiGLU, which replaces the sigmoid gate with Swish:

$$\mathrm{FFN}(x) = (\mathrm{Swish}(xW_1) \odot (xW_2))W_3$$

SwiGLU combines the smooth, self-gated nonlinearity of Swish with the multiplicative two-stream structure of GLU, so rather than simply applying a nonlinearity to a feature, it learns how two projected feature streams interact through multiplicative gating. This shift from pointwise transformation to learned feature interaction makes it significantly more expressive than standard activation functions.

5.10.3.9 Visual intuition: from tanh to GELU-like behavior

A useful way to build intuition for modern activation functions is to see how they evolve from simple bounded nonlinearities into smooth, ReLU-like gating mechanisms that combine stability with adaptive feature control.

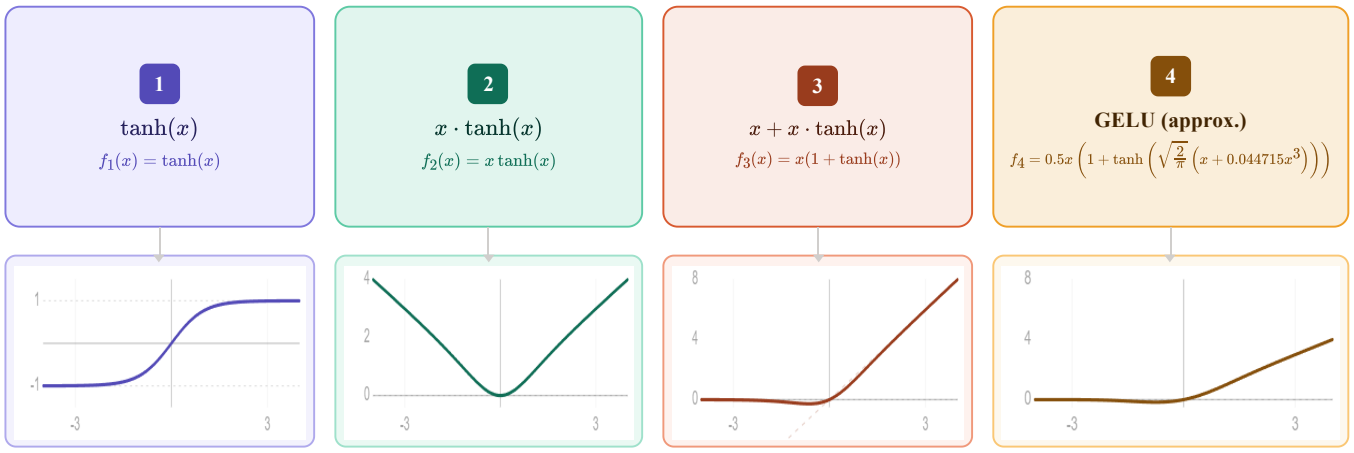


Figure 5.4: Tanh to GELU.

5.10.3.10 Visual intuition: from sigmoid to Swish-like behavior

A useful way to build intuition for modern activation functions is to see how they evolve from simple probabilistic gating functions into smooth, self-modulated nonlinearities. Sigmoid starts as a soft on–off switch, squeezing inputs

into ((0,1)), while Swish generalizes this idea by letting the input itself be modulated by a smooth gate, producing a function that behaves like a continuous, input-dependent scaling mechanism rather than a fixed probabilistic filter.

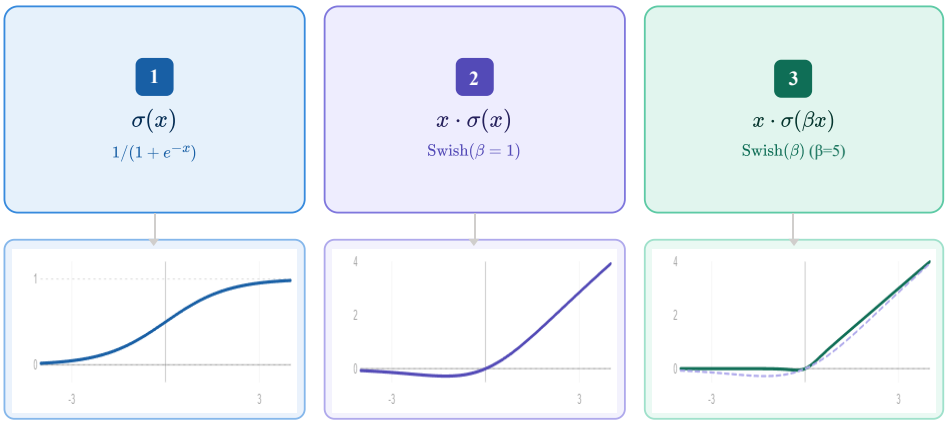


Figure 5.5: Sigmoid to swish.

5.10.4 Mixture of Experts (MoE)

Mixture of Experts (MoE) is used by Mixtral, reportedly GPT-4, it is a sparse Transformer architecture that increases model capacity without proportional increase in computation. It replaces the standard Feed-Forward Network (FFN) with multiple parallel FFNs (experts) and uses a learned routing mechanism to activate only a small subset per token.

Formulation

Given a set of experts $\text{FFN}_1, \ldots, \text{FFN}_N$, a gating function computes routing scores:

$$g(x) = \text{softmax}(W_r x)$$

MoE enables sparse computation, where only a fraction of parameters are active per token. This allows model capacity to scale significantly while keeping per-token compute similar to dense models. For example, a model with 8 experts of 7B parameters (~47B total) may activate only 1–2 experts per token, resulting in compute comparable to a much smaller dense model.

5.11 The output layer

After the final transformer block, the hidden state at the last token position is projected back to vocabulary size:

$$\text{logits} = h_{\text{final}} W_{\text{embed}}^{\top} \quad \text{shape: [vocab\_size]}$$

This produces one unnormalized score per vocabulary token. Softmax converts logits to probabilities:

$$P(\text{token}_i) = \frac{\exp(\text{logits}_i)}{\sum_j \exp(\text{logits}_j)}$$

The training objective (cross-entropy loss) maximizes the probability assigned to the actual next tokenand that is the only thing the model is explicitly trained to do. Chapter REFF covers training objectives in detail.

5.12 What the model actually learns

The transformer is trained to predict the next token. That is all. There is no explicit objective to learn grammar, facts, reasoning, or common sense, yet models trained at scale acquire all of these.

Why? Because predicting the next token well requires understanding everything that determines what comes next. Next-token prediction is a seems to be a simple objective that implicitly rewards a comprehensive world model. In order to predict `"Paris"` after `"The capital of France is"`, the model must have encoded the fact that Paris is the capital of France or to predict grammatically correct continuations, it must have internalized syntactic rules.

5.13 Key takeaways

- The transformer replaced recurrence with attention, enabling full parallelism during training and direct token-to-token interaction at any distance
- Token embeddings map integer IDs to continuous vectors; positional encodings inject order information; RoPE encodes relative rather than absolute position
- Self-attention allows every token to gather information from every other token via query-key dot products, scaled and softmaxed into attention weights over value vectors
- The causal mask enforces that tokens cannot attend to future positions, enabling autoregressive training on the full sequence in a single forward pass
- Multi-head attention runs multiple attention operations in parallel, each specializing in a different type of relationship.
- The feed-forward network processes each token independently after attention, acting as associative memory for facts learned during training
- Residual connections and layer normalization stabilize training of very deep networks; Pre-LN and RMSNorm are the current standard
- Modern variants (RoPE, GQA, SwiGLU, MoE) improve efficiency and performance without changing the fundamental architecture
- The next-token prediction objective implicitly rewards a comprehensive world model because everything that makes text predictable (grammar, facts, reasoning) improves prediction quality

---

5.14 Further reading

- Vaswani et al. (2017). *Attention Is All You Need.*: The original transformer paper.
- Alammar, J. (2018). *The Illustrated Transformer.*: The clearest visual explanation of the architecture available. Essential companion to this chapter.

Transformer Architecture — Cheat Sheet

Why Transformers Won

- RNNs: sequential → no parallelism
- Long-range info bottleneck in hidden state
- Transformers: attention → full parallelism + direct token interaction

High-Level Pipeline

Tokens → Embeddings → Positional Encoding → Transformer Blocks → Logits → Softmax

`P(next token | context)`

Embeddings & Position

Token embedding: lookup table (vocab → vectors)

Weight tying: input embedding = output projection

Positional encodings:

- Sinusoidal (fixed, original transformer)
- Learned embeddings
- RoPE: rotates Q/K → relative position

Self-Attention

`Q = XWq, K = XWk, V = XWv`

`Attention = softmax(QKᵀ / √d) V`

Each token attends to all previous tokens

Creates contextualized representation

Attention Masks

- Causal (decoder): only past tokens visible
- Bidirectional (encoder): full sequence visible (BERT)
- Encoder-decoder: cross-attention between memory + generation
- Prefix LM: prefix bidirectional, suffix causal

Multi-Head Attention

Multiple parallel attention heads

Each head learns different relation type

Outputs concatenated + projected

`Specialization emerges from training`

Feed-Forward Network

`FFN(x) = W2 · activation(W1 x)`

Per-token independent transformation

Modern activations:

- GELU (default GPT/BERT)
- SwiGLU (modern LLaMA, Mistral)

Residual + Normalization

x = x + Sublayer(LayerNorm(x)) (Pre-LN)

Stabilizes deep training

LayerNorm → RMSNorm (modern models)

Pre-LN = stable gradients at scale

Modern Variants

- RoPE: relative position encoding
- GQA: shared K/V across heads
- MoE: sparse expert FFNs
- Sliding window attention (Mistral)

Output Layer

`logits = h Wᵀ → softmax → P(next token)`

Training objective: next-token prediction only

Figure 5.6: Cheat sheet.

Chapter 6

Attention Computation

The canonical question for this chapter: *When the model processes your prompt, how does each token decide what to look at and what does the hardware actually do to compute that?*

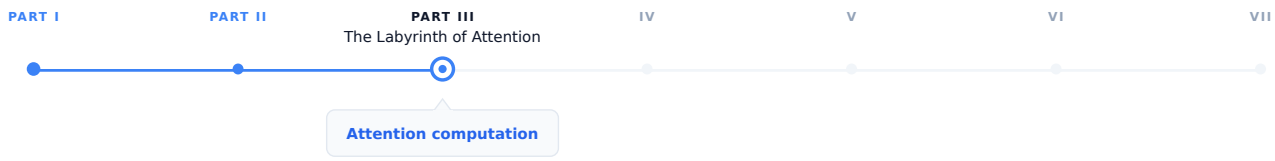


Figure 6.1: The journey through the Model Mind.

In the previous chapter we introduced attention conceptually: queries, keys, values, dot products, softmax, weighted sum. This chapter covers what attention computation actually looks like on real hardware, under the constraints of latency, memory bandwidth, and concurrent requests.

6.1 The quadratic problem

Standard self-attention has a fundamental scaling property where both computation and memory grow quadratically with sequence length. For a sequence of `n` tokens:

Attention score matrix:	$n \times n$
Memory for scores:	$n^2 \times 2$ bytes (BF16)
Compute for scores:	$n^2 \times d_k$ multiplications ($QK^\top$)
Compute for output:	$n^2 \times d_v$ multiplications (scores $\times V$)

At small sequence lengths this is manageable:

Sequence length	Score matrix	Memory
512 tokens	512 × 512	0.5 MB
2,048 tokens	2,048 × 2,048	8 MB
8,192 tokens	8,192 × 8,192	128 MB
32,768 tokens	32,768 × 32,768	2 GB

Sequence length	Score matrix	Memory
131,072 tokens	131,072 × 131,072	32 GB

At 128k tokens on the other hand which is currently supported by several models the attention score matrix alone would require 32 GB of GPU memory per layer per head which means a model with 96-layer and 96 heads would need $96 \times 96 \times 32$ GB = 295 TB and that is clearly impossible.

In reality the score matrix is never actually materializes in practice with the help of FlashAttention but the compute still scales quadratically. Doubling the sequence length quadruples the attention computation and it is central cost driver for long-context inference.

6.2 What the GPU actually does during attention

Understanding attention at the hardware level requires understanding two main bottlenecks in GPU computation, arithmetic throughput and memory bandwidth. **Arithmetic throughput** refers to how many floating-point operations per second the GPU can perform (an H100 delivers approximately 1,000 TFLOPS for BF16 matrix multiplications). **Memory bandwidth** refers to how fast data can be read/write from/to GPU high bandwidth memory (an H100 provides approximately 3.35 TB/s). The ratio between these, known as arithmetic intensity, determines whether a computation is compute-bound or memory-bandwidth-bound.

For matrix multiplication, the main operation in attention and FFNs, large matrices have high arithmetic intensity and are compute-bound, while small matrices or element-wise operations have low arithmetic intensity and are memory-bandwidth-bound.

During the decoding phase (processing one new token against the KV cache), attention is severely memory-bandwidth-bound. In this phase GPU must read the entire KV cache (potentially gigabytes of data) to compute attention scores for a single new token. The arithmetic work is tiny relative to the memory access while the GPU's massive arithmetic throughput sits idle, waiting for data. This is the reason why decoding throughput scales with memory bandwidth, not arithmetic throughput and why increasing sequence length primarily increases memory pressure and not the compute pressure, during generation.

6.3 Standard attention: the naive computation

Before FlashAttention, attention was computed in the obvious way:

```
# Standard attention (naive)
# Q: [seq_len, d_k]
# K: [seq_len, d_k]
# V: [seq_len, d_v]

scores = Q @ K.T                          # [seq_len, seq_len] - written to HBM
scores = scores / math.sqrt(d_k)          # scaling - read/write from/to HBM
scores = scores + causal_mask             # masking - read/write from/to HBM
scores = softmax(scores, dim=-1)          # softmax - read/write from/to HBM
output = scores @ V                       # [seq_len, d_v] - read/write from/to HBM
```

Each step read/write from/to HBM. The score matrix of shape `[seq_len, seq_len]` is materialized in HBM multiple times during scaling, masking, and softmax.

For a sequence of 8,192 tokens in BF16:

- Score matrix: 8,192 × 8,192 × 2 bytes = 128 MB

- Each read/write of the score matrix: 128 MB / 3.35 TB/s 38 microseconds
- Across multiple reads and writes: hundreds of microseconds in memory traffic alone

The actual arithmetic is fast, while the movement of data between compute units and HBM is the bottleneck, which making this a classic I/O-bound computation.

6.4 FlashAttention

FlashAttention (Dao et al., 2022) computes exact attention which is basically same numerical result as standard attention, while using dramatically less memory and running significantly faster. The trick is reorganizing the computation to minimize HBM traffic.

6.4.1 The key insight: tiling

FlashAttention divides the Q, K, and V matrices into tiles that fit in SRAM, the fast on-chip memory sitting directly next to the compute units, with much higher bandwidth than HBM but much smaller capacity:

```
SRAM bandwidth:  ~20 TB/s   (fast, ~50 MB on H100)
HBM bandwidth:   ~3.35 TB/s (slower, 80 GB on H100)
```

By loading tiles into SRAM and performing the full attention computation on those tiles before writing results back to HBM, FlashAttention reduces HBM reads and writes from $O(n^2)$ to $O(n)$ while the score matrix is never written to HBM at all.

6.4.2 Online softmax

The challenge: softmax requires knowing the maximum value across all scores in a row before normalizing while in the tiled computation, you process one tile at a time without seeing the full row. How do you compute correct softmax without materializing the full score matrix?

FlashAttention uses an online softmax algorithm that maintains running statistics (the running maximum and running sum of exponentials) and updates them as each tile is processed. After all tiles, the statistics are complete and the output is correctly normalized:

```
running_max = -infinity
running_sum = 0
running_output = zeros(d_v)

for each key/value tile (K_tile, V_tile):
    scores_tile = Q_row @ K_tile.T / sqrt(d_k)
    tile_max = max(scores_tile)

    # Update running statistics
    new_max = max(running_max, tile_max)
    running_sum = (running_sum * exp(running_max - new_max)
                   + sum(exp(scores_tile - new_max)))
    running_output = (running_output * exp(running_max - new_max)
                      + exp(scores_tile - new_max) @ V_tile)
    running_max = new_max

# Final normalization
output = running_output / running_sum
```

This procedure produces the identical result to naive attention while never storing more than one tile's worth of scores at a time.

6.4.3 FlashAttention in practice

FlashAttention is 2–4× faster than standard attention on typical sequence lengths, with the speedup increasing as sequences grow longer and more importantly, its memory usage is O(n) rather than O(n^2).

6.5 Multi-head attention at inference

Multi-head attention runs `h` independent attention operations in parallel. For a model with `h = 96` heads and `d_model = 12,288`:

$$d_k = d_v = \frac{d_{\text{model}}}{h} = 128 \quad \text{(per head)}$$

Projections:

$$Q : [n,\, 12288] \,@\, [12288,\, 96 \times 128] \;\rightarrow\; [n,\, 96,\, 128]$$
$$K : [n,\, 12288] \,@\, [12288,\, 96 \times 128] \;\rightarrow\; [n,\, 96,\, 128]$$
$$V : [n,\, 12288] \,@\, [12288,\, 96 \times 128] \;\rightarrow\; [n,\, 96,\, 128]$$

Per-head attention:

$$\text{scores} = \frac{Q_h K_h^\top}{\sqrt{128}} \;\rightarrow\; [n,\, n]$$

$$\text{output} = \text{softmax}(\text{scores})\, V_h \;\rightarrow\; [n,\, 128]$$

Concatenate: $[n,\, 96,\, 128] \;\rightarrow\; [n,\, 12288]$

Output proj: $[n,\, 12288] \,@\, [12288,\, 12288] \;\rightarrow\; [n,\, 12288]$

In practice, all 96 heads are computed in parallel by treating the head dimension as a batch dimension. The Q, K, V projections produce batched tensors processed simultaneously by the matrix multiplication kernel.

6.5.1 MHA vs. MQA vs. GQA

Multi-Head Attention (MHA) uses separate Q, K, and V projection matrices for each head. While this increases expressiveness, it is memory-intensive because each head requires its own stored K and V tensors in the KV cache.

Multi-Query Attention (MQA) (Shazeer, 2019): All attention heads share a single set of K and V projections, while Q remains separate per head. This significantly reduces KV cache usage by a factor of h, for example, a 96-head model achieves a ~96× smaller cache and in general, this comes with only a modest impact on output quality.

MHA: $Q \in \mathbb{R}^{n \times h \times d_k},\ K \in \mathbb{R}^{n \times h \times d_k},\ V \in \mathbb{R}^{n \times h \times d_v} \;\Rightarrow\; \text{KV cache} \propto 2 \cdot h \cdot d_k$

MQA: $Q \in \mathbb{R}^{n \times h \times d_k},\ K \in \mathbb{R}^{n \times 1 \times d_k},\ V \in \mathbb{R}^{n \times 1 \times d_v} \;\Rightarrow\; \text{KV cache} \propto 2 \cdot d_k$

Grouped-Query Attention (GQA) (Ainslie et al., 2023): the middle ground in which Heads are divided into groups; each group shares one K and V projection:

Component	Shape	Details
Q (Query)	$[n, 96, d_k]$	per head
K (Key)	$[n, 8, d_k]$	per group

Component	Shape	Details
V (Value)	$[n, 8, d_v]$	per group
KV Cache	$\propto 2 \times 8 \times d_k$	12× smaller than MHA

LLaMA 2 70B uses GQA with 8 KV heads and Mistral 7B uses GQA with 8 KV heads. Quality loss compared to MHA is negligible on standard benchmarks while the KV cache reduction is significant for serving.

6.6 The KV cache at inference

During decoding, the model generates one token at a time and at each step, this new token must attend to all previous tokens. Recomputing the K and V vectors for all previous tokens at every step would be extremely wasteful due to the fact that those tokens have not changed, so their projections have not changed either.

The KV cache stores K and V tensors for all previous tokens and reuses them at each generation step. Only the new token's Q, K, V vectors need to be computed fresh. This is covered in full detail in chapter REFF nevertheless the momory cost understanding will be discussed here

6.6.1 KV cache memory per token

For LLaMA 2 70B (L=80 layers, 8 KV heads, d_k=128, BF16):

Per token per layer: $2 \times 8 \times 128 \times 2$ bytes = 4 KB

Per token total: 4×80 layers = 320 KB

For a 4,096-token sequence: 320 KB × 4,096 **1.28 GB of KV cache** per sequence. At batch size 64 concurrent sequences: 64 × 1.28 GB = 82 GB which is more than the 80 GB on an H100. This is the reason KV cache management is the central serving constraint for long-context workloads.

6.6.2 Prefill vs. decode attention

During prefill (processing the input prompt), all tokens' K and V vectors are computed simultaneously and written to the KV cache it is a batch matrix multiplication, efficient and compute-bound. On the othe hand during decoding (generating output), each new token's K and V are computed and appended to the cache, then attention is computed between that single new token's Q and the full KV cacheand it is a vector-matrix multiply, memory-bandwidth-bound:

```
Prefill: [n_prompt, d_k] @ [d_k, n_prompt]      → large matmul, compute-bound
Decode:  [1, d_k] @ [d_k, n_prompt + n_gen]     → vector-matrix, bandwidth-bound
```

This asymmetry is why decoding is slower per token than prefill, and why disaggregated serving (separate hardware for prefill and decode phases) is considered as an active area of optimization.

6.7 Sparse and linear attention

The quadratic cost of standard attention has motivated significant research into more efficient alternatives yet None has been able to displace full attention in frontier models, but several are worth understanding.

6.7.1 Sparse attention

Instead of every token attending to every other token, sparse attention restricts each token to a subset of positions:

Local window attention: each token attends only to the surrounding `w` tokens which reduces the complexity to O(n × w). With this approach the information still can propagate across long distances through multiple layers of local attention.

Strided attention: alternating layers use different patterns in which some layers use local windows and some use strided patterns (attend every k steps) allowing attention to distant tokens. Used in Longformer and BigBird.

Sliding window with global tokens: certain tokens (typically the first ones) attend to all other tokens and are attended to by all others. These global tokens propagate long-range information while the rest of the sequence uses local attention.

The limitation: the sparsity pattern must be specified in advance. If it does not match the actual information dependencies in the data, quality degrades. Full attention is flexible that helps the model to learn any pattern which is why it has remained dominant.

6.7.2 Linear attention

The key idea behind **linear attention** is to reinterpret softmax attention as a kernel operation whose structure allows the order of computation to change.

Ignoring scaling and normalization constants for clarity, the softmax attention weight between tokens can be written as

$$A_{ij} = \exp(q_i^\top k_j).$$

The attention output for token i therefore becomes

$$\text{output}_i = \frac{\sum_j \exp(q_i^\top k_j)\, v_j}{\sum_j \exp(q_i^\top k_j)}.$$

This reveals that attention is fundamentally a **kernel similarity** $\kappa(q_i, k_j) = \exp(q_i^\top k_j)$ meaning attention computes a weighted kernel regression over values.

With linear attention we can assume that this exponential kernel can be approximated by an inner product in a feature space, $\exp(q^\top k) \approx \phi(q)^\top \phi(k)$, where the feature map $\phi : \mathbb{R}^d \to \mathbb{R}^{d_\phi}$ embeds queries and keys into a space where their similarity become linear and substituting this approximation into attention gives

$$\text{output}_i = \frac{\sum_j \phi(q_i)^\top \phi(k_j)\, v_j}{\sum_j \phi(q_i)^\top \phi(k_j)}.$$

Because $\phi(q_i)$ does not depend on j, it can be factored outside the sums:

$$\text{output}_i = \frac{\phi(q_i)^\top \sum_j \phi(k_j) v_j^\top}{\phi(q_i)^\top \sum_j \phi(k_j)}.$$

All interactions with the sequence are now contained in two global statistics,

$$S = \sum_j \phi(k_j) v_j^\top, \qquad z = \sum_j \phi(k_j),$$

so the output becomes

$$\text{output}_i = \frac{\phi(q_i)^\top S}{\phi(q_i)^\top z}.$$

Stacking all queries yields the linear attention form

$$\boxed{\text{Attn}(Q,K,V) = \phi(Q)(\phi(K)^\top V)} \quad \text{(up to normalization)}.$$

The crucial consequence is that computation can be reordered:

$$(QK^\top)V \;\longrightarrow\; \phi(Q)(\phi(K)^\top V),$$

allowing the key-value product to be computed once and reused for all queries.

Method	Formula	Complexity
Standard attention	$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$	$O(n^2)$
Linear attention	$\phi(Q)(\phi(K)^\top V)$	$O(n)$

The limitation is that the approximation works for some tasks but degrades on others, particularly tasks requiring precise comparison of specific tokens. Softmax produces a peaky distribution (concentrating attention on a small number of tokens) which is important for retrieval-like tasks that linear approximations cannot replicate faithfully.

6.7.3 State space models and hybrid architectures

Another direction for scaling sequence models is to move away from attention completely and instead model sequences through **learned dynamical systems**. State space models (SSMs) replace pairwise token interaction with a recurrent state that evolves over time, allowing sequences to be processed in linear time.

Rather than computing interactions between all tokens, an SSM maintains a compact hidden state updated sequentially,

$$h_t = Ah_{t-1} + Bx_t, \qquad y_t = Ch_t,$$

so information is carried forward through the state instead of recomputed via an attention matrix. Models such as Mamba achieve $O(n)$ computation and memory while retaining long-range dependency modeling, since past context is compressed into the evolving state.

Hybrid architectures combine both paradigms. Instead of choosing between attention and recurrence, models like Jamba and Griffin interleave attention layers with SSM layers. Attention provides flexible global reasoning when precise token interactions matter, while SSM blocks handle long-context processing efficiently by propagating information through state updates.

These hybrid designs represent an active research direction: as context lengths grow, future architectures may rely less on dense attention and more on mixtures of attention, recurrence, and continuous-time sequence modeling.

6.8 Ring attention and sequence parallelism

For extremely long sequences, even FlashAttention with $O(n)$ memory may exceed a single GPU's capacity with the help of Ring attention the sequence distributions can go across multiple GPUs.

Each GPU holds a slice of the Q, K, and V matrices. The attention computation proceeds in a ring: each GPU computes a partial attention using its local Q slice against the current K/V slice, then passes the K/V to the next GPU while receiving the previous GPU's K/V:

```
Ring of 4 GPUs:

Step 1: GPU0 computes Q0×K0V0, GPU1 computes Q1×K1V1, ...
        K/V rotate: K0V0 → GPU1, K1V1 → GPU2, ...

Step 2: GPU0 computes Q0×K3V3 (received from GPU3)
        GPU1 computes Q1×K0V0 (received from GPU0), ...

Step 4: Each GPU has accumulated contributions from all K/V slices
        → complete attention output for its Q slice
```

After one full revolution, each GPU has computed its complete attention output using K/V from all other GPUs and enables sequence lengths that scale linearly with the number of GPUs.

6.9 Attention in the decode loop: step by step

Here is exactly what happens during one attention computation step during decoding, after prefill is complete and the model is generating token by token:

```
State at step t:
  KV cache contains K, V for tokens 0 through t-1
  New token t has been embedded with positional encoding applied

1. Compute Q, K, V for token t:
   Q_t = x_t @ W_Q    → [1, d_k] per head
   K_t = x_t @ W_K    → [1, d_k] per head  (stored in KV cache)
   V_t = x_t @ W_V    → [1, d_v] per head  (stored in KV cache)

2. Append K_t, V_t to KV cache:
   K_cache[:t+1] = [K_0, K_1, ..., K_{t-1}, K_t]

3. Compute attention scores:
   scores = Q_t @ K_cache[:t+1].T / sqrt(d_k)    → [1, t+1] per head

4. Softmax:
   attn_weights = softmax(scores)    → [1, t+1] per head

5. Weighted sum:
   attn_output = attn_weights @ V_cache[:t+1]    → [1, d_v] per head

6. Concatenate heads and project:
   output = concat(attn_output_per_head) @ W_O    → [1, d_model]

7. Add to residual stream:
   x_t = x_t + output
```

Step 3 is the memory-bandwidth bottleneck: it reads the entire KV cache from HBM. At 4,096 tokens for LLaMA 2 70B, step 3 reads approximately 1.28 GB of KV cache per generation step. At 3.35 TB/s bandwidth, that is 0.38 milliseconds per step (for attention alone, before FFN and other operations). This is why long-context generation is slower per token than short-context generation.

6.10 Key takeaways

- Attention computation scales quadratically with sequence length in both memory and compute; this is the central cost driver for long-context inference, at 128k tokens, the naive score matrix would require 32 GB per layer per head
- Standard attention materializes the full $n \times n$ score matrix in HBM, causing excessive memory traffic; FlashAttention eliminates this by tiling the computation into SRAM and using online softmax, reducing HBM access from $O(n^2)$ to $O(n)$
- During decode, attention is memory-bandwidth-bound: the GPU reads the entire KV cache for a single new token; throughput scales with HBM bandwidth, not arithmetic throughput
- GQA shares K and V projections across groups of heads, reducing KV cache by the ratio of total heads to KV heads, the primary reason GQA is now the default for new models
- Prefill is compute-bound (large batch matrix multiply); decode is memory- bandwidth-bound (vector-matrix multiply against the growing KV cache), the same model, two different hardware bottlenecks
- Sparse attention and linear attention reduce $O(n^2)$ cost but sacrifice flexibility; state space models and hybrid architectures are competitive alternatives for extreme sequence lengths

---

6.11 Further reading

- Dao et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.*: The foundational paper.
- Shazeer (2019). *Fast Transformer Decoding: One Write-Head is All You Need.*: The original Multi-Query Attention paper.
- Ainslie et al. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.*: Grouped-Query Attention.
- Liu et al. (2023). *Ring Attention with Blockwise Transformers for Near-Infinite Context.*: Ring attention for distributed long-context computation.
- Gu & Dao (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.*: The leading SSM alternative to attention.

Core Attention Equation

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$

QK^T → similarity scores between tokens

softmax → attention weights (probability distribution)

V → content being mixed based on weights

Tensor Shapes

Q: $[n, d_k]$

K: $[n, d_k]$

V: $[n, d_v]$

QK^T: $[n, n]$ (attention matrix)

Output: $[n, d_v]$

Intuition

Each token asks: "what should I look at?"

Q = query (what I want)

K = key (what I offer)

V = value (content to pass)

Output = weighted memory retrieval

Causal Mask (Decoder Attention)

Prevents access to future tokens

Implemented by adding $-\infty$ to invalid positions before softmax

$j > i \rightarrow \text{mask}(i,j) = -\infty \rightarrow \text{softmax} = 0$

Ensures autoregressive generation: predict next token only from past

GPU Reality

Two bottlenecks:

- Compute (TFLOPS) → fast matrix math
- Memory bandwidth → KV cache dominates decode

Decode is memory-bound: GPU waits for KV cache, not compute

KV Cache

Stores past Keys & Values

Avoids recomputation every step

Memory grows with sequence length:

$O(n \times \text{layers} \times \text{heads} \times d_k)$

Bottleneck for long-context inference

FlashAttention

Key idea: never store full n×n matrix

Tiling into SRAM (fast on-chip memory)

Online softmax computes statistics incrementally

Result: same output, much less memory traffic

$O(n^2$ memory → $O(n)$ HBM traffic)

Multi-Head Attention

Parallel attention heads

Each head learns different relation type

Concat heads → linear projection

`Specialization emerges from training`

Attention Variants

- MHA: full expressivity, high KV cost
- MQA: shared K/V, minimal cache
- GQA: grouped sharing (LLaMA, Mistral)
- Sparse/Linear: efficiency vs quality tradeoff

Figure 6.2: Cheat sheet.

Chapter 7

KV Cache

The canonical question for this chapter: *Generating one token requires attending to every previous token. How do we avoid recomputing all of that work on every single generation step and what does managing that memory look like in a production system?*

Where are we?

Figure 7.1: The journey through the Model Mind.

The previous chapter showed how attention is computed in inference and this chapter focuses on the engineering implications of that process when generation happens sequentially: at each step, the model must retain access to the keys and values from all previously generated tokens. While the KV cache makes this computationally practical, it becomes the primary memory management challenge in production scale LLM serving.

7.1 The problem that KV cache solves

Autoregressive generation is fundamentally sequential and generating each token requires attending to all previously generated tokens. To generate token `t`, the model must attend to all preceding tokens `0` through `t-1`, and generating each subsequent token expands the context by one, requiring the model to repeatedly process the entire accumulated sequence.

Without caching, generating a 500 token response to a 100 token prompt would require:

```
Step 1:   process 101 tokens → generate token 101
Step 2:   process 102 tokens → generate token 102
...
Step 500: process 600 tokens → generate token 600
```

Total compute proportional to $101 + 102 + ... + 600 \approx 175,000$ token processing steps. For large models, this process is highly inefficient: by step 500, most of the computation power is spent to reprocess tokens that were already handled

at step 1, recomputing the same key and value vectors even though the underlying inputs have not changed.

The KV cache eliminates this redundancy by computing key and value vectors only once and stores them in memory. At each new generation step, the model computes the new token's Q, K, and V vectors, retrieves the cached keys and values for all previous tokens, and performs attention over the full cached context.

With the KV cache:

```
Prefill:          process all 100 prompt tokens once, store their K and V
Each decode step: compute K, V for one new token, append to cache, attend
Total work:       proportional to 100 (prefill) + 500 (one step per token)
```

The savings grow with sequence length. Without the KV cache, practical LLM deployment at today's context lengths would be economically infeasible.

7.2 What is stored

For each token in the sequence, for each transformer layer, for each KV head, the cache stores two vectors: the key and value produced by the linear projections of that token's hidden state.

```
KV cache structure:

Layer 0:
  K: [seq_len, n_kv_heads, d_k]   ← key vectors for all tokens so far
  V: [seq_len, n_kv_heads, d_v]   ← value vectors for all tokens so far

Layer 1:
  K: [seq_len, n_kv_heads, d_k]
  V: [seq_len, n_kv_heads, d_v]

...

Layer L-1:
  K: [seq_len, n_kv_heads, d_k]
  V: [seq_len, n_kv_heads, d_v]
```

Query vectors are not cached because they are used only once: the query for token `t` is needed solely to compute attention at generation step `t` and becomes irrelevant afterward. On the other hand, the key and value vectors for token `t` are reused at every subsequent step until generation ends. The KV cache exploit this asymmetry by storing persistent keys and values while recomputing only the transient query vectors.

7.3 Attention with and without a KV cache

Consider a single attention head. After processing a sequence of length (t),

$$K_t = \begin{bmatrix} k_1 \\ k_2 \\ \vdots \\ k_t \end{bmatrix}, \quad V_t = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_t \end{bmatrix}.$$

To generate the next token, the model first computes the hidden state (h_{t+1}) and projects it to

$$q_{t+1} = W_Q h_{t+1}, \quad k_{t+1} = W_K h_{t+1}, \quad v_{t+1} = W_V h_{t+1}.$$

The attention output is

$$o_{t+1} = \text{softmax}\left(\frac{1}{\sqrt{d_k}} q_{t+1} \begin{bmatrix} K_t & k_{t+1} \end{bmatrix}^T\right) \begin{bmatrix} V_t & v_{t+1} \end{bmatrix}.$$

The important observation is that all previous keys and values already exist:

$$K_t = [k_1, \dots, k_t], \quad V_t = [v_1, \dots, v_t].$$

Only the final column k_{t+1}, v_{t+1} must be computed and appended.

7.3.1 Without caching

At step $t+1$, the model recomputes

$$\{k_1, \dots, k_t, k_{t+1}\}, \quad \{v_1, \dots, v_t, v_{t+1}\}.$$

The cost of the key/value projections is therefore $O(t+1)$. Across an entire generation of length T,

$$\sum_{t=1}^{T} O(t) = O(T^2).$$

7.3.2 With caching

The projections

$$k_i = W_K h_i, \quad v_i = W_V h_i$$

are computed only once when token i is created. During generation of step $t+1$, the model only needs to compute the new projections k_{t+1}, v_{t+1} which results in $O(1)$ projection cost per step. Over a full sequence of length T, this accumulates to $\sum_{t=1}^{T} O(1) = O(T)$.

The attention computation itself still grows linearly with context length, but the repeated projection cost has been eliminated.

7.4 Memory arithmetic

KV cache memory is deterministic and computable from model hyperparameters and sequence length:

`KV cache bytes = 2 × L × n_kv_heads × d_k × seq_len × bytes_per_element`

Where `2` is for K and V, `L` is the number of transformer layers, `n_kv_heads` is the number of KV heads (equals `n_heads` for MHA, fewer for GQA), `d_k` is the key/value dimension per head, and `bytes_per_element` is 2 for BF16.

7.4.1 Examples

LLaMA 2 7B (L=32, n_kv_heads=32, d_k=128, BF16):

```
Per token: 2 × 32 × 32 × 128 × 2 = 524,288 bytes = 512 KB
4,096 tokens: 512 KB × 4,096 = 2 GB
```

LLaMA 2 70B (L=80, n_kv_heads=8, d_k=128, BF16):

```
Per token: 2 × 80 × 8 × 128 × 2 = 327,680 bytes = 320 KB
4,096 tokens:  320 KB × 4,096  = 1.28 GB
32,768 tokens: 320 KB × 32,768 = 10 GB
```

GPT-3 175B (L=96, n_kv_heads=96, d_k=128, BF16):

```
Per token: 2 × 96 × 96 × 128 × 2 = 4,718,592 bytes   4.5 MB
4,096 tokens: 4.5 MB × 4,096 = 18 GB
```

The GPT-3 numbers explain why GQA was adopted: reducing `n_kv_heads` from 96 to 8 reduces the per-token cache from 4.5 MB to 375 KB (12 times reduction) with modest quality loss. GQA is now standard in nearly every new model architecture for exactly this reason.

7.4.2 The memory budget

A serving instance has a fixed GPU memory budget, divided between competing claims:

```
Total GPU memory (e.g., 80 GB H100)
   Model weights     (fixed, e.g., ~35 GB for LLaMA 2 70B in BF16 with GQA)
   Activation memory (fixed per batch, ~1-3 GB)
   KV cache          (variable that grow with sequence length and batch size)
   Reserved          (CUDA overhead, fragmentation buffer, ~3 GB)

Available for KV cache: 80 - 35 - 2 - 3   40 GB
```

For LLaMA 2 70B with 40 GB available:

```
Max tokens in cache: 40 GB / 320 KB   125,000 tokens

Batch size 32, avg sequence 4,096 tokens:
  32 × 4,096 = 131,072 tokens → barely fits

Batch size 32, avg sequence 8,192 tokens:
  32 × 8,192 = 262,144 tokens → does not fit
```

This is the core serving constraint: KV cache capacity limits concurrency at long sequence lengths. Every additional token in context competes directly with additional concurrent users for the same GPU memory.

7.5 The lifecycle of a KV cache entry

Understanding the sequence of operations that touch the KV cache during a single request makes the memory management problem concrete.

7.5.1 Prefill

When a request arrives, the full prompt is tokenized and processed in a single forward pass (the prefill phase). For each layer, K and V are computed for every prompt token simultaneously and written to the KV cache:

```
Input:  [tok_0, tok_1, tok_2, ..., tok_n]  ← full prompt
Output: KV cache populated for positions 0 through n
        Model output: logits at position n → first generated token
```

Prefill is compute-bound and processing all prompt tokens in parallel is a large batch matrix multiply that saturates GPU arithmetic throughput. It is the phase where prompt length most directly affects latency: time to first token scales with prompt length.

7.5.2 Decode

Each decode step generates exactly one new token:

```
1. Embed new token → hidden state
2. Compute K, V for new token → append to KV cache
3. Compute Q for new token
4. Attend: Q against full KV cache (all positions 0 through current)
5. FFN and residual → logits → sample next token
```

Step 4 is memory-bandwidth-bound. The GPU reads the entire KV cache to compute attention for a single query vector. Arithmetic work is trivial compared to the memory transfer. This is why decode throughput is limited by HBM bandwidth, not by compute, and why longer sequences are slower per token.

7.5.3 Request completion

When generation ends (stop token, max_tokens, stop sequence), the KV cache entries for that request are freed. In a naive contiguous allocator this means marking a region of memory as available. In PagedAttention it means returning individual pages to the free pool.

7.6 Naive allocation and its problems

The simplest KV cache allocation strategy: at request admission, allocate a contiguous block of memory sized for the maximum possible output length. Hold it until generation completes, then free it.

This produces two forms of waste:

Internal fragmentation. If the request was allocated 8,192 token positions but only generated 200 tokens, the remaining 7,992 positions were reserved but unused for the lifetime of the request. At 320 KB per token for LLaMA 2 70B, that is 2.55 GB of unusable memory per such request.

External fragmentation. As requests arrive and complete with variable lengths, the free memory becomes fragmented into non-contiguous gaps. A new request requiring 1 GB may fail to be admitted even if 2 GB is technically free, because no single contiguous 1 GB region is available.

On real workloads with variable request lengths, naive allocation wastes 60–80% of available KV cache memory.

7.7 PagedAttention

PagedAttention (Kwon et al., 2023) solves the fragmentation problem by borrowing the virtual memory paging idea from operating systems.

Instead of allocating a contiguous block per request, PagedAttention divides the KV cache into fixed-size pages which typically has 16 tokens per page. A page table maps logical token positions to physical page locations. Pages are allocated on demand as generation proceeds and freed immediately on completion.

```
Page table for three concurrent requests:

Request A (generating, 48 tokens so far):
  Logical positions 0-15  → Physical page 3
  Logical positions 16-31 → Physical page 7
  Logical positions 32-47 → Physical page 12

Request B (generating, 32 tokens so far):
  Logical positions 0-15  → Physical page 1
  Logical positions 16-31 → Physical page 9

Request C (generating, 64 tokens so far):
  Logical positions 0-15  → Physical page 2
  Logical positions 16-31 → Physical page 4
  Logical positions 32-47 → Physical page 5
  Logical positions 48-63 → Physical page 11

Free pages: [0, 6, 8, 10, 13, 14, 15, ...]
```

Physical pages are non-contiguous. Logical positions map to whatever physical page is available. A new request needs four pages for a 64-token sequence; it gets four pages from the free pool regardless of where they are in physical memory.

Internal fragmentation is bounded to at most one partially filled page per request (the current page being written). External fragmentation is eliminated entirely and all free pages are usable regardless of their physical location. PagedAttention achieves over 90% KV cache utilization on typical workloads, compared to 20–40% for naive contiguous allocation.

7.7.1 Prefix sharing

PagedAttention enables a further optimization: memory sharing between requests with a common prefix.

If two requests share the same system prompt like a 1,000-token system prompt that appears in every request to a production API, their KV cache pages for that prefix are identical. There is no reason to store them twice.

With prefix sharing, the pages for the common prefix are marked as shared and reference-counted. Multiple requests point to the same physical pages via their page tables:

```
Request A page table:
  Positions 0-15   → Physical page 7  (shared system prompt, page 1)
  Positions 16-31  → Physical page 8  (shared system prompt, page 2)
  Positions 32-47  → Physical page 12 (private request A's user message)

Request B page table:
  Positions 0-15   → Physical page 7  (shared same physical pages)
  Positions 16-31  → Physical page 8
  Positions 48-63  → Physical page 15 (private request B's user message)
```

When both requests complete, the shared pages are not freed until all references are released. This is copy-on-write semantics: pages are shared until a write is required, at which point a private copy is made.

For a 1,000-token system prompt with 16 tokens per page: 63 pages of KV cache are shared across all concurrent requests using that prompt. At 320 KB per token for LLaMA 2 70B, this is 320 MB of KV cache that does not need to be recomputed or stored per-request.

7.7.2 Radix attention

Radix attention (SGLang, Zheng et al., 2023) generalizes prefix sharing to arbitrary prefix trees rather than just common prefixes.

In applications with structured multi-step prompts system like agentic workflows, few-shot templates, and multi-document retrieval, requests may share prefixes at multiple points in their structure, not just at the beginning. Radix attention maintains a trie (prefix tree) of all cached KV sequences. When a new request arrives, the longest matching prefix in the trie is found and reused:

```
Prefix tree:
  [System prompt] → shared by all requests
       [Document A] → shared by requests querying document A
           [Query 1] → private to request 1
           [Query 2] → private to request 2
       [Document B] → shared by requests querying document B
            [Query 3] → private to request 3
```

This turns prefix sharing from a single-level optimization into a general caching system for the KV cache. For agentic applications where many requests share substantial common context, radix attention can dramatically reduce both memory and compute.

7.8 KV cache quantization

The KV cache can be stored in reduced precision to trade a small amount of quality for significant memory savings.

INT8 KV cache stores key and value vectors as 8-bit integers rather than 16-bit floats. This provides a 2× memory reduction. Quality loss is typically negligible for standard generation tasks K and V vectors have bounded ranges as outputs of linear projections of layer-normalized hidden states, making them well-suited to quantization.

FP8 KV cache uses 8-bit floating point rather than integer quantization. H100 GPUs have native FP8 support. Better precision characteristics than INT8 for values with non-uniform distributions, with the same 2× memory reduction.

INT4 KV cache reduces to 4 bits for a 4× memory reduction. More quality loss; requires careful calibration. Used in memory-constrained scenarios where the alternative is rejecting requests.

The quality tradeoff for KV cache quantization is typically smaller than for weight quantization for two reasons: quantization errors in the KV cache are averaged across many attention heads (reducing their impact), and modern calibration techniques can minimize the effect on attention score distributions.

INT8 KV cache quantization is now standard in production and is supported by vLLM, TensorRT-LLM, and most major inference frameworks. The 2× memory savings effectively doubles serving concurrency on fixed hardware.

7.9 KV cache eviction

Some applications require sequences longer than the KV cache can hold. Rather than rejecting these requests, The serving system can evict cached entries to make room for new ones, at the cost of a quality trade-off.

7.9.1 Eviction policies

Sliding window evicts the oldest tokens, keeping only the most recent `w` token positions in cache. Simple to implement. Degrades quality for any task requiring long-range context, the model simply cannot attend to evicted positions.

H2O (Heavy-Hitter Oracle) evicts tokens with the lowest cumulative attention scores. Tokens that have received little attention weight in recent steps are unlikely to be needed in future steps. Empirically preserves quality better than sliding window eviction at the same cache budget.

Attention sink preservation always keeps the BOS (beginning of sequence) token regardless of other eviction decisions. The BOS token functions as an attention sink and many heads route excess attention weight to it when no

other token is highly relevant. Evicting the BOS token causes attention distributions to become erratic and output quality to degrade unpredictably. Any eviction policy should treat the BOS token as eviction-exempt.

SnapKV identifies important KV vectors per head using attention patterns from a recent observation window, then evicts the remaining positions. Can maintain quality at 10–20% of the full KV cache size for long documents make it useful for very long context workloads where even INT8 quantization is insufficient.

7.10 Key takeaways

- The KV cache stores key and value vectors for all previous tokens, avoiding quadratic recomputation cost; query vectors are not cached because they are used exactly once per step
- KV cache memory is precisely computable: `2 × L × n_kv_heads × d_k × seq_len × bytes_per_element`; GQA's reduction in KV heads is the primary architectural lever for reducing this cost
- Naive contiguous allocation wastes 60–80% of KV cache memory through internal and external fragmentation; PagedAttention eliminates both by allocating fixed-size pages on demand, achieving 90%+ utilization
- Prefix sharing lets multiple requests reference the same physical KV cache pages for a shared prompt prefix — identical K/V vectors computed once, stored once; radix attention generalizes this to arbitrary shared structure
- INT8 KV cache quantization provides a 2× memory reduction with negligible quality loss, effectively doubling serving concurrency on fixed hardware
- Eviction policies (sliding window, H2O, SnapKV) enable sequences longer than the cache capacity; attention sink preservation is mandatory — evicting the BOS token causes unpredictable quality degradation
- StreamingLLM enables unbounded generation on a fixed cache budget by combining BOS token preservation with a sliding recent window
- CPU offloading moves inactive request caches to DRAM, freeing GPU HBM for active requests at the cost of PCIe transfer latency on resumption
- KV cache metrics — utilization, prefix hit rate, eviction rate, throughput vs. sequence length — are the primary production signals for diagnosing memory pressure and serving efficiency

---

7.11 Further reading

- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* — the foundational work on paged KV cache management.
- Zheng et al. (2023). *SGLang: Efficient Execution of Structured Language Model Programs.* — Introduces radix attention for generalized prefix tree sharing.
- Xiao et al. (2023). *Efficient Streaming Language Models with Attention Sinks.* — StreamingLLM and the attention sink phenomenon; directly relevant to eviction strategy design.
- Zhang et al. (2023). *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models.* — Attention-score-based eviction.
-

Core Idea

- Autoregressive decoding recomputes attention over all previous tokens
- KV cache stores Keys & Values once and reuses them for all future steps
- Eliminates redundant projection + recomputation during generation

`Without cache: O(T²) recomputation → With cache: O(T)`

Why It Matters in Production

Each new token attends to all previous tokens in KV cache

Decode cost becomes memory-bandwidth bound (HBM), not compute bound

Long context → KV cache dominates GPU memory and throughput

What is Stored

For each layer:

`K: [seq_len, heads, d_k]`

`V: [seq_len, heads, d_v]`

Stored per token, per layer, per head

Q is NOT stored (used once per step)

Growth: linear in sequence length

Attention with KV Cache

`o_t = softmax(Q_t K_cache^T) V_cache`

At step t:

- Q_t computed fresh
- K/V appended once per token
- All past K/V reused

No recomputation of history

Without vs With KV Cache

Without cache:

`Recompute K,V for all tokens at every step → O(T²)`

With cache:

`Compute K,V once → reuse → O(T)`

Memory Cost

`KV bytes = 2 × L × n_kv_heads × d_k × seq_len × bytes`

Key drivers:

- Layers (L)
- KV heads (GQA reduces this)
- Context length (linear growth)

Decode Bottleneck

Each token requires reading full KV cache

→ HBM bandwidth bound (not FLOPs)

At long context:

`1–10 GB memory read per token step`

Result: longer context = slower generation

Prefill vs Decode

Prefill:

`Parallel matmul → compute-bound`

Decode:

`Vector × matrix → bandwidth-bound`

Same model, different bottleneck regime

Key Optimizations

- GQA: reduce KV heads → smaller cache
- PagedAttention: avoid fragmentation (paging instead of contiguous memory)
- Prefix sharing: reuse KV for shared prompts
- INT8/FP8 KV cache: 2× memory reduction
- Eviction policies: sliding window / H2O / SnapKV

`Goal: maximize GPU utilization under fixed HBM budget`

Figure 7.2: Cheat sheet.

Chapter 8

GPU Execution

The canonical question for this chapter: *When the model runs, what is GPU actually doing? Why does the same operation take wildly different amounts of time depending on how it is scheduled, and how the data is laid out in memory?*

Where are we?

Figure 8.1: The journey through the Model Mind.

The previous chapters established what the model computes: embeddings, attention, feed-forward networks, the KV cache. This chapter covers *where* that computation actually happens and why it behaves the way it does. Every optimization discussed so far (FlashAttention, PageAttention, continuous batching, quantization) exists because of specific GPU execution constraints. This chapter makes those constraints explicit.

8.1 Why GPU execution deserves its own chapter

Every previous chapter has mentioned GPU constraints in passing: memory bandwidth, arithmetic throughput, HBM capacity. This chapter makes those concepts precise.

Understanding GPU execution has direct impact ob system performance and it is not just an academic exercise for LLM engineers. A serving system operating at 60% of a GPU's theoretical peak throughput can deliver 50% more throughput than one running at just 40%, without any additional hardware. This knowledge also explains why certain operations become bottlenecks, why quantization improves performance, why kernel fusion is so effective, and why the prefill and decode phases show different latency characteristics.

More directly: every optimization technique in serving (FlashAttention, continuous batching, tensor parallelism) exists because of specific GPU execution constraints. Understanding those constraints explains why the optimizations take the form they do, rather than appearing as a collection of unrelated tricks.

8.2 GPU architecture fundamentals

A modern GPU is not a fast CPU. It is a fundamentally different compute architecture optimized for a different class of problems.

8.2.1 The execution model

A CPU has a small number of powerful cores (typically 8–128), each capable of complex out-of-order execution, branch prediction, and deep caches. CPUs are optimized for latency and completing a single thread of execution as fast as possible.

A GPU has a massive number of simpler cores (an H100 has 16,896 CUDA cores) organized into streaming multiprocessors (SMs). Each SM is not particularly fast for a single operation, but thousands of them executing simultaneously produce enormous total throughput. GPUs are optimized to complete as many simple operations as possible per second, not completing any one operation quickly.

The programming model reflects on this: GPU programs are written as kernels that execute the same operation on many data elements simultaneously. A matrix multiplication kernel runs the same multiply operation on thousands of elements in parallel. If there is data parallelism in your problem, a GPU exploits it. If there is not, a GPU is slower than a CPU for that specific task.

LLM inference has enormous data parallelism: the same transformer operations apply to every token in the sequence, and every element of every weight matrix participates in the same type of computation. This structural match is why GPUs dominate LLM inference.

8.2.2 The memory hierarchy

GPU memory has multiple tiers with very different bandwidth and capacity:

Memory tier	Bandwidth	Capacity	Latency
Registers	~80 TB/s	256 KB/SM	~1 cycle
L1/SRAM	~20 TB/s	256 KB/SM	~30 cycles
L2 cache	~5 TB/s	50 MB	~200 cycles
HBM (VRAM)	~3.35 TB/s	80 GB	~600 cycles
PCIe (CPU RAM)	~64 GB/s	TBs	~microseconds
NVLink (GPU-GPU)	~900 GB/s	-	~microseconds

SRAM (often called shared memory or L1 cache) is the GPU's fast on-chip memory. It offers extremely low latency but limited capacity. HBM (High Bandwidth Memory), is the GPU's large off-chip memory although it provides massive bandwidth, accessing HBM is still significantly more expensive than accessing on-chip memory.

Every GPU computation ultimately reads data from HBM and writes results back to it. As a result, the central challenge of GPU kernel optimization is reducing HBM traffic by keeping frequently accessed data in SRAM for as long as possible which is the exact key insight behind FlashAttention.

8.2.3 Streaming multiprocessors and warps

The Streaming Multiprocessor (SM) is the fundamental compute unit of an NVIDIA GPU. An H100 GPU contains many SMs operating in parallel, with each SM responsible for executing thousands of lightweight threads concurrently.

Each SM on an H100 contains:

- 128 CUDA cores for FP32 arithmetic operations.
- 4 Tensor Cores specialized for matrix multiplication.
- 256 KB of registers for storing temporary values used by active threads.
- 228 KB of on-chip SRAM, configurable between:
  - L1 cache for automatically caching memory accesses.

  - Shared memory for explicitly sharing data between threads in the same thread block.
- Warp schedulers that determine which groups of threads execute next.

GPU threads execute in groups called warps, where each warp consists of 32 threads. Unlike CPU threads, which execute independently, all threads within a warp execute the same instruction at the same time on different pieces of data. NVIDIA refers to this execution model as SIMT (Single Instruction, Multiple Threads).

For example, consider the following code executed by a warp:

```
if x > 0:
    y = a + b
else
    y = c + d
```

Suppose that among the 32 threads in the warp 20 threads satisfy $x > 0$ and 12 threads satisfy $x <= 0$ The GPU cannot execute both branches simultaneously within the same warp. Instead, it execute $y = a + b$ for the 20 relevant threads while the other 12 threads remain temporarily inactive and than execute $y = c + d$ for the remaining 12 threads while the first 20 threads remain inactive. Conceptually, execution looks like:

```
Cycle 1:
[Active]   Threads 0-19  -> y = a + b
[Inactive] Threads 20-31

Cycle 2:
[Inactive] Threads 0-19
[Active]   Threads 20-31 -> y = c + d
```

Although each thread only needs one branch, the warp executes both branches sequentially, reducing parallel efficiency whcih is known as branch divergence.

An H100 has 132 SMs. With 4 warp schedulers per SM and 32 threads per warp, the H100 can keep $132 \times 4 \times 32 = 16{,}896$ threads active simultaneously.

8.2.4 Tensor cores

Standard CUDA cores perform scalar floating point operations. Tensor cores perform matrix multiplications directly in hardware, at much higher throughput.

An H100 tensor core performs a 16×16×16 matrix multiplication in a single operation. The H100 has 528 tensor cores, delivering approximately:

- 1,000 TFLOPS for BF16/FP16 matrix multiplications
- 1,979 TFLOPS for FP8 matrix multiplications
- 66 TFLOPS for FP32 scalar operations

The gap between tensor core throughput and scalar throughput explains why matrix multiplication is fast and why all other operations are relatively expensive on modern GPUs.

8.3 The roofline model

The roofline model is a simple framework for understanding whether a given operation is compute-bound or memory-bandwidth-bound, and what the theoretical maximum performance is:

```
Attainable performance = min(Peak FLOPS, Memory bandwidth × Arithmetic intensity)
```

Where arithmetic intensity = FLOPs executed / bytes read from memory.

The ridge point is where the roofline transitions from memory-bound to compute-bound which is approximately 300 FLOPs/byte for the H100 (1,000 TFLOPS / 3.35 TB/s). Operations with arithmetic intensity above 300 are compute-bound and those below are memory-bandwidth-bound.

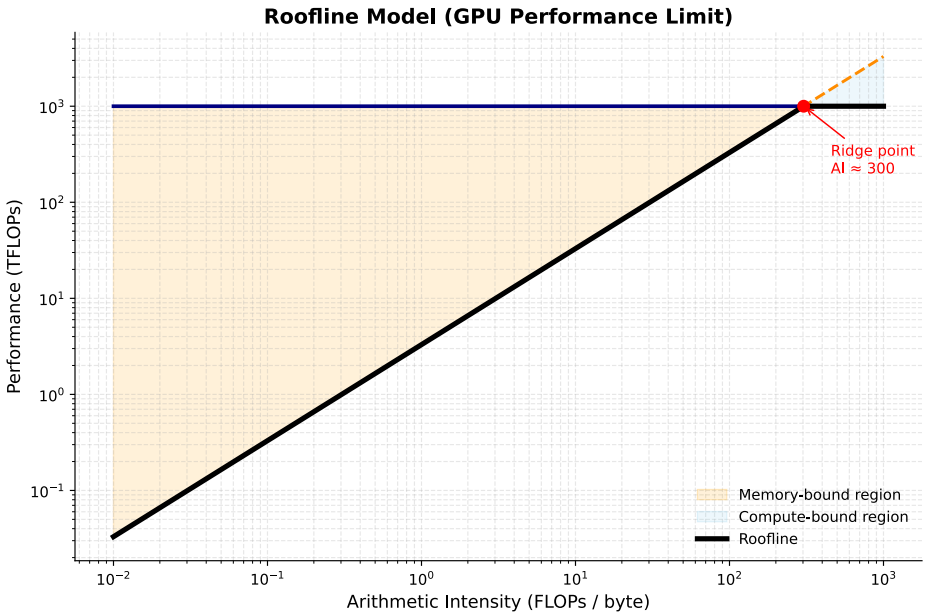


Figure 8.2: The roofline Model.

8.3.1 LLM operations on the roofline

Different operations in LLM inference have very different arithmetic intensities:

Large matrix multiplications (prefill, large batches): Matrix multiplication achieves high arithmetic intensity by reusing each byte of weight data for every sequence in the batch. For example, with a batch size of 32 and $d_model = 4,096$, the arithmetic intensity is approximately 32 FLOPs/byte. This value approaches the roofline model's ridge point, indicating that performance is primarily limited by computational throughput rather than memory bandwidth.

Vector-matrix products (decode, batch size 1): The Q projection at a batch size of 1 exhibits low arithmetic intensity because each byte of weight data is used in only one multiply operation. Consequently, the arithmetic intensity is approximately 1 FLOP/byte, which is about 300× below the roofline model's ridge point. As a result, performance is severely limited by memory bandwidth, leaving the tensor cores underutilized.

Attention with KV cache (decode): extremely low arithmetic intensity. Reading the full KV cache and computing a dot product with the query vector. Memory- bandwidth-bound with near-zero compute utilization.

Softmax, layer norm, element-wise ops: essentially zero arithmetic intensity. Read data, apply a simple function, write it back. Pure memory bandwidth operations. No tensor core involvement.

The practical implication: for decode at small batch sizes, the GPU is almost entirely idle arithmetically, waiting for data from HBM. Increasing batch size is the primary lever for improving GPU utilization during decode, because it amortizes weight reads across multiple sequences and increases arithmetic intensity toward the ridge point.

8.4 Prefill vs. decode: two different regimes

The roofline analysis explains why prefill and decode behave so differently and they are not just slow and fast versions of the same operation. They are different hardware bottlenecks entirely.

8.4.1 Prefill: compute-bound

During prefill, all prompt tokens are processed simultaneously. The Q projection operates on a matrix of shape `[n_prompt, d_model]`:

```
Q projection: [n_prompt, d_model] @ [d_model, d_model]
FLOPs:        2 × n_prompt × d_model²
Bytes:        d_model² × 2    (weight matrix, loaded once)

Arithmetic intensity = n_prompt FLOPs/byte
```

For n_prompt = 1,024 and d_model = 4,096: arithmetic intensity = 1,024 FLOPs/byte which is above the ridge point. Prefill is compute-bound. The H100 runs near its theoretical FLOPs ceiling. In order to make prefill faster it requires more FLOPs (more GPUs, higher clock speeds, or more efficient kernels).

8.4.2 Decode: memory-bandwidth-bound

During decode, only one new token is processed at a time. The Q projection operates on a vector of shape `[1, d_model]`:

```
Q projection: [1, d_model] @ [d_model, d_model]
FLOPs:        2 × 1 × d_model²
Bytes:        d_model² × 2    (same weight matrix)

Arithmetic intensity = 1 FLOPs/byte
```

For d_model = 4,096: arithmetic intensity = 1 FLOPs/byte which makes decode memory-bandwidth-bound. The H100 runs at approximately 1/300th of its peak FLOPs utilization. Making decode faster requires loading weights faster, higher bandwidth memory, quantization (fewer bytes per parameter), or larger batches (sharing weight reads across sequences).

8.5 Kernel fusion

Each operation in a transformer block, such as layer normalization, Q/K/V projection, softmax, and residual addition, can be implemented as a separate CUDA kernel. In this approach, every kernel reads its inputs from HBM and writes its outputs back to HBM before the next kernel executes. While this design is straightforward to implement, it is inefficient because intermediate results are repeatedly transferred to and from HBM despite being needed only by subsequent operations.

Kernel fusion addresses this inefficiency by combining multiple operations into a single CUDA kernel. Intermediate results are retained in registers or on-chip shared memory (SRAM) rather than being written to HBM after each operation. Only the final output is stored in HBM, reducing memory traffic, increasing arithmetic intensity, and improving overall performance.

8.5.1 Example: fused layer norm and linear projection

Without fusion:

```
x_norm = layer_norm(x)     # read x from HBM, write x_norm to HBM
q = x_norm @ W_Q           # read x_norm from HBM, write q to HBM
```

Two HBM reads, two HBM writes, for data that is only an intermediate result.

With fusion:

```
Single kernel:
  Read x from HBM once
```

```
Compute layer norm in registers/SRAM
Apply W_Q projection immediately
Write q to HBM once
```

One HBM read, one HBM write. Memory traffic halved for this pair of operations. The intermediate `x_norm` never leaves the chip.

8.5.2 FlashAttention as kernel fusion

FlashAttention is the most impactful example of kernel fusion in LLM inference. The naive attention computation reads and writes the n×n score matrix multiple times (scaling, masking, softmax, and the final weighted sum). FlashAttention fuses all of these into a single kernel that keeps intermediate scores in SRAM.

The memory traffic reduction is from $O(n^2)$ to $O(n)$. At long sequence lengths, this is the difference between attention fitting in available memory and not fitting at all. Chapter REFF covers FlashAttention in detail; the point here is that it is fundamentally a kernel fusion optimization, not an algorithmic approximation. The result is mathematically identical to naive attention.

8.5.3 Kernel fusion in practice

Production inference frameworks apply kernel fusion throughout the model:

- Layer norm fused into linear projections (Q, K, V, output projections)
- Attention scores, masking, and softmax fused (FlashAttention)
- Residual addition fused with layer norm
- Activation functions fused into FFN computation
- RoPE application fused into Q and K projection

The cumulative impact is typically 1.5–2× throughput improvement over naively separate kernels, primarily by reducing HBM traffic.

8.6 Precision and quantization at the kernel level

Quantization affects GPU execution at the kernel level, not just memory footprint. Understanding this distinction separates knowing what quantization does from knowing why it helps.

8.6.1 Weight-only quantization

The most common serving quantization scheme: model weights are stored in INT4 or INT8, but computation happens in BF16/FP16.

During a weight-only quantized matrix multiplication:

```
1. Load INT4 weights from HBM          (4× fewer bytes than BF16)
2. Dequantize to BF16 in registers     (fast, negligible compute overhead)
3. Multiply with BF16 activations via tensor cores
4. Accumulate in FP32, write BF16 output to HBM
```

The benefit: decode is memory-bandwidth-bound. Reducing weight bytes by 4× means 4× more weights can be streamed per second, directly improving decode throughput by approximately 4× for bandwidth-limited operations.

The cost: dequantization adds compute overhead. For bandwidth-bound operations (decode), this overhead is negligible. For compute-bound operations (prefill, large batches), dequantization adds meaningful overhead. Weight-only quantization therefore improves decode more than prefill, which is precisely where the improvement is most needed.

8.6.2 Activation quantization: INT8 and FP8

For compute-bound operations, quantizing both weights and activations to INT8 or FP8 allows tensor cores to operate at higher throughput:

- BF16 tensor cores: 1,000 TFLOPS
- FP8 tensor cores: 1,979 TFLOPS on H100 (approximately 2× faster)

Activation quantization is more complex than weight quantization because activations have more dynamic range and they can contain outlier values that cause large quantization errors when mapped to a narrow integer range.

The outlier problem. LLM activations contain outlier values in a small fraction of channels, values that are orders of magnitude larger than typical. These outliers are consistent across inputs and are a known property of large language models. Naive quantization sets the scale by the outlier, compressing all normal values to near-zero and destroying information.

LLM.int8() (Dettmers et al., 2022) addresses this by decomposing the matrix multiplication: outlier channels are computed in full BF16 precision, and the remaining channels (typically 99%+) are computed in INT8. The results are combined. Quality is preserved while most of the memory savings are retained.

SmoothQuant (Xiao et al., 2023) migrates the quantization difficulty from activations to weights, mathematically equivalent to scaling the activation channels down and the corresponding weight rows up before quantization. The result is more uniform activation distributions that quantize cleanly to INT8 without special outlier handling.

FP8 inference is available on H100 GPUs and has become the recommended precision for large-scale serving: near-BF16 quality at roughly 2× the arithmetic throughput. The H100's Transformer Engine handles FP8 scaling automatically, making it practical without manual calibration.

8.7 Tensor parallelism at the kernel level

When a model is too large for a single GPU, tensor parallelism distributes weight matrices across multiple GPUs. The kernel-level implementation requires communication between GPUs at each layer.

8.7.1 Column and row parallelism

A linear projection `Y = X @ W` can be split in two ways:

Column parallelism splits W by columns:

```
W split into [W  | W ] across 2 GPUs
GPU 0: Y  = X @ W     (partial output - subset of output dimensions)
GPU 1: Y  = X @ W     (partial output - complementary subset)
Combine: Y = [Y  | Y ]    (concatenation, no communication needed here)
```

Row parallelism splits W by rows and requires split input:

```
W split into [W ; W ], X split into [X , X ] across 2 GPUs
GPU 0: Y_partial  = X  @ W
GPU 1: Y_partial  = X  @ W
Combine: Y = Y_partial  + Y_partial     (all-reduce required)
```

In a transformer layer: Q, K, V projections use column parallelism, each GPU computes a subset of attention heads. The output projection uses row parallelism. Attention within each GPU is independent; an all-reduce synchronizes after the output projection. The FFN follows the same pattern: column parallelism on the first layer, row parallelism on the second, with one all-reduce between them.

8.7.2 Pipeline parallelism and bubble overhead

Pipeline parallelism splits a transformer model across multiple GPUs by assigning different sets of layers to different devices. Instead of each GPU holding the full model, computation flows through GPUs sequentially, like an assembly line and each GPU processes a contiguous block of layers. To keep all GPUs busy, the input batch is divided into microbatches, which are then streamed through the pipeline:

```
Time step:         1    2    3  |  4    5    6    7    8  | 9   10   11
                 <-- Fill -->  | <---- Steady state ---> | <-- Drain -->

GPU 0 (L0 -L24): [1]  [2]  [3]   [4]  [5]  [6]  [7]  [8]   ·    ·    ·
GPU 1 (L25-L48):  ·   [1]  [2]   [3]  [4]  [5]  [6]  [7]  [8]   ·    ·
GPU 2 (L49-L72):  ·    ·   [1]   [2]  [3]  [4]  [5]  [6]  [7]  [8]   ·
GPU 3 (L73-L96):  ·    ·    ·    [1]  [2]  [3]  [4]  [5]  [6]  [7]  [8]
```

This creates a wave-like execution pattern where different GPUs work on different microbatches at different stages of the model. However, pipeline parallelism introduces idle time, known as the pipeline bubble. This occurs during pipeline fill (startup) and pipeline drain (shutdown), when not all GPUs are fully utilized.

Increasing the number of microbatches reduces bubble overhead by improving pipeline utilization, but it also increases memory usage and scheduling complexity. In practice, systems use 8–32 microbatches to keep bubble overhead under 10% while maintaining efficiency.

8.8 CUDA graph capture

Every GPU operation requires a kernel launch: the CPU sends an instruction to the GPU specifying which kernel to run with which parameters. For small operations, kernel launch overhead can dominate the actual execution time.

For a transformer decode step at batch size 1:

- Total arithmetic work: small (bandwidth-bound, most of step time is HBM reads)
- Number of kernel launches: hundreds (one per operation across all layers)
- Kernel launch overhead: ~5–10 microseconds per launch

Hundreds of launches × 5–10 microseconds = milliseconds of CPU overhead per decode step, for a step that may itself take only a few milliseconds of actual GPU work. The CPU becomes the bottleneck.

CUDA graphs capture the sequence of kernel launches and their data dependencies as a static graph, replayed without CPU involvement for each step. After the initial capture, subsequent replays bypass CPU overhead entirely.

For decode at batch size 1, CUDA graph capture reduces per-step latency by 30–50%. The tradeoff: the graph is captured for a specific batch size and sequence length; changing these requires recapture. Production inference frameworks use CUDA graphs for decode (fixed batch size and one new token per step) and not for prefill (variable sequence length per request).

8.9 Key takeaways

- GPUs are throughput-optimized processors with thousands of simple cores; LLM inference maps well to GPUs because transformer operations are highly data- parallel — the same computation applied to every token and every weight element
- The GPU memory hierarchy spans registers (80 TB/s) to HBM (3.35 TB/s) to PCIe (64 GB/s); kernel optimization is the art of keeping hot data in fast memory and minimizing HBM round trips
- Tensor cores perform 16×16×16 matrix multiplications in hardware at ~15× the throughput of scalar CUDA cores; every operation expressible as matrix multiplication should be

- The roofline model predicts whether an operation is compute-bound (arithmetic intensity above ~300 FLOPs/byte for H100) or memory-bandwidth-bound (below); prefill is compute-bound, decode is severely memory-bandwidth-bound at ~1 FLOPs/byte
- MFU for decode at batch size 1 is approximately 0.3% — the GPU is nearly idle arithmetically; batching and quantization are the primary levers for improving this
- Kernel fusion eliminates intermediate HBM reads and writes between operations; FlashAttention reduces attention memory traffic from $O(n^2)$ to $O(n)$ by keeping scores in SRAM throughout computation
- Weight-only INT4/INT8 quantization reduces decode latency proportionally to the bit reduction, with negligible dequantization overhead when bandwidth-bound; FP8 activation quantization improves prefill throughput by ~2×
- CUDA graph capture eliminates CPU kernel launch overhead — 30–50% latency reduction for decode at small batch sizes — by replaying a static kernel sequence without CPU involvement

---

8.10 Further reading

- Williams et al. (2009). *Roofline: An Insightful Visual Performance Model for Multicore Architectures.* — The roofline model; essential reading for GPU performance analysis regardless of domain.
- Dao et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* — The most important kernel fusion example in LLM inference; the IO complexity analysis is the key insight.
- Dettmers et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.* — The outlier problem and mixed-precision INT8 quantization.
- Xiao et al. (2023). *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models.* — Migrating quantization difficulty from activations to weights for clean INT8 inference.
- Shoeybi et al. (2019). *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism.* — Tensor and pipeline parallelism; the foundational implementation reference.

Transformer Architecture — Cheat Sheet

Why Transformers Won

- RNNs: sequential → no parallelism
- Long-range info bottleneck in hidden state
- Transformers: attention → full parallelism + direct token interaction

High-Level Pipeline

Tokens → Embeddings → Positional Encoding → Transformer Blocks → Logits → Softmax

`P(next token | context)`

Embeddings & Position

Token embedding: lookup table (vocab → vectors)

Weight tying: input embedding = output projection

Positional encodings:

- Sinusoidal (fixed, original transformer)
- Learned embeddings
- RoPE: rotates Q/K → relative position

Self-Attention

`Q = XWq, K = XWk, V = XWv`

`Attention = softmax(QKᵀ / √d) V`

Each token attends to all previous tokens

Creates contextualized representation

Attention Masks

- Causal (decoder): only past tokens visible
- Bidirectional (encoder): full sequence visible (BERT)
- Encoder-decoder: cross-attention between memory + generation
- Prefix LM: prefix bidirectional, suffix causal

Multi-Head Attention

Multiple parallel attention heads

Each head learns different relation type

Outputs concatenated + projected

`Specialization emerges from training`

Feed-Forward Network

`FFN(x) = W2 · activation(W1 x)`

Per-token independent transformation

Modern activations:

- GELU (default GPT/BERT)
- SwiGLU (modern LLaMA, Mistral)

Residual + Normalization

x = x + Sublayer(LayerNorm(x)) (Pre-LN)

Stabilizes deep training

LayerNorm → RMSNorm (modern models)

Pre-LN = stable gradients at scale

Modern Variants

- RoPE: relative position encoding
- GQA: shared K/V across heads
- MoE: sparse expert FFNs
- Sliding window attention (Mistral)

Output Layer

`logits = h Wᵀ → softmax → P(next token)`

Training objective: next-token prediction only

Figure 8.3: Cheat sheet.

Chapter 9

Decoding and Sampling

The canonical question for this chapter: *The model produces a probability distribution over 100,000 tokens. How do you turn that distribution into the actual text the user sees, and how does that choice shape everything from creativity to factuality to cost?*

Where are we?

Figure 9.1: The journey through the Model Mind.

The transformer forward pass is complete. The model has produced a vector of logits one score per vocabulary token. This chapter covers what happens next: how those logits become a token, and how that single choice, made thousands of times, produces the text the user reads. The decoding strategy is the last step before output also one of the most consequential.

9.1 The decoding problem

After the transformer forward pass, the output is a vector of logits one score per token in the vocabulary. Applying softmax converts these to a probability distribution:

```
logits:        [2.3,  -1.2,  0.8,  4.1,  -0.3,  ...]   shape: [vocab_size]
probabilities: [0.04,  0.001, 0.01, 0.33,  0.004, ...]   shape: [vocab_size]
```

The highest-probability token is not always the right choice. Selecting it every time known as greedy decoding produces text that is repetitive, overconfident, and often worse than sampling from the distribution. Nevertheless sampling randomly from the full distribution produces incoherent text that assigns probability to every token, including terrible ones.

Decoding is the set of strategies for navigating this tradeoff: selecting a token (or sequence of tokens) from the distribution in a way that produces output that is coherent, diverse, and appropriate for the task. It runs once per

generated token, which for a 500 token response means it runs 500 times. Small differences in strategy compound into large differences in output.

9.2 Greedy decoding

The simplest strategy: always select the token with the highest probability.

```
def greedy_decode(logits):
    return logits.argmax(dim=-1)
```

Greedy decoding is deterministic the same prompt always produces the same output. It is fast (no sampling required) and produces coherent local choices at each step.

The problem: local optimality does not imply the global optimality. The highest- probability token at step `t` may lead to a sequence that is less probable overall than a sequence that takes a lower probability token at step `t` and recovers. As a result, greedy decoding can become trapped in a local optimum.

The most visible symptom is repetition. Once a model enters a repetitive pattern, greedy decoding has no escape mechanism, the most likely next token at each step continues the pattern. The model knows, in some sense, that it is repeating itself (it assigns lower probability to the repetition than to alternatives), but greedy decoding ignores that signal and repeats anyway.

Greedy decoding is appropriate for tasks with a single correct answer (extracting specific information, classification, structured output generation where diversity is undesirable) and for debugging, where the deterministic output makes it easier to isolate model behavior from sampling noise.

9.3 Beam search

Beam search maintains a set of `k` candidate sequences and extends all of them at each step, keeping only the `k` most probable overall:

```
Beam width k=2, step 1:
  Candidates: ["The cat", "A dog"]
  Scores:     [log P("The") + log P("cat"|"The"),
               log P("A") + log P("dog"|"A")]

Step 2: extend each candidate:
  "The cat sat", "The cat ran", "The cat is", ...   (vocab_size options)
  "A dog ran",   "A dog sat",   "A dog is", ...     (vocab_size options)

Keep top k=2 by cumulative log probability:
  ["The cat sat", "A dog ran"]
```

At the end of generation, the candidate with the highest cumulative log probability is returned. Beam search finds sequences with higher overall probability than greedy decoding because it can recover from locally suboptimal choices.

9.3.1 The beam search curse

Beam search dominated sequence-to-sequence tasks (translation, summarization) from roughly 2015 to 2020. For tasks with constrained output spaces, it performs well. For open-ended generation, it has a well-documented failure mode: it produces output that is generic and boring.

High-probability sequences tend to be safe, common phrases. The beam converges to the statistical mean of training data rather than producing specific, vivid text. This is the beam search curse: the sequences with highest probability

under the model are not the sequences humans prefer. Human language is not the highest- probability sequence, people regularly choose moderately probable, interesting words over the most probable, expected ones.

Beam search is still used for translation (k=4 is typical), structured data extraction, and code generation with hard constraints. For conversational and creative generation, sampling-based methods have largely replaced it.

9.4 Temperature sampling

Temperature is one of the most important hyperparameters in decoding. It adjusts the probability distribution over the next token before sampling, making the distribution either sharper or flatter.

9.4.1 The temperature transformation

Before sampling, the logits are divided by a temperature parameter (T):

```
adjusted_probs = softmax(logits / T)
```

T < 1.0 (cold): the distribution become sharper and high-probability tokens receive relatively more probability mass, while low-probability tokens receive less. At $T \to 0$, the distribution approaches a point mass on the highest-probability token. Consequently, stochastic sampling becomes deterministic, recovering the behavior of greedy decoding.

T = 1.0: the distribution is unchanged from the model's output.

T > 1.0 (hot): the distribution becomes flatter and probabilities are spread more evenly across tokens. At $T \to \infty$, the distribution becomes uniform.

```
Logits:          [4.0,  2.0,  1.0,  0.5]

T=0.5 (cold):    softmax([8.0, 4.0, 2.0, 1.0]) → [0.86, 0.12, 0.02, 0.01]
T=1.0 (normal):  softmax([4.0, 2.0, 1.0, 0.5]) → [0.68, 0.17, 0.08, 0.07]
T=2.0 (hot):     softmax([2.0, 1.0, 0.5, 0.25])→ [0.45, 0.27, 0.16, 0.12]
```

9.4.2 Choosing temperature

T = 0 (greedy): factual retrieval, classification, structured output. Maximizes accuracy for tasks with correct answers.

T = 0.1–0.4: code generation, factual Q&A, tasks where accuracy matters but some flexibility is useful. Concentrated distribution that rarely samples improbable tokens.

T = 0.7–1.0: conversational responses, general-purpose assistants. Balanced diversity and coherence. Most production systems operate in this range.

T = 1.0–1.4: creative writing, brainstorming, poetry. Higher diversity, more surprising choices. Coherence may occasionally suffer.

T > 1.5: experimental. Useful for exploring the model's probability space, not for production output.

Temperature is not a global constant and it should vary by task. A system that writes creative fiction and also answers factual questions should use different temperatures for each, ideally set automatically based on query classification.

9.5 Top-k sampling

Temperature sampling from the full distribution can still sample very unlikely tokens. If there are 100,000 tokens in the vocabulary, even after temperature adjustment there may be thousands of tokens with non-negligible probability, including tokens that produce incoherent output.

Top-k sampling restricts sampling to the `k` most probable tokens:

```
def top_k_sample(logits, k, temperature=1.0):
    logits = logits / temperature
    top_k_values, top_k_indices = torch.topk(logits, k)
    filtered_logits = torch.full_like(logits, float('-inf'))
    filtered_logits.scatter_(0, top_k_indices, top_k_values)
    probs = torch.softmax(filtered_logits, dim=-1)
    return torch.multinomial(probs, num_samples=1)
```

The remaining probability mass is renormalized across the top-k tokens before sampling. Tokens outside the top-k are masked with `-inf`, giving them zero probability after softmax.

Top-k has a structural limitation: `k` is fixed regardless of the distribution shape. When the model is very confident (one token at probability 0.95, the rest near zero) top-k=50 forces sampling from 50 tokens when 49 of them are clearly wrong choices. When the model is genuinely uncertain (500 tokens each at probability ~0.002) top-k=50 discards 450 plausible options. A fixed `k` handles neither scenario well. This motivates nucleus sampling.

9.6 Nucleus (top-p) sampling

Nucleus sampling adapts the number of candidates to the distribution shape. Instead of a fixed count `k`, it takes the smallest set of tokens whose cumulative probability exceeds `p`:

```
def nucleus_sample(logits, p, temperature=1.0):
    logits = logits / temperature
    probs = torch.softmax(logits, dim=-1)
    sorted_probs, sorted_indices = torch.sort(probs, descending=True)
    cumulative_probs = torch.cumsum(sorted_probs, dim=-1)
    sorted_indices_to_remove = cumulative_probs - sorted_probs > p
    sorted_probs[sorted_indices_to_remove] = 0
    sorted_probs /= sorted_probs.sum()
    sampled_index = torch.multinomial(sorted_probs, num_samples=1)
    return sorted_indices[sampled_index]
```

With `p = 0.9`: include the fewest tokens whose probabilities sum to at least 0.9. Renormalize and sample from those tokens only.

When the model is confident (one token at 0.95 probability), the nucleus contains only that token which is effectively greedy decoding. When the model is uncertain (probability spread widely), the nucleus includes many tokens and sampling is diverse. The nucleus size adapts to the situation rather than being fixed in advance which makes it ideal for production.

9.6.1 Choosing p

p = 0.9–0.95: the most common range. Excludes the long tail of implausible tokens while preserving diversity for plausible choices.

p = 1.0: sample from the full distribution there aren't any nucleus restriction. Equivalent to pure temperature sampling.

p = 0.5: very concentrated. Appropriate for high-accuracy tasks but risks becoming too deterministic.

Top-p and temperature are typically used together. Temperature shapes the distribution; top-p determines the cutoff. A common production configuration: `temperature=0.7, top_p=0.9`.

9.7 Min-p sampling

A more recent alternative to top-p that addresses a subtle failure mode.

Top-p's cutoff is cumulative: if the nucleus already covers 90% of probability mass, tokens beyond that are excluded regardless of their absolute probability. In practice, this can include tokens with very low absolute probability if they accumulate to push the total past `p`, especially when the distribution is flat.

Min-p sets a minimum absolute probability threshold instead: only sample from tokens whose probability exceeds `min_p × max_token_probability`.

```
def min_p_sample(logits, min_p, temperature=1.0):
    probs = torch.softmax(logits / temperature, dim=-1)
    max_prob = probs.max()
    threshold = min_p * max_prob
    filtered_probs = probs.clone()
    filtered_probs[probs < threshold] = 0
    filtered_probs /= filtered_probs.sum()
    return torch.multinomial(filtered_probs, num_samples=1)
```

With `min_p = 0.05`: if the most likely token has probability 0.4, only tokens with probability ≥ 0.02 are considered. If the most likely token has probability 0.9, the threshold rises to 0.045 automatically restricting sampling when the model is confident.

Min-p adapts to distribution shape like top-p but uses the maximum token probability as the reference rather than the cumulative sum. Empirically, it produces more coherent output than top-p for creative tasks while maintaining diversity.

9.8 Repetition penalties

A common failure mode of sampling is that the model becomes trapped in repetitive loops. This issue is not solely caused by the decoding algorithm, it often reflects the model operating in a low-quality mode or producing a poorly calibrated probability distribution. Nevertheless, appropriate decoding strategies can help mitigate this problem.

9.8.1 Frequency penalty

Reduces the logit of any token proportional to how many times it has appeared in the generated output so far:

`adjusted_logit[t] = logit[t] - frequency_penalty × count(t, generated_so_far)`

A token that has appeared 5 times with `frequency_penalty = 0.5` has its logit reduced by 2.5. Frequently used tokens become progressively less likely.

9.8.2 Presence penalty

Reduces the logit of any token that has appeared at all, by a flat amount:

`adjusted_logit[t] = logit[t] - presence_penalty × (1 if t in generated_so_far else 0)`

Unlike frequency penalty, presence penalty does not scale with count, it applies a flat reduction on first appearance and no additional reduction after that. It discourages reusing any token at all, not repeatedly using a token many times.

9.8.3 Repetition penalty (multiplicative)

Used in HuggingFace and most open-source implementations:

```
adjusted_logit[t] = logit[t] / repetition_penalty    if logit[t] > 0
adjusted_logit[t] = logit[t] × repetition_penalty    if logit[t] < 0
```

For `repetition_penalty > 1.0`, this divides positive logits and multiplies negative logits, making previously generated tokens less likely regardless of their sign.

9.8.4 Practical considerations

Repetition penalties must be carefully calibrated. If the penalty is too strong, the model may avoid repeating necessary words, including common function words such as the, is, and and, which naturally occur multiple times in fluent text. Conversely, if the penalty is too weak, repetitive outputs may persist.

Most production systems apply penalties only to the generated portion, not the prompt, you do not want tokens appearing in the prompt to be penalized in the response. Context window position also matters: penalizing tokens from many turns ago that have no relationship to the current generation is rarely useful.

9.9 Structured decoding and constrained generation

For many production use cases, the output must conform to a specific format (a valid JSON, a Python function, a response matching, a defined schema). Unconstrained sampling may produce syntactically invalid output, requiring expensive retry loops. Structured decoding guarantees valid output by constraining the logits at each step.

9.9.1 How constrained generation works

At each decoding step, a parser tracks the current state of the partially generated output and computes the set of tokens that are valid at this position, tokens that would not cause the output to become unparseable. All invalid tokens are masked out (their logits set to `-inf`) before sampling.

```
Target format: JSON object with "name" (string) and "age" (integer)

After generating: {"name": "
  Valid next tokens:   any character that could appear in a JSON string
  Invalid next tokens: }, ], :, numbers, structural tokens

After generating: {"name": "Alice", "age":
  Valid next tokens:   digits (0-9), negative sign
  Invalid next tokens: letters, quotes, braces
```

The parser can be regex-based (valid tokens match the regex at the current position), grammar-based (tokens permitted by a context-free grammar at the current parse state), or schema-based (tokens consistent with a JSON schema or Pydantic model).

9.9.2 Implementations

Outlines (Willard & Louf, 2023): builds a finite state machine from the schema or regex at compile time. At inference, the FSM advances after each token and the valid next-token set is read from the FSM in O(1). The precomputed FSM makes constraint checking negligible overhead.

Guidance (Microsoft): the earlier widely-used structured generation library. Intercepts the generation loop and applies constraints at each step with a broader feature set including branching and conditional generation.

llama.cpp GBNF grammars: built-in grammar-based constrained generation using a BNF-like notation. Ships with llama.cpp without additional dependencies.

9.9.3 Overhead

Constrained generation adds overhead in logit masking (computing the valid token set and applying the mask which is O(1) per step with precomputed FSMs) and in reduced sampling diversity (smaller effective vocabulary). Latency overhead is typically 5–15% relative to unconstrained generation with FSM-based approaches. The alternative (parsing failures and retry loops) is far more expensive.

9.10 Best-of-n sampling

Generate `n` independent completions for the same prompt and select the best one according to some criterion:

```
Generate n=8 completions for the same prompt
Score each:
  - Reward model score (human preference prediction)
  - Verifier (run the code, check the math)
  - Self-consistency (most common answer across completions)
  - Model perplexity (model's own confidence in its output)
Return the highest-scoring completion
```

Best-of-n is a simple yet effective strategy for improving output quality by allocating additional computation during inference. The model generates n candidate outputs and selects the best one according to a scoring criterion. Performance typically scales log-linearly with n: increasing the number of samples from 1 to 4 often yields substantial improvements, whereas increasing it from 8 to 16 provides smaller, diminishing gains.

Verifiable tasks are where best-of-n works best. For math problems, the answer can be verified for correctness. For code, it can be executed and tested. Generate `n` solutions, keep those that pass verification, return the best or majority-vote answer.

Reward model reranking generates `n` responses and scores them with a reward model trained to predict human preference, effectively the RLHF reward signal applied at inference time rather than training time.

Self-consistency (Wang et al., 2023) generates `n` answers with chain-of- thought, returns the most common final answer. This improves accuracy on multi- step reasoning tasks significantly without requiring a separate verifier.

9.11 Speculative sampling

Autoregressive generation has an uncomfortable property: producing token 50 requires having already produced tokens 1 through 49, one at a time, each requiring a full forward pass through the model.

Speculative decoding is a way to avoid paying that tax token-by-token. A small, cheap **draft model** runs ahead and proposes a candidate sequence of `k` tokens. The target model then evaluates all `k` candidates in a *single* forward pass. The target model isn't generating here; it's grading a draft that's already been written.

Drafting versus grading raises the obvious question: how does the target model decide which of the draft's guesses to keep? The naive answer is a threshold: keep what the target model agrees with and discard the rest. This naive answer is wrong, and understanding *why* it's wrong is the key to understanding the whole algorithm.

9.11.1 The acceptance rule

For each draft token `x_t`, accept it with probability:

```
P(accept x_t) = min(1, p_target(x_t) / p_draft(x_t))
```

Two regimes:

- **The target model likes the token at least as much as the draft did** ($p_{target} \geq p_{draft}$): accept with probability 1.
- **The target model likes it less** ($p_{target} < p_{draft}$): accept only with probability p_{target}/p_{draft}. The draft model was, in effect, overconfident relative to the target, so the token survives only in proportion to how much the target actually endorses it.

9.11.2 What happens on rejection and why the obvious fix is wrong

Suppose a draft token is rejected. The intuitive move is to sample a replacement directly from `p_target` at that position after all, the target model already computed that full distribution during verification. This turns out to *distort* the final output, and a small numeric example makes the distortion concrete.

Take a two-token vocabulary, `{A, B}`, where:

```
p_target(A) = 0.5,  p_target(B) = 0.5
p_draft(A)  = 0.9,  p_draft(B)  = 0.1
```

The draft model samples from its own distribution, so it proposes A 90% of the time. When it does, the acceptance probability is `min(1, 0.5/0.9) = 0.556`. If we reject and then naively resample straight from `p_target`:

```
P(final = A) = P(draft=A)·P(accept) + P(draft=A)·P(reject)·P(naive resample = A)
             = 0.9 × 0.556 + 0.9 × 0.444 × 0.5
             = 0.5 + 0.2 = 0.7
```

The final probability of A comes out to **0.7**, even though the target model only wanted A 50% of the time. Token A is getting double-counted: it wins a first chance through acceptance, and then (because a naive resample doesn't know that) a second, independent chance through rejection. Any token the draft model over-favored relative to the target ends up over-represented in the output. This is precisely the kind of quality drift speculative decoding is supposed to avoid.

9.11.3 The residual distribution

The fix is to correct only into the probability mass that acceptance *hasn't already claimed*:

```
p_correction(x) = max(0, p_target(x) - p_draft(x)) / Z
```

This is **not** the target's raw distribution, it is the target's distribution with the draft's contribution subtracted out, clipped at zero, and renormalized by `Z`. Any token the draft over-proposed is zeroed out here, since it already had its fair shot via acceptance and cannot be picked a second time. Any token the draft under-proposed keeps some mass, since that's the only place where the target's true preference hasn't yet been accounted for.

Reworking the example: `p_correction(A) = max(0, 0.5 - 0.9) = 0`, so A can never be selected on correction. `p_correction(B) = max(0, 0.5 - 0.1) = 0.4`, and since it's the only token with nonzero mass, `Z = 0.4` and correction always yields B. Recomputing:

```
P(final = A) = 0.9 × 0.556 = 0.5                                matches p_target(A)
P(final = B) = 0.1 × 1 (direct accept) + 0.9 × 0.444 × 1 (correction) = 0.1 + 0.4 = 0.5   matches p_target(B
```

Both tokens land exactly on the target's true probabilities.

9.11.4 Walking through a full sequence

The acceptance/correction step happens *per position*, scanned left to right, and stops at the first rejection. Suppose the draft model proposes five tokens continuing "The cat sat on the":

```
Draft sequence:  x1="mat"   x2="and"   x3="looked"  x4="very"   x5="happy"
```

One target-model forward pass over the prompt plus all five draft tokens yields the target's distribution at every position simultaneously, thanks to causal masking position 1 conditioned only on the real prompt, position 2 on the real prompt plus `x1`, and so on.

```
Position 1: x1="mat"     p_draft=0.7, p_target=0.8  → accept (prob. 1)
Position 2: x2="and"     p_draft=0.5, p_target=0.5  → accept (prob. 1)
Position 3: x3="looked"  p_draft=0.6, p_target=0.2  → accept w.p. 0.33 → suppose REJECTED
```

Once position 3 is rejected, positions 4 and 5 are discarded without ever being checked `x4` and `x5` were drafted under the assumption that "looked" was the accepted prefix, and that assumption no longer holds. Position 3 is then resolved by drawing from the residual distribution at that position, which excludes "looked" (already rejected) and favors whatever the target under-weighted relative to the draft say this yields "leapt."

```
Accepted this round:   "mat", "and", "leapt"
Discarded:              "looked", "very", "happy"
```

Three tokens survive from five drafted, produced by a single target forward pass rather than three sequential ones. The next round starts fresh: the draft model proposes another `k` tokens continuing from "…sat on the mat and leapt," and the cycle repeats.

9.12 Chain-of-thought and reasoning tokens

A decoding-time technique that improves performance on reasoning tasks: before producing the final answer, the model generates intermediate reasoning steps.

```
Without chain-of-thought:
  Q: "If a train travels 120 miles in 2 hours, what is its speed?"
  A: "60 miles per hour."

With chain-of-thought:
  Q: "If a train travels 120 miles in 2 hours, what is its speed?"
  A: "To find speed, I divide distance by time.
      Speed = 120 miles / 2 hours = 60 miles per hour."
```

Chain-of-thought is a decoding strategy in that it requires generating more tokens before reaching the answer. The intermediate tokens are not just scaffolding, they change the probability distribution over the final answer. When the model writes out an intermediate calculation, that calculation is encoded into the KV cache. Subsequent tokens attend to it. The final answer is generated with the full reasoning chain as context, information that was not available at the first token.

This is in-context computation: the model uses its own generated text as working memory, offloading complex reasoning to the sequence rather than encoding it entirely within the forward pass. The context window is not just for input but also it is the model's scratch pad.

9.13 Decoding strategy selection guide

A practical reference for choosing decoding parameters by task:

Task	Strategy	Temperature	Top-p	Notes
Factual Q&A	Greedy or low-T	0–0.3	1.0	Consistency over diversity
Code generation	Low temperature	0.2–0.4	0.95	Correctness matters; some diversity for alternatives

Task	Strategy	Temperature	Top-p	Notes
JSON / structured output	Constrained	0.0–0.3	—	Use constrained decoding; redundant with high-T
Translation	Beam or low-T	0.1–0.3	—	Beam k=4 or low-T sampling
Summarization	Low-medium	0.3–0.6	0.9	Some diversity; avoid repetition penalties for common words
Conversation	Medium	0.7–0.9	0.9	Naturalness and variety
Creative writing	High	0.9–1.2	0.95	Maximize diversity; consider min-p
Reasoning / math	Very low or CoT	0.0–0.1	—	Greedy for extraction; chain-of-thought for reasoning
Brainstorming	High	1.0–1.4	0.95	Divergent thinking; coherence secondary

---

9.14 Key takeaways

- The model outputs a probability distribution over the vocabulary at each step; decoding selects a token from that distribution — this choice, made thousands of times, determines the character of the output
- Greedy decoding (always pick the most probable token) is deterministic and accurate for factual tasks but produces repetitive, generic output for open- ended generation because local optimality does not imply global optimality
- Beam search maintains `k` candidate sequences and returns the highest cumulative probability sequence; better than greedy for constrained tasks but suffers the beam search curse for open-ended generation — highest-probability most interesting
- Temperature scales logits before softmax: below 1.0 sharpens the distribution (more deterministic), above 1.0 flattens it (more random); vary temperature by task rather than using a single global value
- Top-p (nucleus sampling) adaptively samples from the smallest token set covering probability mass `p` — the nucleus shrinks when the model is confident, expands when uncertain; it is the dominant production sampling strategy
- Min-p uses the maximum token probability as a reference threshold rather than cumulative mass, producing more coherent output in cases where top-p includes low-probability tokens
- Structured decoding masks invalid tokens at each step using FSMs or parsers, guaranteeing format correctness with ~5–15% overhead — far cheaper than retry loops
- Best-of-n generates multiple completions and selects the best; most effective for verifiable tasks (math, code) where correctness can be checked programmatically
- Chain-of-thought uses the KV cache as working memory — intermediate reasoning tokens change the probability distribution over the final answer; reasoning models extend this to thousands of deliberation tokens
- Higher temperature increases hallucination rate; factual and structured tasks should use low temperature or constrained decoding to respect the model's already-correct probability signal

---

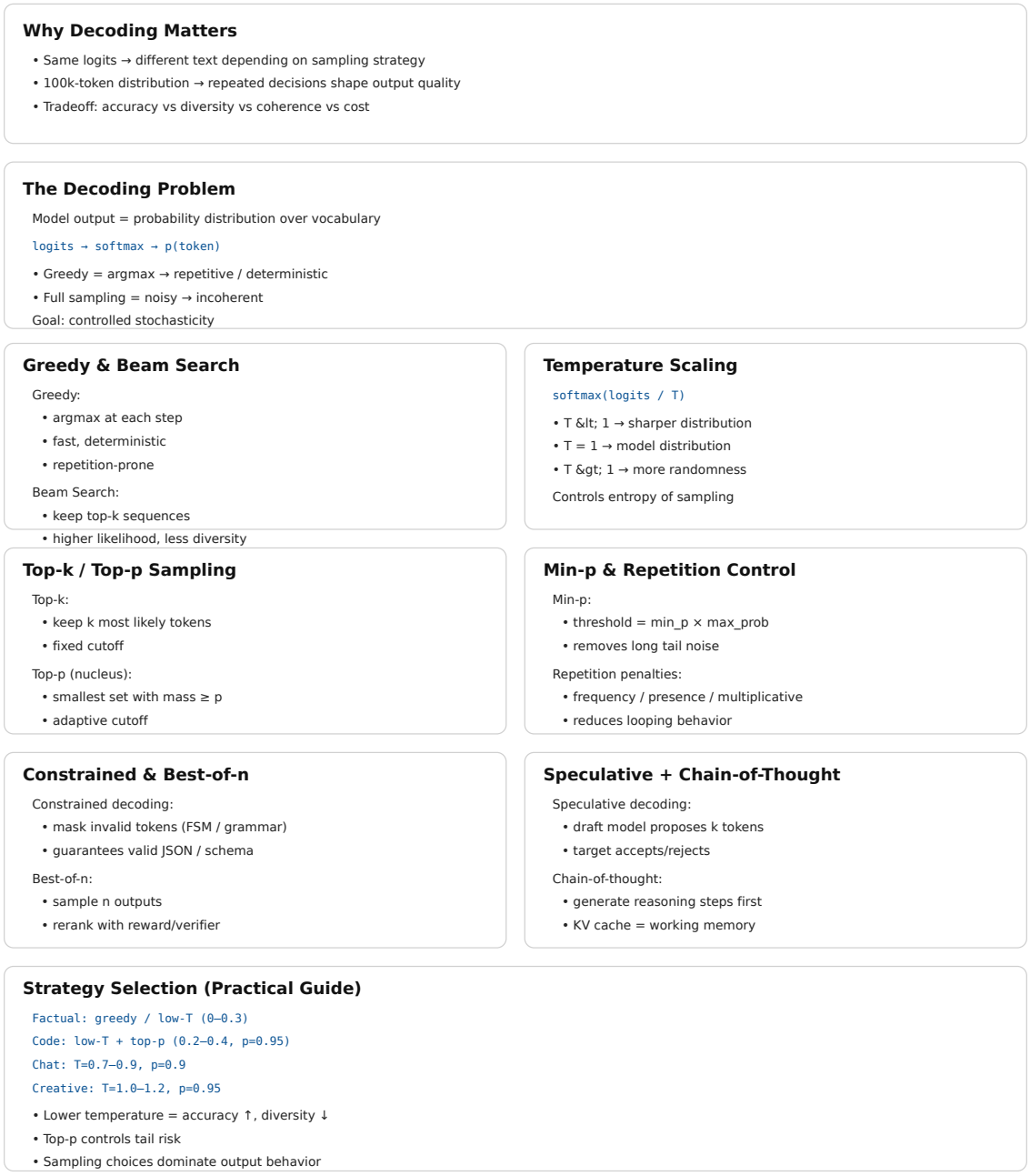


Figure 9.2: Cheat sheet.

9.15 Further reading

- Holtzman et al. (2020). *The Curious Case of Neural Text Degeneration.* — Established why greedy and beam search fail for open-ended generation and introduced nucleus sampling. The analysis of probability maximization failure modes is essential.
- Fan et al. (2018). *Hierarchical Neural Story Generation.* — Introduced top-k sampling for neural text generation; the original motivation and evaluation.
- Leviathan et al. (2023). *Fast Inference from Transformers via Speculative Decoding.* — Speculative sampling with exact distribution preservation; the proof that the correction step maintains the target distribution is the key contribution.
- Wei et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* — Established chain-of-thought as a decoding-time technique.
- Willard & Louf (2023). *Efficient Guided Generation for Large Language Models.* — The Outlines paper; FSM-based constrained decoding with O(1) per-step overhead.

Part IV

The Forbidden Archive

Chapter 10

Parametric Memory vs External Memory

The canonical question for this chapter: *A language model knows things. A retrieval system finds things. These are different cognitive acts and understanding exactly how they differ is what lets you design systems that use each one where it works best.*

Where are we?

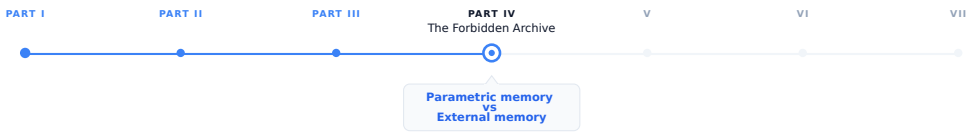


Figure 10.1: The journey through the Model Mind.

Retrieval exists because parametric memory has fundamental limits. This chapter explores those limits in greater depth by examining the nature of parametric memory, the role of external memory, how the two interact, and how to design systems that leverage both effectively.

10.1 Two different ways of knowing

Consider two employees at a company. The first has been there for twenty years. He knows the company's history, culture, and procedures intuitively. When someone asks a question, he answers from internalized understanding, he does not need to look anything up. Her knowledge is fast, fluid, and deeply integrated. It is also frozen: she knows things as they were when she last learned them, and it takes effort to update.

The second is a researcher who joined last week. She does not know the company's history, but she has access to every document the company has ever produced. When someone asks a question, she searches the archives, finds the relevant documents, and reads the answer. Her knowledge is current, precise, and verifiable but it depends entirely on what is in the archives and whether she can find it.

A language model with retrieval is both employees working together.

The first employee is **parametric memory** (knowledge encoded in the model's weights through training). The second is **external memory** (knowledge stored in a document corpus, retrieved at inference time). Neither is sufficient alone. Together, they cover each other's weaknesses.

10.2 What parametric memory is

Parametric memory is the knowledge encoded in a model's parameters basically its weights. It is everything the model learned by predicting the next token across a training corpus of trillions of words.

This knowledge is not stored as facts in labeled locations. It is distributed across billions of floating-point numbers, each contributing fractionally to the model's behavior. When the model "knows" that Paris is the capital of France, that knowledge is not a database entry, it is a statistical pattern spread across the embedding table, the attention heads, and the feed-forward layers of every transformer block.

10.2.1 Properties of parametric memory

Always available. Parametric memory requires no lookup. Every piece of knowledge the model has is accessible on every forward pass, with no retrieval latency.

Deeply integrated. Parametric knowledge is woven into the model's reasoning. When the model answers a question about French geography, it can simultaneously apply its knowledge of European history, political structures, and linguistic patterns without explicit retrieval of any of them. The integration is seamless because all of it lives in the same substrate.

Compressed and approximate. The training corpus contains far more information than fits in the parameter count. What gets encoded is statistical regularities, patterns that appeared consistently enough across training examples to survive gradient descent. Specific facts, especially rare ones, are encoded imprecisely. The model reconstructs them from learned patterns rather than retrieving them verbatim.

Fixed at training time. Parametric memory cannot be updated without retraining. Events after the training cutoff, information not in the training corpus, corrections to facts the model learned incorrectly, none of these can be incorporated without a new training run.

Opaque. There is no mechanism to ask parametric memory for its source. When the model produces a fact, it cannot say which document that fact came from. The provenance is lost in the compression.

Confident under uncertainty. When the model does not know something, its parametric memory does not respond with silence. It responds with whatever continuation is statistically plausible in context. This is hallucination, the model generating text that sounds like knowledge but is not grounded in actual facts.

10.3 What external memory is

External memory is a structured store of information that the model can query at inference time (a vector database, a document corpus, a knowledge graph, or a traditional database). It is external to the model: stored separately, maintained independently, and accessed through a retrieval interface.

In a RAG system, the external memory is typically a collection of documents chunked into passages, embedded as vectors, and stored in a vector database. A query arrives, it is embedded, the most similar passages are retrieved, and they are inserted into the prompt as context.

10.3.1 Properties of external memory

Updatable in real time. New documents can be added, old documents expired, and corrections made without touching the model's weights. The knowledge base can be current as of minutes ago.

Precise and verbatim. Retrieved documents contain exact text from the source. The model does not reconstruct the content, it reads it. Specific numbers, dates, names, and quotations can be retrieved and cited exactly as they appear in the source.

Verifiable and citable. Every retrieved passage has a source. The system knows which document a fact came from, can provide citations, and can display the source for user verification. The provenance is preserved.

Bounded by what was indexed. External memory can only surface what is in the corpus. Knowledge not indexed (information not added, documents not processed) is invisible to retrieval. The corpus boundary defines the limits of what can be found.

Quality-dependent on retrieval. If retrieval fails to surface the relevant passage, the model has no external knowledge to work with. A RAG system is only as good as its retrieval.

Slower than parametric access. External memory requires embedding the query, searching the index, and retrieving documents before the model can use the information. This adds latency (typically 50–500ms) that is absent for parametric knowledge.

10.4 The complementary structure

Parametric and external memory have complementary strength profiles. A well- designed system delegates to each where it excels:

	Parametric Memory	External Memory
Speed	Fast	Slower
Update cost	High	Low
Currency	Stale	Current
Coverage	Training data	Indexed corpus
Precision	Approximate	Exact
Verifiability	None	Full
Private data	No	Yes
Reasoning ability	High	None (the model reasons)
Integration depth	Deep	Shallow
Failure mode	Hallucination	Retrieval miss

10.5 When to rely on parametric memory

Certain categories of knowledge are better served by parametric memory than by retrieval.

10.5.1 General reasoning and language

Grammar, syntax, logical inference, mathematical operations, programming patterns, writing style, none of these are facts to be retrieved. They are capabilities that emerged from training and are deeply integrated into the model's generation process. Retrieval cannot help with "write a recursive function" in the way it helps with "what is the current interest rate."

10.5.2 Stable, well-documented knowledge

The capital of France, the speed of light, the date of World War II, the syntax of Python, facts that are stable, widely documented, and unlikely to have changed since training. For these, retrieval adds latency and complexity without adding information. The model's parametric memory is sufficient.

10.5.3 Synthesis and inference

When a question requires combining multiple pieces of knowledge in ways that are not explicitly stated in any document, parametric memory's deep integration is an advantage. "What are the implications of X for Y?" may not have a document that answers it directly. The model's ability to reason across integrated knowledge (drawing on history, economics, and science simultaneously) is a parametric capability.

10.5.4 Common knowledge about common things

Facts about widely documented subjects that appeared many times in the training corpus are more reliably stored in parametric memory than rare or specialized facts. The model's reconstruction of common knowledge is accurate; its reconstruction of rare knowledge is not.

10.6 When to rely on external memory

10.6.1 Current information

Anything that changes after the training cutoff: news, prices, personnel, policies, software versions, research findings. Parametric memory cannot help here. External memory with regular updates is the only option.

10.6.2 Private and proprietary data

Enterprise documents, customer records, internal policies, proprietary research, none of this is in the training corpus. Retrieval is the only mechanism to give the model access to organization-specific knowledge.

10.6.3 Precise facts and citations

Specific numbers, quotations, contractual terms, regulatory requirements, cases where "approximately right" is not acceptable. External memory provides the exact text; parametric memory approximates.

10.6.4 Accountability and auditability

Cases where the source of information matters, legal, medical, financial, regulatory. External memory provides citable sources. Parametric memory provides assertions without provenance.

10.6.5 Long-tail and specialized knowledge

Domain-specific jargon, rare technical standards, specialized procedures, topics underrepresented in the training corpus. The model's parametric knowledge of these is unreliable. A well-indexed specialized corpus outperforms parametric reconstruction for niche domains.

10.7 The interaction between the two

When both memories are available (a model with access to a retrieval system) they interact in ways that require careful system design.

10.7.1 Which memory does the model prefer?

Research shows that the model's preference for parametric vs. retrieved knowledge depends on the relevance and quality of the retrieved context:

- When retrieved context is highly relevant and clearly addresses the question, the model generally uses it

- When retrieved context is irrelevant or contradicts the model's parametric memory, the model may ignore the retrieved content and answer from parametric knowledge
- When parametric memory is strong (the subject was well-represented in training), the model may underweight retrieved context even when it is relevant

This is not a binary choice the model makes consciously. It is an emergent property of how attention weights retrieved context against parametric associations. The model does not "decide" to trust one over the other it attends to the full context and generates the most likely continuation.

10.7.2 Parametric memory as noise

A counterintuitive failure mode: for questions where the correct answer is in the retrieved context, a model with strong parametric associations to a different answer may generate the wrong answer despite the retrieved evidence.

Example: a company changed its refund policy. The new policy is in the retrieved document. The old policy appeared many times in the training data. The model may generate the old policy from parametric memory rather than the new one from the retrieved document.

This is one of the strongest arguments for grounded generation techniques explicitly instructing the model to prefer retrieved context over internal assumptions. But even with careful prompting, the failure mode persists for strongly-encoded parametric beliefs.

10.7.3 Parametric memory as context interpreter

The model's parametric knowledge is also what allows it to use retrieved context intelligently. When a retrieved document contains technical jargon, the model can interpret it because of parametric knowledge of the domain. When a document is ambiguous, the model uses parametric context to resolve the ambiguity. External memory provides the raw content; parametric memory provides the interpretive framework.

A model with poor parametric knowledge of a domain will retrieve relevant documents and still answer poorly, because it cannot reason about the content effectively. This is why domain-specific fine-tuning (to improve parametric knowledge of the domain) and domain-specific retrieval are complementary rather than substitutes.

10.8 Forms of external memory beyond RAG

RAG (retrieving passages from a vector database) is one form of external memory. Others exist and are increasingly used in production systems.

10.8.1 Structured databases

SQL databases, key-value stores, and graph databases provide structured external memory. A model with tool use can query a SQL database to retrieve exact records (sales figures, customer attributes, product specifications) with guarantees of precision that vector search cannot provide.

For questions with exact, structured answers, a database query is superior to vector search: "What was the revenue in Q3 2024?" is better answered by a database query than by retrieving a passage that mentions Q3 revenue.

10.8.2 Episodic memory stores

For agentic systems that operate over extended periods, episodic memory stores record what the agent has done, observed, and learned in previous sessions. This is a form of external memory that accumulates over time and can be retrieved to inform current decisions.

10.8.3 Tool-accessed knowledge

Search engines, APIs, calculators, and code executors are forms of external memory accessed through tools rather than vector retrieval. A model that can search the web has access to a form of external memory that is more current and more comprehensive than any prebuilt corpus. The tradeoff is latency, reliability, and the need to process retrieved results rather than receiving them in a controlled format.

10.8.4 The knowledge graph

Knowledge graphs represent facts as typed relationships between entities: (Paris, capital_of, France), (France, located_in, Europe). They are more structured than document corpora and support logical inference over stored relationships.

10.9 Designing the memory split

For any application, the decision of what goes in parametric memory and what goes in external memory is an architectural one that affects quality, cost, maintainability, and update frequency.

10.9.1 The update frequency heuristic

Knowledge that changes frequently → external memory. Knowledge that is stable → parametric memory (or cached retrieval).

A customer support application for a product with rapidly evolving features needs product documentation in external memory which changes with every release. The general reasoning and communication style it needs from the model is parametric.

10.9.2 The precision heuristic

Knowledge that must be exact (legal terms, financial figures, technical specs) → external memory. Knowledge that can be approximate (general explanations, background context) → parametric memory.

10.9.3 The privacy heuristic

Knowledge that is confidential or proprietary → external memory with access controls. Knowledge that is public and stable → parametric memory is fine.

10.9.4 The volume heuristic

A small number of highly-accessed facts can be effectively encoded through fine-tuning (moving them into parametric memory). A large, diverse, or growing corpus must go in external memory, there is no practical way to encode millions of documents into model weights through fine-tuning.

10.9.5 The accountability heuristic

When the source of information must be traceable → external memory. When the answer is general knowledge without attribution requirement → parametric memory.

10.10 The emerging continuum

The sharp distinction between parametric and external memory is increasingly a simplification. Several techniques blur the boundary:

Fine-tuning for domain knowledge moves external knowledge into parametric memory through additional training. A model fine-tuned on a company's internal documentation partially encodes that documentation into its weights. The result is faster access and deeper integration, but at the cost of update agility.

Memory-augmented models like RETRO (Borgeaud et al., 2022) integrate retrieval directly into the model architecture, retrieved documents are processed by dedicated cross-attention layers rather than being inserted into the context. The line between the model's parameters and the retrieval corpus is architecturally blurred.

Caching retrieved content in the context creates a form of working memory that persists across turns, more dynamic than parametric memory (it can change with each session) but more structured than ad-hoc retrieval (it is explicitly managed). Long-context models with large KV caches make this increasingly practical.

Continuous learning progressively updates parametric memory from new data, approaching the update agility of external memory while maintaining the access speed of parametric memory. This remains computationally expensive but is an active research direction.

10.11 Key takeaways

- Parametric memory is knowledge encoded in model weights — always available, deeply integrated, fast, but fixed at training time, approximate, and opaque
- External memory is knowledge stored in a retrieval corpus — current, precise, verifiable, and updatable, but dependent on retrieval quality and slower to access
- The two memory types have complementary strength profiles: parametric memory handles reasoning, stable knowledge, and synthesis; external memory handles current information, private data, precise facts, and citable sources
- When both are available, the model does not make a binary choice — it attends to both, and strong parametric associations can override retrieved evidence even when retrieval is correct; grounded generation techniques address but do not fully eliminate this
- Structured databases, episodic memory stores, tool-accessed knowledge, and knowledge graphs are forms of external memory beyond vector search, each better suited to different knowledge types
- The decision of what goes in parametric vs. external memory is an architectural one driven by update frequency, required precision, privacy constraints, volume, and accountability requirements
- The boundary is blurring: fine-tuning moves external knowledge into parametric memory; memory-augmented architectures integrate retrieval into the model; long-context caching creates working memory that spans the two

---

10.12 Further reading

- Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* — The original RAG paper; frames the parametric/non-parametric distinction explicitly and demonstrates complementarity.
- Borgeaud et al. (2022). *Improving Language Models by Retrieving from Trillions of Tokens (RETRO).* — Memory-augmented architecture integrating retrieval into the model rather than the context.
- Mallen et al. (2023). *When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories.* — Empirical study of when models rely on which memory type and when each is more accurate.
- Shi et al. (2023). *Large Language Models Can Be Easily Distracted by Irrelevant Context.* — Retrieved context is not always used; irrelevant retrieval can degrade performance below the parametric-only baseline.
- Guu et al. (2020). *REALM: Retrieval-Augmented Language Model Pre-Training.* — Integrating retrieval into pre-training itself; the predecessor to RAG.

Chapter 11

Document Ingestion

The canonical question for this chapter: *You upload a PDF, a Word document, a webpage, or a database export. Before any of it can be retrieved or reasoned over, it must be transformed into something a retrieval system can work with. What actually happens during that transformation?*

Where are we?

Figure 11.1: The journey through the Model Mind.

Before a document can be retrieved, it must be ingested, parsed, cleaned, structured, and prepared for the pipeline that follows. This is where most RAG systems fail, and it happens long before any query is ever issued.

11.1 The ingestion problem

A language model's knowledge is frozen at training time. It knows what was in its training data and nothing more. For most production applications, this is insufficient, users ask about documents the model has never seen, databases that change daily, proprietary internal knowledge that was never part of any public corpus.

Retrieval-Augmented Generation solves this by giving the model access to external knowledge at inference time. But before any document can be retrieved, it must be ingested: parsed, cleaned, structured, and stored in a form that retrieval systems can query efficiently.

Ingestion is the foundation of every RAG system and where most RAG failures actually originate. It is not in the retrieval algorithm or the language model, but in poor parsing, inadequate cleaning, or naive chunking that destroys the coherence of the source material. Getting ingestion right is a prerequisite for optimizing anything else.

11.2 Document formats and their challenges

The first step in ingestion is parsing: extracting usable text from whatever format the document arrives in. Different formats present different challenges.

11.2.1 PDF

The most common and most problematic format. PDF is a presentation format, not a semantic one. It describes where each character is placed on a page, not what structure the document has.

A PDF does not have "paragraphs" or "sections" in any structural sense. It has positioned text elements, each with a font, size, and (x, y) coordinate. Reconstructing reading order, identifying headings, distinguishing body text from captions, and separating columns requires heuristics that work well on simple documents and poorly on complex ones.

Specific PDF challenges:

Multi-column layouts. Text from different columns is interleaved in the raw extraction order. Naive extractors produce text that mixes columns: "The company revenue | left column second line | right column second line | grew significantly." Correct column separation requires spatial analysis.

Tables. Tabular data in PDFs is often just positioned text with no structural information. Extractors must infer table structure from spatial relationships between text elements. This fails on complex merged cells, rotated headers, and tables that span pages.

Scanned PDFs. PDFs that contain scanned images rather than text contain no extractable text at all, what appears as "text" is actually just pixels. Extraction requires OCR, which adds its own error rate. A 98% OCR accuracy rate sounds high until you realize it means roughly one error every fifty words in a dense technical document.

Headers and footers. Page numbers, document titles, and section headers in running headers and footers are often extracted inline with body text. A 100-page document might have "CONFIDENTIAL Page N of 100" injected into the text every few paragraphs.

Mathematical notation. Equations in PDFs are often stored as positioned symbols rather than structured mathematical expressions. Extraction typically produces garbled sequences of symbols.

PDF libraries for Python:

- **PyMuPDF (fitz)**: fast, good text extraction for standard PDFs, reasonable layout analysis
- **pdfplumber**: better table extraction than PyMuPDF, slower
- **pypdf**: pure Python, simpler, adequate for basic extraction
- **Adobe PDF Extract API**: commercial, highest quality for complex layouts

For high-stakes RAG applications (legal documents, financial filings, technical documentation) investing in high-quality PDF parsing pays off significantly. A poorly parsed PDF produces retrieval failures that are invisible at the extraction stage but manifest as hallucinations and incorrect answers at inference time.

11.2.2 Word documents (DOCX)

Structurally richer than PDF. DOCX files have an explicit document model: paragraphs, headings (with levels), tables (with cell structure), lists, and styles. Extraction can respect this structure rather than reconstructing it from layout.

Challenges: embedded objects (images, embedded PDFs, charts) are opaque blobs whose content is inaccessible without separate processing. Tracked changes and revision marks can clutter extracted text.

11.2.3 HTML and web pages

HTML is semantically rich when used correctly. Heading tags (`<h1>` through `<h6>`), paragraph tags, list tags, table tags, and semantic elements like `<article>`, `<section>`, and `<nav>` provide structural information that text extractors can use.

In practice, web HTML is rarely so clean. Navigation menus, cookie banners, advertisements, and footers must be removed. JavaScript-rendered content requires a headless browser to extract.

11.2.4 Plain text and Markdown

The easiest cases. Text files require no parsing; they are read directly. Markdown adds lightweight semantic structure (headings via `#`, code blocks via backticks, lists) that parsers can respect.

Markdown is increasingly common as a RAG source format because it is both human-readable and machine-parseable. Documents intentionally written for RAG ingestion are sometimes authored in Markdown for this reason.

11.2.5 Spreadsheets (XLSX, CSV)

Tabular data presents a different challenge: how do you convert a table into text that can be retrieved and understood by a language model?

Options:

- **Row-by-row serialization**: "Row 3: Name=Alice, Age=32, Department=Engineering, Salary=95000"
- **Markdown table format**: convert to a Markdown table that the LLM can parse
- **Schema + sample rows**: describe the schema and include representative examples; retrieve the schema for schema-level questions and specific rows for value-level questions

For large spreadsheets (thousands of rows), row-by-row serialization produces too much text to embed meaningfully. SQL or structured databases with text-to-SQL generation are often a better architecture than RAG for tabular data. Retrieval finds relevant rows; the model generates SQL; the database executes it exactly.

11.2.6 Images and multimodal documents

Documents with meaningful visual content (diagrams, charts, photographs with captions, infographics) require multimodal processing.

OCR extracts text from images. Tesseract is the standard open-source OCR library; AWS Textract, Google Cloud Vision, and Azure Computer Vision are commercial alternatives with higher accuracy on complex layouts.

Vision-language models can describe image content, answer questions about figures, and extract structured information from charts. For documents where figures carry significant information, routing image content through a VLM before ingestion produces much better retrieval coverage than ignoring figures or relying on captions alone. A technical diagram with no caption is invisible to text-based retrieval without VLM description.

11.3 Text cleaning

After parsing, raw extracted text often contains artifacts that reduce retrieval quality and cleaning is used to remove or normalize them.

11.3.1 Common cleaning steps

Whitespace normalization. Collapse multiple spaces to one, remove leading/trailing whitespace from paragraphs, normalize line endings. Necessary because PDF extraction produces variable whitespace depending on character positioning.

Header and footer removal. Identify and remove page numbers, document titles repeated on each page, copyright notices, and other non-content text. Pattern matching (regex for "Page N of M") handles simple cases and spatial analysis during PDF parsing handles complex ones.

Hyphenation repair. PDF extraction often breaks hyphenated words at line breaks: `trans-\naction` appears as two tokens rather than "transaction". Detecting and joining these is straightforward for end-of-line hyphens but ambiguous for legitimate compound hyphenation.

Encoding normalization. Documents from multiple sources may have different encodings (UTF-8, Latin-1, Windows-1252). Normalize to UTF-8. Handle common encoding artifacts: curly quotes that appear as `â€œ`, em dashes that appear as `â€"`.

Boilerplate removal. Repeated passages that appear across many documents (terms of service, disclaimer language, standard legal notices) add noise to embeddings and can dominate retrieval for questions that accidentally match boilerplate patterns. Deduplication at the passage level or explicit blocklisting removes this.

Table of contents removal. Tables of contents in PDFs and Word documents are often extracted as body text, producing a list of section titles that duplicates content the model will encounter in the actual sections. Identifying and removing ToC passages prevents this duplication from dominating retrieval.

Reference list handling. Bibliographies and reference lists at the end of academic papers are often extracted as body text. Whether to include or exclude them depends on whether the references themselves are useful for retrieval. For scientific RAG, including references can help answer "what papers support this claim" questions.

11.3.2 Cleaning for specific domains

Legal documents. Case citations, statutory references, and defined terms have specific patterns. Preserving these patterns rather than normalizing them is important for legal search. Removing citation formatting destroys the semantic signal that distinguishes a reference to a case from a discussion of it.

Medical records. PHI (Protected Health Information) removal is legally required before storing patient records in retrieval systems. Names, dates, locations, and identifiers must be de-identified before ingestion. This is not optional and not a post-processing step, it must happen before any content is stored.

Code. Programming language syntax should not be normalized like prose. Tab/space preservation matters for Python. Comment blocks, docstrings, and variable names are semantically important and should not be striped. Code is often better chunked by function or class boundary than by token count.

Financial filings. Numerical tables in SEC filings have specific conventions (negative numbers in parentheses, thousands vs. millions) which should be preserved or explicitly normalized consistently across the corpus.

11.4 Document metadata extraction

Every ingested document should be accompanied by metadata that enables filtering, ranking, and attribution during retrieval.

11.4.1 Core metadata fields

```
{
    "document_id": "uuid-abc123",
    "source_url": "https://example.com/docs/api-reference.pdf",
    "source_type": "pdf",
    "title": "API Reference Guide v3.2",
    "author": "Engineering Team",
    "created_at": "2024-01-15T10:30:00Z",
    "modified_at": "2024-03-22T14:15:00Z",
    "language": "en",
    "page_count": 48,
    "word_count": 23500,
```

```
    "section": "Technical Documentation",
    "tags": ["api", "reference", "v3"],
    "access_level": "internal",
}
```

11.4.2 Why metadata matters

Filtering. Retrieval can be limited to documents matching metadata criteria. "What does our API documentation say about authentication?" should retrieve from technical documentation, not from marketing materials or HR policies. Without metadata filtering, vector similarity is the only selection mechanism, and it will surface whatever is most semantically similar regardless of source.

Recency ranking. For questions about current state, recently modified documents should rank higher. Metadata enables this without modifying embeddings.

Access control. Metadata carries authorization information. Users should not retrieve documents their access level does not permit, even if those documents are semantically relevant to their query. Access control must be enforced at retrieval time, not just at ingestion time.

Attribution. When the model uses retrieved content to generate a response, metadata provides the citation: "According to the API Reference Guide v3.2, section 4…" Users who need to verify information can follow the citation to the source.

11.4.3 Automatic metadata extraction

Some metadata fields must be inferred rather than extracted directly:

Language detection. langdetect and fastText language identification models classify language from text samples. Important for multilingual corpora where language-specific retrieval is needed, you do not want a French query retrieving German documents.

Topic classification. Embedding the document title and first paragraph and comparing to topic embedding clusters can assign broad topical categories. Useful for filtering and for routing queries to the right sub-corpus.

Entity extraction. Named entity recognition (NER) identifies people, organizations, locations, and other entities mentioned in the document. These can be stored as metadata fields enabling entity-based filtering: "find documents that mention Acme Corp."

11.5 The ingestion pipeline assembled

Putting the components together, a production document ingestion pipeline:

```
Raw document (PDF, DOCX, HTML, etc.)


Format detection


Parser selection and execution
     PDF: PyMuPDF / pdfplumber / LlamaParse
     DOCX: python-docx
     HTML: trafilatura + BeautifulSoup
     Image: OCR + VLM captioning
     Plain text: direct read
```

```
Raw text + structural information
  (paragraphs, headings, tables, page numbers)


Text cleaning
     Whitespace normalization
     Header/footer removal
     Encoding normalization
     Boilerplate removal
     Domain-specific cleaning


Metadata extraction
     Core fields (title, source, timestamps)
     Language detection
     Entity extraction (optional)


Chunking


Embedding


Vector store + metadata store
```

11.5.1 Idempotency and incremental updates

A production ingestion pipeline runs continuously as new documents arrive and existing documents are updated. Two properties are essential:

Idempotency. Ingesting the same document twice should produce the same result as ingesting it once. Without idempotency, re-running the pipeline on a partially-updated corpus duplicates documents and corrupts the retrieval index. Implementation: checksums enable detection of already-ingested documents. If a document's checksum matches an existing entry, skip re-ingestion. If the checksum differs (document has been updated), delete the old chunks and embeddings and re-ingest.

Atomic updates. When a document is updated, the retrieval index should not be in a mixed state (some chunks from the old version, some from the new). Atomic updates require: ingesting the new version to a staging area, deleting all old chunks and embeddings for this document ID, and promoting the new chunks and embeddings from staging to production. These steps are typically transactional in a properly designed metadata store.

11.5.2 Error handling and monitoring

Document ingestion fails silently in ways that are hard to detect. A document that fails to parse produces no chunks and no embeddings, it simply does not exist in the retrieval index. Users asking questions about that document get answers based on whatever other documents are retrieved instead. Without logging and monitoring, these failures are invisible.

Instrument the pipeline to track: documents processed, failed, and skipped per run; parse failures by format and error type; quality gate rejection rates; embedding failures; and processing latency by document type and size.

Alert on anomalies: if the PDF parse failure rate suddenly spikes, the parser library may have a bug or the input format may have changed. If the quality gate rejects an unusually large fraction of documents, the source may be producing degraded content.

11.6 Ingestion at scale

For large corpora (millions of documents) the ingestion pipeline must be designed for scale from the start.

11.6.1 Parallelization

Document parsing is CPU-bound and embarrassingly parallel, each document can be processed independently. Use a task queue (Celery, Ray, AWS SQS + Lambda) to distribute documents across multiple workers. Each worker handles parsing, cleaning, chunking, and embedding for its assigned documents.

The embedding step often calls an external API (OpenAI, claude). API rate limits impose a ceiling on parallelism. Design the pipeline to batch embedding requests at the maximum allowed batch size and implement rate limit handling with exponential backoff.

11.6.2 Incremental ingestion

For corpora that change frequently, full re-ingestion on every update is too slow. Incremental ingestion processes only new and modified documents:

1. Maintain a manifest of all ingested documents and their checksums
2. On each run, compare new document checksums to the manifest
3. Process only documents with new or changed checksums
4. Update the manifest after successful ingestion

For web crawls, compare last-modified headers or ETags to detect changes without re-downloading content.

11.6.3 Storage tiering

Ingested documents at various stages should be stored durably:

- **Raw documents**: store the original files in object storage (S3, GCS). Enables re-ingestion if the pipeline changes without retrieving the original from the source.
- **Parsed text**: store the cleaned extracted text alongside metadata. Enables re-chunking and re-embedding without re-parsing.
- **Chunks**: store chunk text and metadata. Enables re-embedding without re-chunking.
- **Embeddings**: stored in the vector database. The final retrieval index.

Each tier enables rerunning from that point forward without rerunning earlier stages. When you improve your embedding model, you only need to re-embed, not re-parse. When you improve your chunking strategy, you only need to re-chunk and re-embed, not re-parse. This separation pays off every time a component of the pipeline changes.

11.6.4 Versioning the pipeline

The ingestion pipeline is code that runs over data to produce embeddings. Changes to any component (parser version, cleaning rules, chunking strategy, embedding model) invalidate some or all of the stored artifacts.

Track the pipeline version with each stored artifact. When the pipeline version changes, identify which artifacts need to recompute. This is analogous to dependency tracking in build systems and is essential for maintaining a coherent retrieval index. Without it, a corpus may silently contain chunks embedded with an old model alongside chunks embedded with a new one, making similarity comparisons meaningless.

11.7 Key takeaways

- Document ingestion is the foundation of every RAG system; most RAG failures originate in poor parsing, inadequate cleaning, or naive structure destruction rather than in retrieval algorithms or language models

- PDF is the most common and most problematic format: a presentation format with no semantic structure, requiring heuristic reconstruction of reading order, headings, columns, and tables; for high-stakes applications, invest in high-quality PDF parsing
- Different formats require different parsers and present different challenges; text files and Markdown are easy; scanned PDFs and multi-column layouts are hard
- Text cleaning removes artifacts introduced by parsing — whitespace, headers and footers, encoding issues, boilerplate — that degrade retrieval quality without conveying content; domain-specific cleaning (PHI removal, citation preservation, code formatting) requires domain knowledge
- Metadata is not optional: it enables filtering by source, date, and access level; provides attribution for citations; and allows quality gating and deduplication before embedding
- A production ingestion pipeline must be idempotent (re-running is safe) and support atomic updates (document updates replace all old chunks, not mix old and new)
- Silent failures — documents that fail to parse or are rejected by quality gates — are invisible without explicit logging and monitoring; instrument the pipeline before you need to debug it
- At scale, ingestion must be parallelized, incremental, and tiered: store artifacts at each stage so that pipeline improvements can be applied from that stage forward without rerunning from scratch
- Track pipeline versions with stored artifacts; a corpus that mixes chunks embedded with different models has corrupted similarity comparisons

---

11.8 Further reading

- Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* — The foundational RAG paper; establishes the ingestion → retrieval → generation framework.
- Edge et al. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization.* — GraphRAG; extends ingestion to extract entities and relationships, not just text chunks.
- Unstructured.io (2023). *Unstructured: Document parsing for LLM applications.* — The leading open-source document parsing library for RAG; handles many formats with layout analysis.
- Shi et al. (2023). *REPLUG: Retrieval-Augmented Black-Box Language Models.* — Analysis of how retrieval quality affects end-to-end RAG performance; ingestion quality is the upstream dependency.

Chapter 12

Chunking Strategies

The canonical question for this chapter: *A document might be 50,000 words while an embedding model can process 512 tokens. A retrieved context window might be limited to 5,000. How do you divide the document into pieces that are retrievable, coherent, and useful to the language model?*

Where are we?

Figure 12.1: The journey through the Model Mind.

Document ingestion produced clean text. Now that text must be divided into retrievable chunks which is the most consequential preprocessing decision in a RAG system. Poor chunking produces retrieval failures that no downstream component can fix.

12.1 Why chunking exists and why it is hard

Embedding models have a maximum input length which is typically 512 to 8,192 tokens. Embedding an entire document as a single vector loses granularity: a 10,000-word technical manual embedded as one vector will match queries about any topic covered in the manual, making retrieval imprecise. You want the chunk about authentication to be retrieved for authentication questions and the chunk about rate limiting to be retrieved for rate limiting questions, not the entire manual for both.

At the same time, chunks must be semantically coherent. A chunk that cuts mid-sentence, mid-argument, or mid-table is harder for the language model to use correctly. The model receives a fragment without the context that makes it interpretable.

The fundamental tension: small chunks are precise for retrieval but too small to be self-contained. Large chunks are self-contained but imprecise for retrieval. The right chunk size is a function of the document type, the query distribution, the embedding model, and the language model's context window.

Getting chunking wrong is one of the most common RAG failure modes. A retrieval system with perfect relevance

ranking that retrieves incoherent chunks will produce worse answers than a system with imperfect ranking that retrieves coherent chunks. The language model cannot compensate for fragmented context.

12.2 Fixed-size chunking

The simplest strategy: split the document into chunks of a fixed number of tokens, with an optional overlap between adjacent chunks.

```
def fixed_size_chunk(text: str, chunk_size: int, overlap: int) -> list[str]:
    tokens = tokenize(text)
    chunks = []

    start = 0
    while start < len(tokens):
        end = min(start + chunk_size, len(tokens))
        chunk_tokens = tokens[start:end]
        chunks.append(detokenize(chunk_tokens))
        start += chunk_size - overlap

    return chunks
```

For `chunk_size=512` and `overlap=50`: chunk 1 covers tokens 0–511, chunk 2 covers tokens 462–973, chunk 3 covers tokens 924–1435. Each chunk overlaps with its neighbors by 50 tokens.

12.2.1 Why overlap exists

Without overlap, a sentence that straddles a chunk boundary is split: the first half is at the end of one chunk, the second half at the start of the next. Neither chunk contains the complete sentence, and a query that matches the complete sentence may not match either fragment.

Overlap ensures that every point in the document appears in at least one complete chunk. For overlap of `k` tokens, any contiguous span of `chunk_size - k` tokens or fewer is guaranteed to appear entirely within at least one chunk.

The cost of overlap: the same content is embedded and stored multiple times, increasing storage and compute cost. An overlap of 10% (50 tokens on a 512- token chunk) increases storage by approximately 10% which is a small price for significantly better boundary coverage.

12.2.2 When fixed-size chunking is appropriate

Fixed-size chunking is fast, simple, and predictable. It works reasonably well for homogeneous documents without meaningful structure (transcripts, continuous prose), corpora where you cannot rely on structural markers (inconsistent heading styles, poorly formatted PDFs), and prototyping.

It is inappropriate for documents with meaningful structure (technical documentation, legal contracts) where semantic boundaries do not align with token count boundaries, for tables and code which should not be split arbitrarily, and for documents where context continuity is critical.

12.2.3 Token-based vs. character-based splitting

Splitting on token count is more precise than splitting on character count because token count directly maps to embedding model input length. Character- based splitting produces chunks of unpredictable token length, a chunk of 2,000 characters might be 400 tokens for English prose or 700 tokens for code with many short tokens.

12.3 Sentence-based chunking

Instead of splitting at fixed token counts, split at sentence boundaries. This preserves semantic units and avoids splitting mid-sentence.

```
import spacy

nlp = spacy.load("en_core_web_sm")

def sentence_chunk(text: str,
                   max_tokens: int,
                   overlap_sentences: int) -> list[str]:
    doc = nlp(text)
    sentences = [sent.text.strip() for sent in doc.sents]

    chunks = []
    current_chunk = []
    current_length = 0

    for sentence in sentences:
        sentence_tokens = len(tokenize(sentence))

        if current_length + sentence_tokens > max_tokens and current_chunk:
            chunks.append(" ".join(current_chunk))
            current_chunk = current_chunk[-overlap_sentences:]
            current_length = sum(len(tokenize(s)) for s in current_chunk)

        current_chunk.append(sentence)
        current_length += sentence_tokens

    if current_chunk:
        chunks.append(" ".join(current_chunk))

    return chunks
```

Sentence detection requires a sentence boundary detector, spaCy, NLTK's Punkt tokenizer, or regex-based heuristics for simple cases. Sentence detectors are imperfect: abbreviations ("Dr.", "U.S."), decimal points, and ellipses can confuse them.

Sentence-based chunking works well for dense prose where individual sentences carry complete thoughts. It works poorly for bullet points and numbered lists (items may not be complete sentences), tables (rows are not sentences), code (statements are not sentences in the linguistic sense), and very long sentences that exceed the chunk size on their own.

12.4 Recursive character text splitting

LangChain's `RecursiveCharacterTextSplitter` is the most widely used chunking implementation in production RAG systems. It splits on a hierarchy of separators, trying each in order until chunks are within the size limit:

```
Separator hierarchy (default):
  1. "\n\n"   (paragraph break)
  2. "\n"     (line break)
  3. " "      (word boundary)
  4. ""       (character)
```

The algorithm tries to split on `"\n\n"`. If all resulting pieces are within the size limit, done. If some pieces are still too large, it recursively splits those on `"\n"`. If still too large, on `" "`. If still too large, on individual characters.

This preserves the largest meaningful structure possible within the size limit. For a well-formatted document, most chunks will be split at paragraph boundaries. The character-level fallback is rarely triggered for natural text.

Custom separator hierarchies for specific document types:

```
# For Markdown documents
markdown_separators = [
    "\n# ",    # H1 heading
    "\n## ",   # H2 heading
    "\n### ",  # H3 heading
    "\n#### ", # H4 heading
    "\n\n",    # Paragraph
    "\n",      # Line break
    " ",       # Word
]

# For code
code_separators = [
    "\nclass ",    # Class definition
    "\ndef ",      # Function definition
    "\n\n",        # Blank line
    "\n",          # Line
    " ",           # Token
]
```

The key insight: splitting at paragraph boundaries is almost always better than splitting at arbitrary token counts, and the recursive approach tries for paragraph boundaries first.

12.5 Structure-aware chunking

For documents with explicit structure, the best chunking strategy uses that structure directly rather than approximating it with text splitting.

12.5.1 Heading-based chunking

For documents with hierarchical headings (HTML, Markdown, DOCX with proper heading styles), chunk at heading boundaries. Each chunk is one section: from a heading to the next heading at the same or higher level.

```
def heading_chunk(document: ParsedDocument) -> list[Chunk]:
    chunks = []
    current_section = []
    current_heading = None
    current_level = 0

    for element in document.elements:
        if element.type == "heading":
            if current_section:
                chunks.append(Chunk(
                    text="\n".join(current_section),
                    heading=current_heading,
                    heading_level=current_level,
```

```
                    metadata=document.metadata
                ))
            current_section = [element.text]
            current_heading = element.text
            current_level = element.level
        else:
            current_section.append(element.text)

    if current_section:
        chunks.append(Chunk(
            text="\n".join(current_section),
            heading=current_heading,
            heading_level=current_level,
            metadata=document.metadata
        ))

    return chunks
```

This produces chunks that correspond to meaningful sections rather than arbitrary text blocks. The heading becomes part of the chunk metadata, enabling heading-based filtering and citation.

The limitation: sections vary enormously in length. A one-paragraph section produces a tiny chunk; a multi-page section produces a huge one. Post-processing applies fixed-size splitting to chunks that exceed the embedding model's context limit while keeping chunks below the limit intact.

12.5.2 Hierarchical chunking

Documents with nested structure (chapters contain sections contain subsections) can be chunked at multiple levels simultaneously:

```
Chapter level:    "Chapter 4: Authentication"
  Section level:  "4.1 API Keys"
  Section level:  "4.2 OAuth 2.0"
    Subsection:   "4.2.1 Authorization Code Flow"
    Subsection:   "4.2.2 Client Credentials Flow"
  Section level:  "4.3 JWT Tokens"
```

A query about "authentication" might retrieve the chapter-level chunk. A query about "OAuth authorization code flow" retrieves the specific subsection. Both are indexed; retrieval chooses the appropriate level. Hierarchical chunking requires a retrieval system that can handle multi-granularity documents, typically by indexing all levels and using metadata to filter or re-rank by granularity.

12.5.3 Table handling

Tables should almost never be split across chunks. A partial table (some rows at the end of one chunk, remaining rows at the start of the next) is nearly useless to the language model. Two strategies:

Keep tables whole. Treat each table as an atomic chunk regardless of size. If the table exceeds the embedding model's limit, summarize it or split into row groups that include the header row in each group.

Serialize by row group. Convert the table to serialized text and chunk by row groups, including the header in each chunk:

```
def serialize_table_chunk(table: Table, rows_per_chunk: int) -> list[str]:
    header = "| " + " | ".join(table.headers) + " |"
    separator = "| " + " | ".join(["---"] * len(table.headers)) + " |"
```

```
    chunks = []
    for i in range(0, len(table.rows), rows_per_chunk):
        row_group = table.rows[i:i + rows_per_chunk]
        rows_text = "\n".join(
            "| " + " | ".join(str(cell) for cell in row) + " |"
            for row in row_group
        )
        chunks.append(f"{header}\n{separator}\n{rows_text}")

    return chunks
```

12.5.4 Code chunking

Code has its own natural units: functions, classes, modules. Chunking at these boundaries is almost always preferable to fixed-size splitting.

For Python, the AST provides exact function and class boundaries:

```
import ast

def chunk_python_file(source_code: str) -> list[Chunk]:
    tree = ast.parse(source_code)
    chunks = []

    for node in ast.walk(tree):
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef)):
            start_line = node.lineno - 1
            end_line = node.end_lineno
            chunk_text = "\n".join(source_code.split("\n")[start_line:end_line])

            chunks.append(Chunk(
                text=chunk_text,
                metadata={
                    "type": type(node).__name__,
                    "name": node.name,
                    "start_line": node.lineno,
                    "end_line": node.end_lineno,
                }
            ))

    return chunks
```

12.6 Semantic chunking

Rather than splitting on structural or syntactic boundaries, semantic chunking splits where the meaning changes. Adjacent sentences discussing the same topic are grouped together; a split is introduced when the topic shifts.

12.6.1 Embedding-based semantic chunking

Embed each sentence independently, then compute the cosine similarity between adjacent sentence embeddings. A significant drop in similarity indicates a topic shift and marks the boundary for a new chunk.

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def semantic_chunk(sentences: list[str],
                   embedding_model,
                   percentile: float = 95) -> list[str]:

    embeddings = embedding_model.encode(sentences)

    similarities = [
        cosine_similarity([embeddings[i]], [embeddings[i+1]])[0][0]
        for i in range(len(embeddings) - 1)
    ]

    # Adaptive threshold: split where similarity is in the bottom percentile
    breakpoint_threshold = np.percentile(similarities, 100 - percentile)
    breakpoints = [
        i for i, sim in enumerate(similarities)
        if sim < breakpoint_threshold
    ]

    chunks = []
    start = 0
    for bp in breakpoints:
        chunks.append(" ".join(sentences[start:bp+1]))
        start = bp + 1
    chunks.append(" ".join(sentences[start:]))

    return chunks
```

12.6.2 Advantages and costs of semantic chunking

Semantic chunking produces chunks that are more topically coherent than fixed- size or structure-based chunking. Queries match chunks about the same topic as the query, not just chunks that happen to contain the query terms at an arbitrary text position.

The cost is significant: every sentence must be embedded individually during chunking. For a document with 1,000 sentences and an embedding model that takes 10ms per sentence, that is 10 seconds per document. At corpus scale (millions of documents), this is a substantial offline cost that is usually justified for high-value corpora where retrieval quality is critical.

12.6.3 LLM-based semantic chunking

A more sophisticated approach: use a language model to identify thematic boundaries directly.

```
def llm_chunk(document: str, llm) -> list[str]:
    prompt = f"""Identify the major thematic sections in the following document.
Return a JSON list of character indices where each new section begins.

Document:
{document}

Return only the JSON array of character indices."""

    response = llm.complete(prompt)
```

```
    section_starts = json.loads(response)

    chunks = []
    for i, start in enumerate(section_starts):
        end = section_starts[i+1] if i+1 < len(section_starts) else len(document)
        chunks.append(document[start:end])

    return chunks
```

LLM-based chunking is expensive (one model call per document) and slow but produces the highest-quality semantic boundaries. It is practical for high- value documents (contracts, research papers, technical specifications) where chunking quality directly affects critical business outcomes.

12.7 Chunk enrichment

Raw chunks often lack context that was available in the document but is not present in the chunk itself. Chunk enrichment adds this context before embedding, making the resulting embeddings more informative.

12.7.1 Prepending metadata

Include the document title, section heading, and other metadata in the chunk text before embedding:

```
def enrich_chunk(chunk: Chunk) -> str:
    parts = []

    if chunk.metadata.get("document_title"):
        parts.append(f"Document: {chunk.metadata['document_title']}")

    if chunk.metadata.get("section_heading"):
        parts.append(f"Section: {chunk.metadata['section_heading']}")

    if chunk.metadata.get("source_date"):
        parts.append(f"Date: {chunk.metadata['source_date']}")

    parts.append(chunk.text)

    return "\n".join(parts)
```

This changes what the embedding represents: instead of just the chunk text, the embedding represents the chunk in the context of its document and section. Queries about "authentication" will now match chunks in authentication sections more reliably, because the section heading "Authentication" is part of the embedded text.

12.7.2 Contextual retrieval

Anthropic's contextual retrieval technique uses a language model to generate a brief context description for each chunk, prepended before embedding:

```
def add_chunk_context(document: str, chunk: str, llm) -> str:
    prompt = f"""Given the following document:
<document>
{document}
</document>
```

```
And this chunk from the document:
<chunk>
{chunk}
</chunk>

Provide a brief (2-3 sentence) description of what this chunk is about and
how it relates to the broader document. This will be prepended to the chunk
to improve retrieval."""

    context = llm.complete(prompt)
    return f"{context}\n\n{chunk}"
```

The result: instead of embedding "The token expires after 3600 seconds by default," you embed "This chunk describes the default expiration time for authentication tokens in the API. It is from the Authentication section of the API reference guide. The token expires after 3600 seconds by default."

The enriched embedding retrieves more accurately for queries about token expiration, authentication configuration, or API defaults. The cost: one LLM call per chunk during ingestion. Batch the calls, use a smaller model for context generation, and cache results so that re-chunking does not require regenerating contexts.

12.7.3 Hypothetical document embeddings (HyDE)

A related technique applied at query time rather than ingestion time: instead of embedding the raw query, use a language model to generate a hypothetical document that would answer the query, then embed that hypothetical document for retrieval.

The intuition: a query ("what is the default token expiration time?") and a document chunk ("The token expires after 3600 seconds by default") may be phrased very differently and have dissimilar embeddings. A hypothetical answer ("The default token expiration time is 3600 seconds, configurable via the token_ttl parameter") is phrased similarly to the actual document content and embeds more similarly.

```
def hyde_retrieve(query: str, llm, retriever) -> list[Chunk]:
    hypothetical_doc = llm.complete(
        f"Write a brief passage that answers this question:\n{query}"
    )
    return retriever.retrieve(hypothetical_doc, top_k=5)
```

HyDE consistently improves retrieval quality for factual queries where the query phrasing differs significantly from the document phrasing. It adds one LLM call per query which is acceptable for most production systems where answer quality is the priority.

12.7.4 Query rewriting

A related technique operates at query time rather than ingestion time: rewrite the user's query so that it more closely matches the language, terminology, and writing style of the underlying document corpus before retrieval.

The motivation is the same as HyDE: retrieval systems operate in embedding space, and semantically identical concepts may be expressed very differently by users and documents. A user might ask:

> "How long does a login session last?"

while the documentation states:

> "Authentication tokens expire after 3600 seconds."

Although these refer to the same concept, the phrasing differs substantially. A query rewriting model transforms the user query into language that more closely resembles the corpus:

> "What is the default expiration time for authentication tokens?"

The rewritten query is then embedded and used for retrieval.

```
def rewrite_retrieve(query: str, llm, retriever) -> list[Chunk]:
    rewritten_query = llm.complete(
        f"""Rewrite the following search query so that it matches the
terminology and writing style of technical documentation while preserving
its original meaning.

Query: {query}

Return only the rewritten query."""
    )

    return retriever.retrieve(rewritten_query, top_k=5)
```

Query rewriting differs from HyDE in an important way. HyDE generates a hypothetical answer document and embeds that document for retrieval. Query rewriting preserves the query format but adapts its vocabulary and style to better align with the corpus.

This approach is particularly effective for domain-specific corpora where users and documents use different terminology, such as medical records, legal documents, internal company jargon, and technical documentation.

12.8 Chunk size selection

There is no universal optimal chunk size. The right value depends on the embedding model's context window (chunks cannot exceed the maximum input length), the query type (short factual queries match short precise chunks; long analytical queries match longer contextual chunks), the document type (technical documentation with short precise sections warrants smaller chunks; legal contracts with long interconnected clauses warrant larger ones), and the retrieval top-k and context window (if you retrieve top-5 chunks into a 128k context window, chunks can be large; if the context window is 4k tokens, keep chunks under 1,000 tokens each).

12.8.1 Empirical chunk size selection

The right approach: treat chunk size as a hyperparameter and evaluate empirically.

1. Build a small evaluation set: 50–100 questions with known answers from your document corpus
2. Ingest the corpus with multiple chunk sizes (256, 512, 1024, 2048 tokens)
3. For each chunk size, measure retrieval recall (does the answer appear in the top-k retrieved chunks?) and answer quality (does the LLM produce a correct answer?)
4. Select the chunk size that maximizes answer quality on the evaluation set

This takes time but produces chunk sizes calibrated to your specific documents and queries rather than arbitrary defaults. The defaults in most frameworks (512 tokens, 10% overlap) are reasonable starting points, not optimal values.

12.9 Multi-granularity retrieval

A single chunk size is a compromise. A better approach: index documents at multiple granularities and retrieve from the most appropriate level.

12.9.1 Parent-child chunking

Index small chunks for precise retrieval, but retrieve and pass larger parent chunks to the language model for context:

```
Document
    Parent chunk (1,000 tokens): "Section 4.2: OAuth 2.0"
          Child chunk (200 tokens): "Authorization Code Flow"
          Child chunk (200 tokens): "Client Credentials Flow"
          Child chunk (200 tokens): "Token Refresh"
```

Retrieval operates on child chunk embeddings (precise matching). When a child chunk is retrieved, the system returns its parent chunk (rich context). The language model receives the full parent chunk, not the narrow child chunk.

This combines the retrieval precision of small chunks with the contextual richness of large chunks which is the best of both without the tradeoff.

12.9.2 Small-to-big retrieval

Similar to parent-child, but the relationship is sentence-to-paragraph: embed sentences for retrieval, return the surrounding paragraph to the language model.

```python
def small_to_big_retrieve(query: str, retriever, document_store) -> list[str]:
    sentence_chunks = retriever.retrieve(query, top_k=10)

    paragraphs = set()
    for chunk in sentence_chunks:
        paragraph_id = chunk.metadata["paragraph_id"]
        paragraphs.add(document_store.get_paragraph(paragraph_id))

    return list(paragraphs)
```

12.9.3 Summary indexing

For very long documents, index a summary of each section alongside the sections themselves. Broad queries ("what is this document about?") match the summary. Specific queries ("what is the default timeout?") match the section chunks. The summary provides a navigational layer above the detailed content.

12.10 Key takeaways

- Chunking is the most consequential preprocessing decision in a RAG system; incoherent chunks cannot be compensated for by better retrieval or a better language model
- Fixed-size chunking with overlap is simple and works reasonably well for homogeneous prose; 10% overlap costs 10% more storage and dramatically reduces boundary failures
- Recursive character splitting preserves the largest meaningful structure (paragraphs, then lines, then words) within the size limit — better than pure fixed-size splitting for most documents
- Structure-aware chunking at heading boundaries produces the most semantically coherent chunks for well-formatted documents; tables and code should always be treated as atomic units with splitting only at row or function boundaries
- Semantic chunking (embedding-based similarity breakpoints) produces the most topically coherent chunks at the cost of significant offline compute — worth it for high-value corpora, expensive at scale
- Chunk enrichment — prepending metadata, section headings, or LLM-generated context descriptions — improves retrieval by making embeddings more informative about the chunk's place in the document; contextual retrieval is high-impact when quality justifies the extra LLM calls
- HyDE improves retrieval at query time by embedding a hypothetical answer rather than the raw query, bridging the vocabulary gap between queries and documents
- Parent-child chunking combines retrieval precision (embed small child chunks) with contextual richness (return large parent chunks to the language model)

- Treat chunk size as a hyperparameter: build an evaluation set of 50–100 representative questions and select chunk size empirically rather than using arbitrary defaults

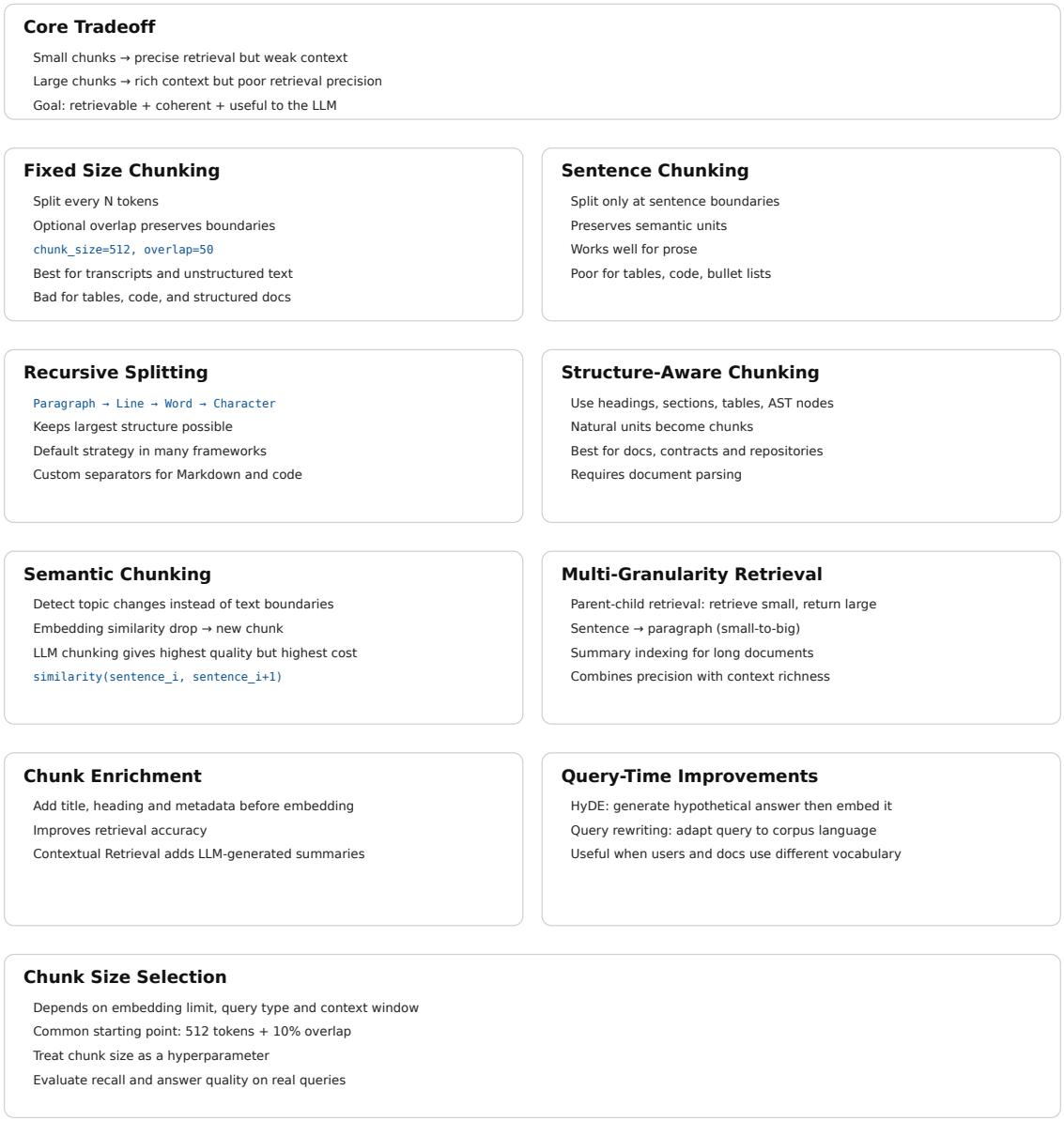


Figure 12.2: Cheat sheet.

12.11 Further reading

- Anthropic (2024). *Introducing Contextual Retrieval.* — The contextual chunk enrichment technique; one of the highest-impact chunking improvements available with documented retrieval quality gains.
- Gao et al. (2023). *Precise Zero-Shot Dense Retrieval without Relevance Labels.* — The HyDE paper; generating

hypothetical documents for retrieval.

- Liu et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts.* — Establishes that chunk position within the context window affects how well LLMs use retrieved content; relevant for context assembly decisions in chapter 24.
- LangChain Documentation. *Text Splitters.* — Comprehensive reference for RecursiveCharacterTextSplitter and other splitters with configuration examples.
- LlamaIndex Documentation. *Node Parsers / Text Splitters.* — Alternative implementation reference with hierarchical chunking and parent-child strategies.

Chapter 13

Embeddings for Search

The canonical question for this chapter: *You have 500,000 chunks from your document corpus. How do you convert them into vectors that make retrieval actually work, choosing the right model, encoding correctly for your domain, and handling the practical realities of embedding at scale?*

Where are we?

Figure 13.1: The journey through the Model Mind.

Chunking produced a list of text fragments. Now each fragment must become a vector, a point in a high-dimensional space where semantic similarity corresponds to geometric proximity. This chapter covers everything that happens between a text chunk and a vector stored in your index: choosing the right embedding model, encoding queries and documents correctly, managing the operational complexity of embedding at scale, and knowing when your embeddings are failing.

13.1 This chapter vs. chapter 06

Chapter 06 covered embeddings from the perspective of what happens inside a generative LLM: the embedding table, the residual stream, how token representations evolve through transformer layers, and what contextual embeddings are.

This chapter covers embeddings from the perspective of building a retrieval system: choosing and evaluating embedding models for your specific corpus and query distribution, encoding documents and queries correctly, handling domain shift, updating embeddings as documents change, and managing the operational complexity of embedding a large corpus.

The underlying mathematics is the same. The engineering concerns are entirely different.

13.2 The retrieval embedding task

Retrieval embedding is a specific task with requirements that differ from other embedding uses such as classification, clustering, or general semantic similarity.

In retrieval, you have two types of inputs with asymmetric properties:

Queries are short, often incomplete, phrased as questions or keyword phrases, written by users who may not use the same terminology as the documents. A user asking "how long before my session times out" is asking about the same thing as a document that says "the default token expiration is 3600 seconds."

Documents (chunks) are longer, complete, written by authors using precise domain terminology, often containing the answer to many possible queries rather than to one specific query.

A retrieval embedding model must map these two types of inputs which are stylistically and lexically different to nearby vectors when they are semantically related. This is the core challenge, and it is different from general semantic similarity, where both inputs have similar style.

The best retrieval embedding models are trained specifically for this asymmetric task, using query-document pairs rather than symmetric text-text pairs. Models trained only for general semantic similarity perform measurably worse at retrieval even if they produce better representations for other tasks.

13.3 Choosing an embedding model

The embedding model is one of the highest-impact decisions in a RAG system. Switching models requires re-embedding the entire corpus, so the choice is sticky and getting it right matters.

13.3.1 The MTEB leaderboard

The Massive Text Embedding Benchmark (MTEB) is the standard reference for comparing embedding models. It evaluates models across 56 datasets covering 8 task categories including retrieval, clustering, classification, reranking, and semantic similarity.

For RAG systems, the retrieval subcategory of MTEB is most relevant. Key retrieval datasets include BEIR (18 heterogeneous retrieval datasets covering biomedical, financial, scientific, and general-purpose text), TREC (standard IR benchmarks), and MIRACL (multilingual retrieval).

Critical caveat: MTEB scores are averages across many datasets. A model that ranks first on average may perform worse than a fifth-place model on your specific domain. Always evaluate on data representative of your actual use case.

13.3.2 Key selection criteria

Retrieval-specific training. Prefer models trained with query-document contrastive objectives over general sentence encoders. Models explicitly listed as "retrieval" or "embedding search" models in their documentation are trained for this task.

Embedding dimension. Higher dimensions (1,536, 3,072) generally provide better retrieval quality but cost more in storage and computation. For millions of documents, the cost difference is significant. Matryoshka models allow dimension reduction post-hoc which could be preferable for flexibility.

Context window. Must accommodate your chunk size. A model with a 512-token context window cannot embed 1,000-token chunks without truncation. Models with 8,192-token contexts handle longer chunks but may not use the extra context efficiently for all document types.

Language coverage. If your corpus is multilingual, use a multilingual model. English-centric models tokenize non-English text inefficiently and produce degraded embeddings for non-English content. mE5, multilingual-e5-large, and similar models cover 100+ languages.

Inference cost and latency. Embedding 500,000 chunks at $0.0001 per 1,000 tokens costs $50. At $0.00002 per 1,000 tokens, it costs $10. For large corpora embedded frequently, cost compounds. Latency also matters for query-time embedding: if queries must be embedded before retrieval, embedding latency adds directly to TTFT.

Self-hosted vs. API. API-based embedding (OpenAI, Cohere, Voyage) avoids infrastructure management but creates a runtime dependency. Self-hosted models (BGE, GTE, E5 via HuggingFace) require GPU infrastructure but give full control over latency, cost, and data privacy.

13.3.3 Evaluating on your data

The only reliable evaluation is on your own data:

1. Sample 200–500 representative queries from your expected query distribution.
2. For each query, identify the ground-truth relevant chunks.
3. Embed your corpus with each candidate embedding model.
4. For each query, retrieve top-k chunks and measure Recall@k (fraction of ground-truth chunks appearing in the top-k), MRR (Mean Reciprocal Rank), and NDCG (Normalized Discounted Cumulative Gain).
5. Select the model that maximizes these metrics on your evaluation set.

13.4 Asymmetric encoding and instruction prefixes

Many retrieval embedding models require different encoding for queries vs. documents. Using the same encoding for both is a common mistake that silently degrades retrieval quality.

13.4.1 Instruction-based models (E5, Instructor)

E5 and Instructor models prepend a task-specific instruction to each input. The instruction tells the model what type of text it is encoding and what the embedding will be used for:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("intfloat/e5-large-v2")

# Queries: prepend "query: "
queries = ["query: how long before session timeout?"]
query_embeddings = model.encode(queries, normalize_embeddings=True)

# Documents: prepend "passage: "
documents = [
    "passage: The default token expiration is 3600 seconds, "
    "configurable via the token_ttl parameter in the authentication settings."
]
doc_embeddings = model.encode(documents, normalize_embeddings=True)
```

The instruction prefix shifts the model's representation toward the appropriate mode. Query embeddings and passage embeddings are trained to be similar when semantically related, even though they are encoded differently.

13.4.2 Cohere and Voyage asymmetric encoding

Cohere and Voyage AI models expose the asymmetry through explicit `input_type` parameters:

```
import cohere

co = cohere.Client(api_key)
```

```
# Encode queries
query_response = co.embed(
    texts=["how long before session timeout?"],
    model="embed-english-v3.0",
    input_type="search_query",       # ← query encoding
)

# Encode documents
doc_response = co.embed(
    texts=["The default token expiration is 3600 seconds..."],
    model="embed-english-v3.0",
    input_type="search_document",    # ← document encoding
)
```

The `input_type` parameter is not optional and omitting it or using the wrong value produces incorrect embeddings. This is a common source of subtle retrieval bugs where the system appears to work (it returns results) but quality is silently degraded.

13.4.3 OpenAI embeddings

OpenAI's text-embedding-3 models do not require explicit query/document differentiation. They can be used for both with the same API call:

```
from openai import OpenAI

client = OpenAI()

def embed(texts: list[str]) -> list[list[float]]:
    response = client.embeddings.create(
        input=texts,
        model="text-embedding-3-large",
        dimensions=1024,  # MRL truncation
    )
    return [item.embedding for item in response.data]
```

The simplicity of the OpenAI interface makes it easier to use correctly, at the cost of potentially lower retrieval quality compared to models with explicit asymmetric training on specialized retrieval tasks.

13.5 Batching and throughput for corpus embedding

Embedding a large corpus efficiently requires batching. Embedding one document at a time is 10–100× slower than batching due to GPU underutilization and per-request overhead.

13.5.1 Optimal batch size

For GPU-hosted models:

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("BAAI/bge-large-en-v1.5")

def embed_corpus(chunks: list[str], batch_size: int = 256) -> np.ndarray:
```

```
    all_embeddings = []

    for i in range(0, len(chunks), batch_size):
        batch = chunks[i:i + batch_size]
        batch_embeddings = model.encode(
            batch,
            normalize_embeddings=True,
            show_progress_bar=False,
            convert_to_numpy=True,
        )
        all_embeddings.append(batch_embeddings)

    return np.vstack(all_embeddings)
```

For API-based embedding, respect provider limits: OpenAI allows up to 2,048 inputs per request (max 8,191 tokens per input); Cohere allows up to 96 inputs per request. Rate limits are typically 1,000–10,000 RPM depending on tier.

Implement retries with exponential backoff for API calls to handle transient failures, which are unavoidable in distributed systems operating at scale.

```
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=1, max=60)
)
def embed_batch_with_retry(
    texts: list[str], client, model: str
) -> list[list[float]]:
    response = client.embeddings.create(input=texts, model=model)
    return [item.embedding for item in response.data]
```

13.5.2 Incremental embedding

For corpora that change continuously, embedding only new and updated documents is essential and re-embedding the full corpus on each run is prohibitively expensive at scale:

```
import hashlib

def compute_chunk_hash(chunk_text: str) -> str:
    return hashlib.sha256(chunk_text.encode()).hexdigest()

def embed_incremental(
    new_chunks: list[Chunk],
    existing_hashes: set[str],
    embedding_store
) -> int:
    """Embed only chunks not already in the store. Returns count of new chunks."""
    chunks_to_embed = [
        chunk for chunk in new_chunks
        if compute_chunk_hash(chunk.text) not in existing_hashes
    ]

    if not chunks_to_embed:
        return 0
```

```
    texts = [chunk.text for chunk in chunks_to_embed]
    embeddings = embed_corpus(texts)

    for chunk, embedding in zip(chunks_to_embed, embeddings):
        embedding_store.upsert(
            id=chunk.id,
            vector=embedding,
            metadata=chunk.metadata,
            text=chunk.text,
        )

    return len(chunks_to_embed)
```

Content hashing ensures idempotency: the same chunk embedded twice produces one entry, not two.

13.6 Domain adaptation

General-purpose embedding models are trained on diverse web text. For specialized domains (medical literature, legal documents, financial filings, source code) general models often underperform domain-specific alternatives.

13.6.1 When domain adaptation is needed

Signs that general models are insufficient: retrieval recall on domain-specific evaluation sets is significantly lower than on general benchmarks; queries use domain terminology that general models do not represent well ("nociceptor," "collateralized debt obligation," "amortization schedule"); documents use domain conventions (citation formats, abbreviations, structured fields) that general models have not seen frequently.

13.6.2 Domain-specific pre-trained models

Several domains have dedicated embedding models: BioMedBERT, PubMedBERT, and BioBERT for biomedical text; Legal-BERT for legal documents; FinBERT for financial text; CodeBERT and StarEncoder for source code. These models start from domain-specific pretraining, meaning their tokenizer and base representations are calibrated for domain vocabulary.

For retrieval specifically, domain-pretrained models still need retrieval fine-tuning and pretraining alone is not sufficient for strong asymmetric retrieval performance.

13.6.3 Fine-tuning an embedding model

If no suitable domain model exists, or if retrieval quality remains insufficient after trying domain models, fine-tune a general embedding model on domain- specific query-document pairs.

Training data: pairs of (query, relevant_document_chunk) where the query is representative of real user questions and the document chunk contains the correct answer. Sources for training pairs include human annotation (most reliable, expensive), synthetic generation (use an LLM to generate queries for each chunk for example: "write 3 questions this passage answers" scales easily and works well), and mined pairs (Stack Overflow questions and accepted answers, FAQ pages, support tickets with resolution notes).

```
from sentence_transformers import SentenceTransformer, losses
from sentence_transformers.training_args import SentenceTransformerTrainingArguments
from datasets import Dataset

train_data = Dataset.from_dict({
```

```
    "anchor": ["how long before session timeout?", ...],          # queries
    "positive": ["The default token expiration is 3600...", ...], # relevant chunks
})

model = SentenceTransformer("BAAI/bge-large-en-v1.5")

train_loss = losses.MultipleNegativesRankingLoss(model)

training_args = SentenceTransformerTrainingArguments(
    output_dir="./fine-tuned-retrieval-model",
    num_train_epochs=3,
    per_device_train_batch_size=32,
    learning_rate=2e-5,
    warmup_ratio=0.1,
)

model.fit(
    train_objectives=[(train_data, train_loss)],
    args=training_args,
)
```

Fine-tuned models consistently outperform general models on domain-specific retrieval by 5–20% on recall metrics. The investment (data collection plus training) pays off quickly for high-value retrieval applications.

13.7 Embedding stability and versioning

Embeddings are not stable across model versions, as a result updating a model changes the vector representation of the same input text. A corpus embedded with `text-embedding-ada-002` is not compatible with `text-embedding-3-large`: they live in different vector spaces, and similarity scores between the two are meaningless.

13.7.1 The model update problem

If you embed your corpus with model version V1 and switch to V2: - Similarity scores between V1 document embeddings and V2 query embeddings are garbage - Retrieval quality can degrade silently: results are still returned, but they may be incorrect without any errors being raised. - The only fix is to re-embed the entire corpus with V2

This creates a strong operational coupling between your corpus embeddings and your query encoder. They must always use the same model version.

13.7.2 Versioning strategy

Track the embedding model version with every stored embedding:

```
{
    "chunk_id": "chunk-abc123",
    "text": "The default token expiration is 3600 seconds...",
    "embedding": [0.23, -0.87, ...],
    "embedding_model": "text-embedding-3-large",
    "embedding_model_version": "2024-02-01",
    "embedding_dimensions": 1024,
    "embedded_at": "2024-03-15T10:30:00Z",
}
```

When switching embedding models, you can either perform a big-bang migration by re-embedding the entire corpus with the new model (simpler but requiring downtime or a parallel index), or use a dual-index approach that maintains both old and new indices, routes new documents to the new index, and gradually migrates data while merging results at query time; this enables zero downtime but adds significant complexity.

13.8 Embedding quality signals

How do you know if your embeddings are working without running full end-to-end evaluation?

13.8.1 Nearest-neighbor inspection

Sample 50 chunks from your corpus. For each, find the 5 nearest neighbors in embedding space. If the neighbors are semantically related to the sample chunk, embeddings are working correctly. If neighbors are unrelated embeddings are failing.

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def inspect_nearest_neighbors(
    sample_chunks: list[str],
    all_chunks: list[str],
    all_embeddings: np.ndarray,
    embedding_model,
    k: int = 5
):
    sample_embeddings = embedding_model.encode(
        sample_chunks, normalize_embeddings=True
    )

    for chunk, embedding in zip(sample_chunks, sample_embeddings):
        similarities = cosine_similarity([embedding], all_embeddings)[0]
        top_k_indices = np.argsort(similarities)[::-1][1:k+1]  # exclude self

        print(f"\nQuery chunk: {chunk[:100]}...")
        print("Nearest neighbors:")
        for idx in top_k_indices:
            print(f"  [{similarities[idx]:.3f}] {all_chunks[idx][:80]}...")
```

13.8.2 Embedding space health check

```
def check_embedding_health(embeddings: np.ndarray):
    # Norm statistics - should be near 1.0 for normalized embeddings
    norms = np.linalg.norm(embeddings, axis=1)
    print(f"Norm stats: mean={norms.mean():.3f}, std={norms.std():.3f}")

    # Pairwise similarity - high average indicates embedding collapse
    sample_indices = np.random.choice(
        len(embeddings), min(1000, len(embeddings)), replace=False
    )
    sample = embeddings[sample_indices]
    pairwise_sims = cosine_similarity(sample)
```

```
    mask = ~np.eye(len(sample), dtype=bool)
    avg_sim = pairwise_sims[mask].mean()
    print(f"Average pairwise similarity: {avg_sim:.3f}")
    print("(< 0.1 is healthy, > 0.3 suggests potential collapse)")
```

High average pairwise similarity (> 0.3) suggests embedding collapse. When all chunks are being mapped to similar vectors it means retrieval failed. This can happen with domain shift (the model has not seen your document type), poor text quality (chunks too short or too noisy), or model configuration issues such as wrong prefix format.

13.8.3 Query-document alignment evaluation

For a sample of known query-document pairs, compute the rank of the correct document:

```
def evaluate_retrieval(
    query_doc_pairs: list[tuple[str, str]],
    all_chunks: list[str],
    all_embeddings: np.ndarray,
    embedding_model
) -> dict:
    queries = [pair[0] for pair in query_doc_pairs]
    correct_chunks = [pair[1] for pair in query_doc_pairs]

    query_embeddings = embedding_model.encode(
        [f"query: {q}" for q in queries],
        normalize_embeddings=True
    )

    ranks = []
    for q_emb, correct in zip(query_embeddings, correct_chunks):
        similarities = cosine_similarity([q_emb], all_embeddings)[0]
        sorted_chunks = [all_chunks[i] for i in np.argsort(similarities)[::-1]]
        rank = (
            sorted_chunks.index(correct) + 1
            if correct in sorted_chunks
            else len(all_chunks)
        )
        ranks.append(rank)

    return {
        "recall@1":  sum(r <= 1  for r in ranks) / len(ranks),
        "recall@5":  sum(r <= 5  for r in ranks) / len(ranks),
        "recall@10": sum(r <= 10 for r in ranks) / len(ranks),
        "mrr":       sum(1/r for r in ranks) / len(ranks),
        "median_rank": np.median(ranks),
    }
```

Run this evaluation before deploying to production, and re-run whenever the embedding model or corpus changes. Recall@5 above 0.8 is a reasonable bar for most retrieval tasks and when it is below 0.6, investigate chunking and model choice before blaming retrieval algorithms.

13.9 Late interaction models: ColBERT

Standard embedding models produce one vector per document, and retrieval is a single vector comparison. Late interaction models like ColBERT produce one vector per token, and retrieval involves matching token-level representations.

13.9.1 How ColBERT works

ColBERT encodes both the query and the document into sequences of token-level embeddings (not pooled to a single vector). Similarity is computed via MaxSim: for each query token, find its most similar document token; sum these maximum similarities across all query tokens.

```
Query: ["how", "long", "session", "timeout"]
  → 4 query token embeddings: [q1, q2, q3, q4]

Document: ["default", "token", "expiration", "3600", "seconds"]
  → 5 doc token embeddings: [d1, d2, d3, d4, d5]

MaxSim score = max_sim(q1, {d1..d5}) + max_sim(q2, {d1..d5})
             + max_sim(q3, {d1..d5}) + max_sim(q4, {d1..d5})
```

This allows fine-grained matching: "session timeout" in the query matches "token expiration" in the document because the token-level embeddings for "session" and "token" are similar in context, and "timeout" and "expiration" are similar. Standard single-vector models must capture all of this in one pooled vector, which is a lossy compression.

13.9.2 ColBERT tradeoffs

Better retrieval quality. ColBERT consistently outperforms single-vector models on retrieval benchmarks, often by significant margins on tasks requiring multi-token matching.

Higher storage cost. Instead of one 1,024-dimensional vector per chunk, ColBERT stores N vectors (one per token). A 200-token chunk requires 200 × 128-dimensional vectors.

More complex retrieval. MaxSim requires comparing all query token vectors against all document token vectors. In practice, ColBERT uses approximate nearest-neighbor retrieval to find candidate documents, then applies MaxSim for precise re-ranking.

13.10 Key takeaways

- Retrieval embedding is an asymmetric task — queries and documents have different styles and vocabulary; use models trained specifically for retrieval, not general semantic similarity models
- Always evaluate embedding models on your own domain data before committing; MTEB rankings reflect average performance across many datasets, not performance on your specific corpus
- Asymmetric encoding is required for models like E5, Instructor, Cohere, and Voyage: queries and documents must be encoded differently; using the wrong prefix or input_type silently degrades retrieval quality with no error thrown
- Batch embedding efficiently using appropriate batch sizes; implement retry with exponential backoff for API-based embedding; use content hashing for incremental updates to avoid re-embedding unchanged chunks
- Domain-specific models and fine-tuning on domain query-document pairs improve retrieval recall by 5–20% for specialized corpora; synthetic query generation from chunks is an effective and scalable way to create training data
- Embedding model versions are incompatible: switching models requires re-embedding the entire corpus; track model version with every stored embedding and plan migrations carefully

- Inspect embedding quality before deploying: nearest-neighbor inspection, pairwise similarity health checks, and known query-document pair evaluation catch failures that would otherwise appear as degraded answers in production
- ColBERT's token-level late interaction achieves higher retrieval quality than single-vector models at the cost of significantly more storage and retrieval complexity — worth it when single-vector recall is insufficient

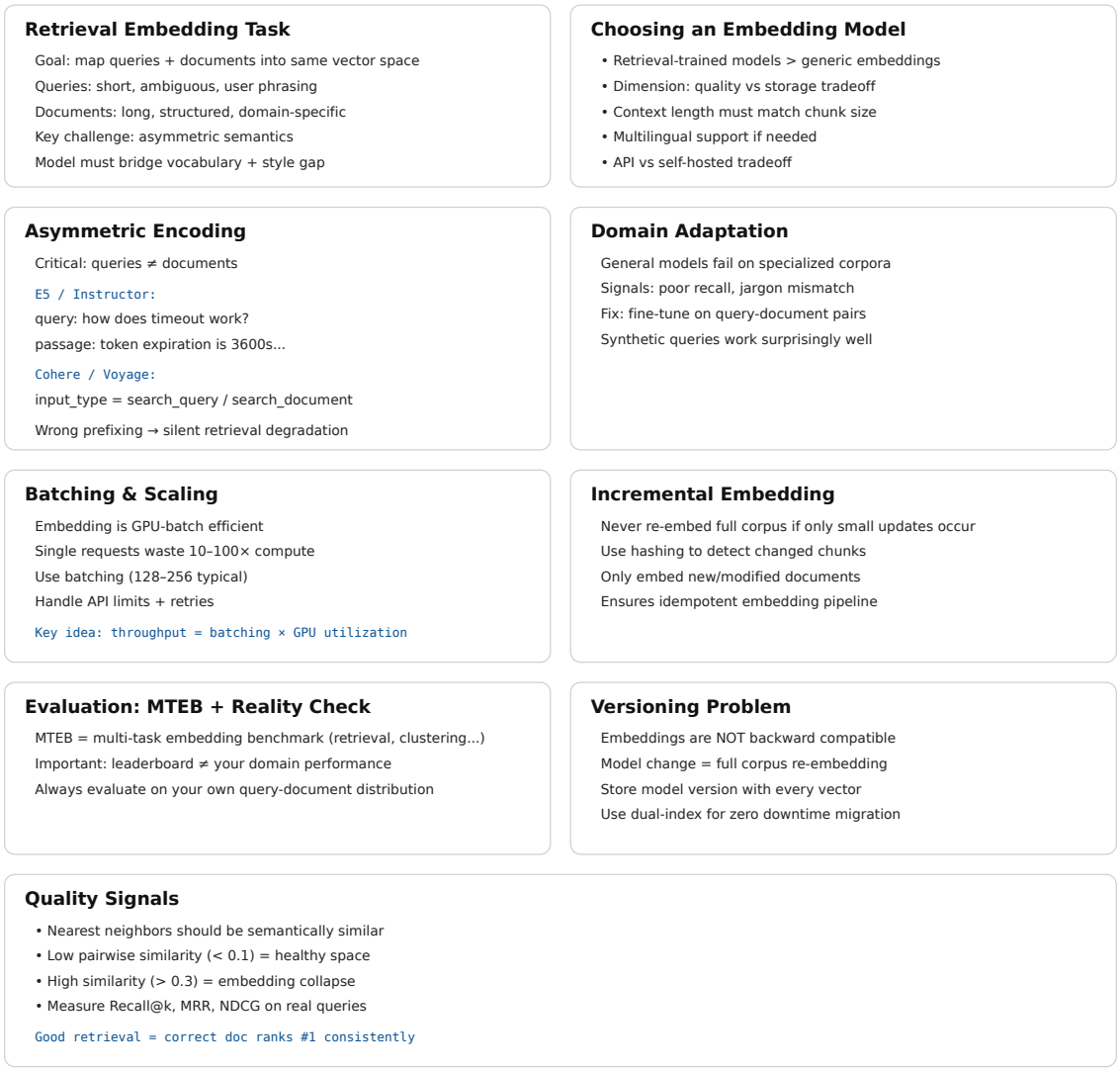


Figure 13.2: Cheat sheet.

13.11 Further reading

- Karpukhin et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering.* — The foundational dense retrieval paper establishing the query-document asymmetric training paradigm.
- Khattab & Zaharia (2020). *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT.* — Token-level late interaction retrieval.

- Wang et al. (2022). *Text Embeddings by Weakly-Supervised Contrastive Pre-training.* — E5; the instruction-prefix asymmetric encoding approach.
- Muennighoff et al. (2023). *MTEB: Massive Text Embedding Benchmark.* — The benchmark; essential reference for comparing embedding models.

Chapter 14

Vector Databases

The canonical question for this chapter: *You have 10 million embeddings and a query arrives. You need the 10 most similar vectors in under 50 milliseconds how does that actually work and which system should you use?*

Where are we?

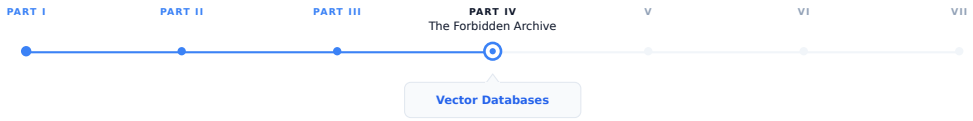


Figure 14.1: The journey through the Model Mind.

Embeddings converted your chunks into vectors. Now those vectors need a home, a system that stores them durably, indexes them for fast search, and returns the most similar ones for any query in interactive latency. This chapter covers how that works algorithmically and which systems implement it, and how to choose between them.

14.1 What a vector database does

A vector database stores high-dimensional vectors and answers nearest-neighbor queries: given a query vector, return the k vectors in the database most similar to it, along with their associated metadata and document text.

Although conceptually simple, efficient vector search at scale is far from trivial. Exact nearest-neighbor search over 10 million 1,536-dimensional vectors requires computing 10 million dot products per query. At 1,536 dimensions and float32 precision, each dot product requires 1,536 multiplications and 1,535 additions. For 10 million vectors, that is 30 billion floating-point operations per query. On a CPU doing 100 GFLOPS, that is 300 milliseconds too slow for an interactive system.

The solution is approximate nearest neighbor (ANN) search: algorithms that find vectors very likely to be among the true top-k, fast enough for interactive latency, with a small probability of missing some true nearest neighbors. The accuracy/speed tradeoff is the central engineering parameter of every vector search system.

Beyond search, a vector database also handles persistent storage of vectors and metadata, metadata filtering (retrieve only vectors where `source == "legal_docs"`), CRUD operations, consistency and durability guarantees, horizontal

scaling as the corpus grows, and access control.

Choosing the right system requires understanding how ANN algorithms work, what the operational tradeoffs are, and what your actual requirements are.

14.2 How ANN Search Works

ANN search is not one algorithm but a family of approaches with different tradeoffs between build time, query latency, memory footprint, and recall. Production vector databases typically implement several of these and let you choose (or choose automatically based on your data). Understanding the whole family, is what lets you diagnose why a system is slow, why recall is lower than expected, or which index type to reach for at a given scale.

14.2.1 The exact search baseline

Before approximation makes sense, it helps to understand what you are approximating away from. Exact nearest-neighbor search is brute-force: compute the similarity between the query vector and every stored vector, sort by similarity, return top-k. This is O(n × d) per query where n is the number of vectors and d is the embedding dimension.

```python
import numpy as np

def exact_search(
    query_embedding: np.ndarray,
    corpus_embeddings: np.ndarray,
    k: int = 10
) -> tuple[list[int], list[float]]:
    # Assumes embeddings are normalized (unit vectors)
    # dot product = cosine similarity for normalized vectors
    similarities = corpus_embeddings @ query_embedding
    top_k_indices = np.argsort(similarities)[::-1][:k]
    top_k_scores = similarities[top_k_indices]
    return top_k_indices.tolist(), top_k_scores.tolist()
```

For small corpora (under ~100,000 vectors), exact search is fast enough. FAISS's flat index, NumPy's matrix multiplication, and pgvector's exact search all use this approach. It is the right choice when correctness matters more than scale yet for larger corpora, you need ANN.

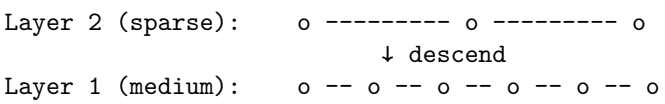
The approximate algorithms fall into four families based on how they narrow the search space: graph-based, cluster-based (partitioning), hashing-based, and tree-based. Each makes a different bet about the structure of your data.

14.2.2 Graph-based methods

Graph-based ANN builds a proximity graph over the vectors (nodes are vectors, edges connect "nearby" vectors) and answers queries by greedily walking the graph toward the query.

HNSW (Hierarchical Navigable Small World). The most widely used ANN algorithm in production vector databases. Weaviate, Qdrant, Milvus, and pgvector all offer it as their primary index type.

The core idea: build a layered graph where each node is a vector and edges connect nearby vectors. Higher layers have fewer nodes but longer-range connections; lower layers have all nodes with short-range connections. Search starts at the top layer (coarse navigation) and descends to lower layers (fine-grained search).



```
                          ↓ descend
Layer 0 (dense):     o-o-o-o-o-o-o-o-o-o-o-o-o
```

Search starts at the entry point at the top layer. Greedily navigate to the nearest neighbor. Descend to the next layer. Repeat until layer 0. At layer 0, perform a beam search over the local neighborhood. Return the top-k candidates found. This gives O(log n) expected search time for well-distributed data, compared to O(n) for exact search.

Key HNSW parameters:

`M` (max connections per node) — higher M means better recall but more memory and slower build time. Typical values: 16–64.

`ef_construction` (beam width during index build) — higher values mean a higher-quality index at the cost of longer build time. Typical values: 100–400.

`ef` (beam width during search, also called `ef_search`) — the primary recall/speed tradeoff lever, adjustable at query time without rebuilding the index. Higher ef means more nodes examined per query, better recall, higher latency. Typical values: 50–500.

Representative recall-latency numbers at different `ef` settings (always benchmark on your own data):

ef	Recall@10	Latency (p99)
50	0.92	8 ms
100	0.97	15 ms
200	0.99	28 ms
500	0.999	65 ms

NSG (Navigating Spreading-out Graph). Predates and influenced HNSW's practical adoption. Builds a single-layer graph (no hierarchy) but constructs edges more carefully, starting from a k-NN graph and pruning edges to minimize graph diameter while preserving navigability. NSG indexes are smaller than HNSW for comparable recall because there is no multi-layer overhead, but build time is longer and the algorithm is less forgiving of insertions after the initial build. Used in some high-performance research systems more than in mainstream managed databases.

Vamana / DiskANN. Developed by Microsoft Research specifically for billion-scale search where the full index cannot fit in RAM. Vamana builds a single flat graph (like NSG) but is explicitly designed so the graph can live on SSD: each node's neighbor list is small and the graph has low diameter, so a query touches only a small number of disk pages. DiskANN (the system built on Vamana) achieves HNSW-competitive recall while indexing billions of vectors on a single machine with far less RAM than an all-in-memory HNSW index would require, at the cost of higher per-query latency (disk I/O instead of RAM access). Milvus and Qdrant both support disk-resident indexes derived from this line of work as an option for corpora too large to hold in memory affordably.

Why graph methods dominate mainstream use. They give the best recall per unit of query latency for in-memory datasets, they support incremental insertion reasonably well (important for RAG corpora that grow continuously), and their query-time recall/speed tradeoff is tunable without rebuilding the index (the `ef` parameter). The cost is build time and memory: graphs store explicit edge lists, and construction requires many nearest-neighbor computations per inserted vector.

14.2.3 Cluster-based / partitioning methods

Rather than building a graph, these methods partition the vector space into regions and restrict search to the regions closest to the query.

IVF (Inverted File Index). FAISS's primary index for large-scale search. IVF clusters vectors into groups (Voronoi cells) using k-means. Each cluster has a centroid, and at search time, the query is compared to all centroids, and only the closest `nprobe` clusters are searched exactly.

```
Preprocessing:
  cluster all vectors into k clusters using k-means
  for each cluster, store the centroid and list of member vectors

Search:
  1. Compare query to all k centroids → find nprobe nearest centroids
  2. Search all vectors in those nprobe clusters exactly
  3. Return top-k from the searched vectors
```

Key parameters: `nlist` (number of clusters, typically `sqrt(n)` to `n/10`); `nprobe` (clusters searched per query, the recall/speed lever). IVF is faster than HNSW for very large datasets (100M+ vectors) because its flat cluster structure maps better to SIMD-vectorized exact search within clusters. HNSW's graph traversal has pointer-chasing that is harder to vectorize. The weakness is recall at cell boundaries: a true nearest neighbor sitting just across a cell boundary from the query can be missed unless `nprobe` is large enough to cover neighboring cells.

Product Quantization (PQ). A compression technique layered on top of IVF (or HNSW) that is important enough to understand on its own. PQ reduces vector storage by dividing each d-dimensional vector into m sub-vectors of d/m dimensions each, and quantizing each sub-vector to one of k centroids (typically 256). The stored representation is m byte values rather than d × 4 bytes.

```
Original:          1024 floats × 4 bytes = 4,096 bytes per vector
PQ(m=64, k=256):   64 bytes per vector   → 64× compression
```

The tradeoff: approximate distances from centroid reconstruction introduce error. PQ is used when memory is the primary constraint and some recall loss is acceptable.

IVF-PQ and IVF-ADC. IVF combined with product quantization for the vectors stored within each cell. This is FAISS's standard configuration for billion-scale search: coarse quantization (IVF) narrows the candidate set, product quantization compresses storage and speeds up distance computation within candidates. ADC (Asymmetric Distance Computation) keeps the query vector unquantized while comparing against quantized database vectors, improving accuracy over quantizing both sides. HNSW + PQ combines HNSW's recall with PQ's memory efficiency and is similarly available in FAISS.

ScaNN (Scalable Nearest Neighbors). Google's production ANN system, notable for a specific insight: not all quantization error matters equally. Standard PQ minimizes reconstruction error uniformly across all vectors, but what actually matters for retrieval is preserving the *relative ordering* of distances near the query. ScaNN uses anisotropic vector quantization, which weights quantization error along the direction that matters most for maximum-inner-product ranking. It consistently tops ANN-Benchmarks leaderboards for recall at a given queries-per-second budget. Available as an open-source library; used internally at Google and adopted by some vector database backends.

When partitioning methods win. At very large scale (hundreds of millions to billions of vectors) where memory is the binding constraint and where insert-heavy workloads make graph maintenance expensive, cluster-based indexes with compression (IVF-PQ, ScaNN) typically beat graph-based indexes on cost per query at a given recall target.

14.2.4 Hashing-based methods

LSH (Locality-Sensitive Hashing). The oldest ANN approach still in use. The idea: design a family of hash functions such that nearby vectors are more likely to collide (hash to the same bucket) than distant vectors. At query time, hash the query, look only at vectors in the same bucket (or nearby buckets via multiple hash tables), and rank by exact distance within that small candidate set.

```
Random hyperplane LSH for cosine similarity:
  Generate k random hyperplanes through the origin
  hash(v) = sign(v · h_1), sign(v · h_2), ..., sign(v · h_k)
  → a k-bit binary code per vector

Vectors with the same (or very similar) k-bit code are likely to be close
```

```
in cosine similarity, the more hyperplanes two vectors agree on, the
smaller the angle between them is likely to be.
```

LSH has clean theoretical guarantees (bounds on the probability of missing a true near neighbor) and very cheap hash computation. In practice it has mostly been superseded by graph and cluster methods for text embedding retrieval: LSH generally needs many hash tables to reach the recall that HNSW achieves with a single graph, which inflates memory and query cost. LSH remains relevant for specific settings in very high-dimensional sparse data, streaming settings where index rebuild cost must be near zero, and some specialized deduplication and fingerprinting tasks (near-duplicate detection, copyright matching) where its collision guarantees are the actual point, not just a means to fast search.

14.2.5 Tree-based methods

Annoy (Approximate Nearest Neighbors Oh Yeah). Spotify's ANN library. Builds a forest of random projection trees: each tree recursively splits the vector space with random hyperplanes, and a query descends each tree to a leaf, collecting candidates from the leaves it lands in across all trees.

Annoy's distinguishing feature is memory-mapped, read-only indexes: once built, an Annoy index can be mmap'd and shared across processes with no per-process memory duplication, and it degrades gracefully under memory pressure since the OS pages it in as needed. This makes it a good fit for recommendation-style workloads with large static catalogs and many concurrent readers. Its weaknesses relative to HNSW: no incremental insertion (the index is built once and is immutable, updates require a full rebuild) and generally lower recall per unit of query time on standard ANN benchmarks.

14.2.6 Comparing the families

Family	Example	Best at	Weak at
Graph	HNSW, NSG, Vamana	Recall/latency for in-memory data; incremental inserts	Memory overhead; build time
Partition	IVF, IVF-PQ, ScaNN	Very large scale; compressed storage; cost per query	Boundary recall; tuning nprobe
Hashing	LSH	Theoretical guarantees; near-duplicate detection; near-zero rebuild cost	Recall per byte vs. graph/partition methods
Tree	Annoy	Read-heavy, static, memory-shared workloads	No incremental updates; lower recall at scale

For text embedding retrieval in RAG systems specifically graph-based HNSW is the default choice up to tens of millions of vectors, and disk-resident graph methods (DiskANN-derived) or IVF-PQ/ScaNN are the choice beyond that, when the index no longer fits affordably in RAM. LSH and tree methods are rarely the right choice for text retrieval today but are worth recognizing when you encounter them in older systems or adjacent problem domains (deduplication, recommendation).

14.3 Vector database landscape

The major production vector databases differ in architecture, operational model, and capability. Choosing the right one depends on corpus size, operational requirements, and existing infrastructure.

14.3.1 Pinecone

Fully managed, serverless vector database. Pinecone handles all infrastructure, you store vectors and query them via API with no server management.

```
from pinecone import Pinecone

pc = Pinecone(api_key="your-api-key")
index = pc.Index("your-index-name")

# Upsert vectors
index.upsert(vectors=[
    {
        "id": "chunk-abc123",
        "values": embedding,
        "metadata": {
            "text": "The default token expiration is 3600 seconds...",
            "source": "api-reference",
            "page": 42,
        }
    }
])

# Query
results = index.query(
    vector=query_embedding,
    top_k=10,
    include_metadata=True,
    filter={"source": {"$eq": "api-reference"}}
)
```

Strengths: zero operational overhead, automatic scaling, built-in metadata filtering, strong consistency. **Weaknesses:** cost at scale (per vector stored $\times$ queries per month), vendor lock-in, no self-hosting option, limited control over index parameters.

14.3.2 Weaviate

Open-source, self-hostable, also available as a managed cloud service. Module-based architecture allows integrating embedding models, rerankers, and generative models directly into the database.

```
import weaviate
import weaviate.classes.config as wc

client = weaviate.connect_to_local()

collection = client.collections.create(
    name="DocumentChunk",
    vectorizer_config=wc.Configure.Vectorizer.text2vec_openai(),
    properties=[
        wc.Property(name="text",   data_type=wc.DataType.TEXT),
        wc.Property(name="source", data_type=wc.DataType.TEXT),
    ]
)

# Hybrid search (vector + BM25) - Weaviate's primary differentiator
results = collection.query.hybrid(
```

```
    query="session timeout default",
    limit=10,
    alpha=0.7,  # 1.0 = pure vector, 0.0 = pure BM25
)
```

Strengths: built-in hybrid search, integrated reranking, active development, self-hostable. **Weaknesses:** complex schema management, operational overhead for self-hosted.

14.3.3 Qdrant

Open-source, designed for high-performance filtered vector search. Written in Rust for low latency and memory efficiency. Supports complex payload-based filtering with high performance even under heavy filter conditions.

```
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    Filter, FieldCondition, MatchValue
)

client = QdrantClient(url="http://localhost:6333")

client.create_collection(
    collection_name="document_chunks",
    vectors_config=VectorParams(size=1024, distance=Distance.COSINE),
)

client.upsert(
    collection_name="document_chunks",
    points=[
        PointStruct(
            id="chunk-abc123",
            vector=embedding,
            payload={
                "text": "The default token expiration is 3600 seconds...",
                "source": "api-reference",
                "access_level": "internal",
            }
        )
    ]
)

results = client.search(
    collection_name="document_chunks",
    query_vector=query_embedding,
    query_filter=Filter(
        must=[FieldCondition(
            key="access_level",
            match=MatchValue(value="internal")
        )]
    ),
    limit=10,
)
```

Strengths: excellent filtered search performance, Rust performance, memory efficiency, native multi-vector support for ColBERT. **Weaknesses:** smaller ecosystem than Pinecone or Weaviate.

14.3.4 Milvus / Zilliz

Milvus is an open-source vector database designed for large-scale deployments (billions of vectors). Zilliz Cloud is the managed version.

```
from pymilvus import MilvusClient

client = MilvusClient(uri="http://localhost:19530")

client.create_collection(
    collection_name="document_chunks",
    dimension=1024,
)

client.insert(
    collection_name="document_chunks",
    data=[{
        "id": 1,
        "vector": embedding,
        "text": "The default token expiration is 3600 seconds...",
        "source": "api-reference",
    }]
)

results = client.search(
    collection_name="document_chunks",
    data=[query_embedding],
    limit=10,
    output_fields=["text", "source"],
)
```

Strengths: scales to billions of vectors, GPU acceleration support, rich index options (HNSW, IVF, DiskANN). **Weaknesses:** complex deployment (multiple components), significant operational overhead.

14.3.5 pgvector

PostgreSQL extension that adds vector storage and search to an existing Postgres database. Supports both exact and HNSW search.

```
CREATE EXTENSION vector;

CREATE TABLE document_chunks (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    text        TEXT NOT NULL,
    source      TEXT,
    embedding   vector(1024),
    created_at  TIMESTAMPTZ DEFAULT now()
);

-- HNSW index
CREATE INDEX ON document_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Filtered vector search in standard SQL
SELECT id, text, source,
```

```
       1 - (embedding <=> $1::vector) AS similarity
FROM document_chunks
WHERE source = 'api-reference'
ORDER BY embedding <=> $1::vector
LIMIT 10;
```

Strengths: no new infrastructure if you already run Postgres, full SQL expressiveness for filtering, ACID transactions, familiar operational model, transactional consistency between vector and non-vector data. **Weaknesses:** performance at very large scale (tens of millions of vectors) lags behind dedicated systems; HNSW index takes significant memory.

14.3.6 ChromaDB

Lightweight, embedded vector database for prototyping and small-scale applications.

```
import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.create_collection("document_chunks")

collection.add(
    ids=["chunk-abc123"],
    embeddings=[embedding],
    documents=["The default token expiration is 3600 seconds..."],
    metadatas=[{"source": "api-reference"}]
)

results = collection.query(
    query_embeddings=[query_embedding],
    n_results=10,
    where={"source": "api-reference"},
)
```

Strengths: zero infrastructure, easy to get started, good for development and prototyping. **Weaknesses:** limited production scalability, no operational features of dedicated systems.

14.4 Metadata filtering

Most RAG applications need to restrict retrieval to a subset of the corpus. A user in the legal department should retrieve from legal documents; a query about 2024 regulations should not retrieve 2018 regulations. Metadata filtering restricts the search space to vectors matching a filter condition before or during similarity search.

14.4.1 Pre-filtering vs. post-filtering

Pre-filtering applies the filter first, then searches only among matching vectors:

```
Filter: source = "api-reference" → 50,000 vectors match
Search: find top-10 nearest among those 50,000
```

Accurate but can be slow if the filter does not reduce the search space significantly, or if the index cannot efficiently restrict to filtered vectors.

Post-filtering searches first, then filters results:

```
Search: find top-100 nearest among all 1,000,000 vectors
Filter: keep only results where source = "api-reference" → 8 results
```

Fast for the search step (we are using ANN, not bruteforce!) but may return fewer than k results after filtering. Inflating the top-k (search for 1,000, filter to 10) compensates but increases search cost.

Hybrid filtering is the most efficient approach: use the filter to restrict the index segments searched (coarse pre-filtering) and apply exact filter checking to candidates from the restricted search. Qdrant's payload index, Weaviate's inverted index, and pgvector's SQL WHERE clauses all enable this.

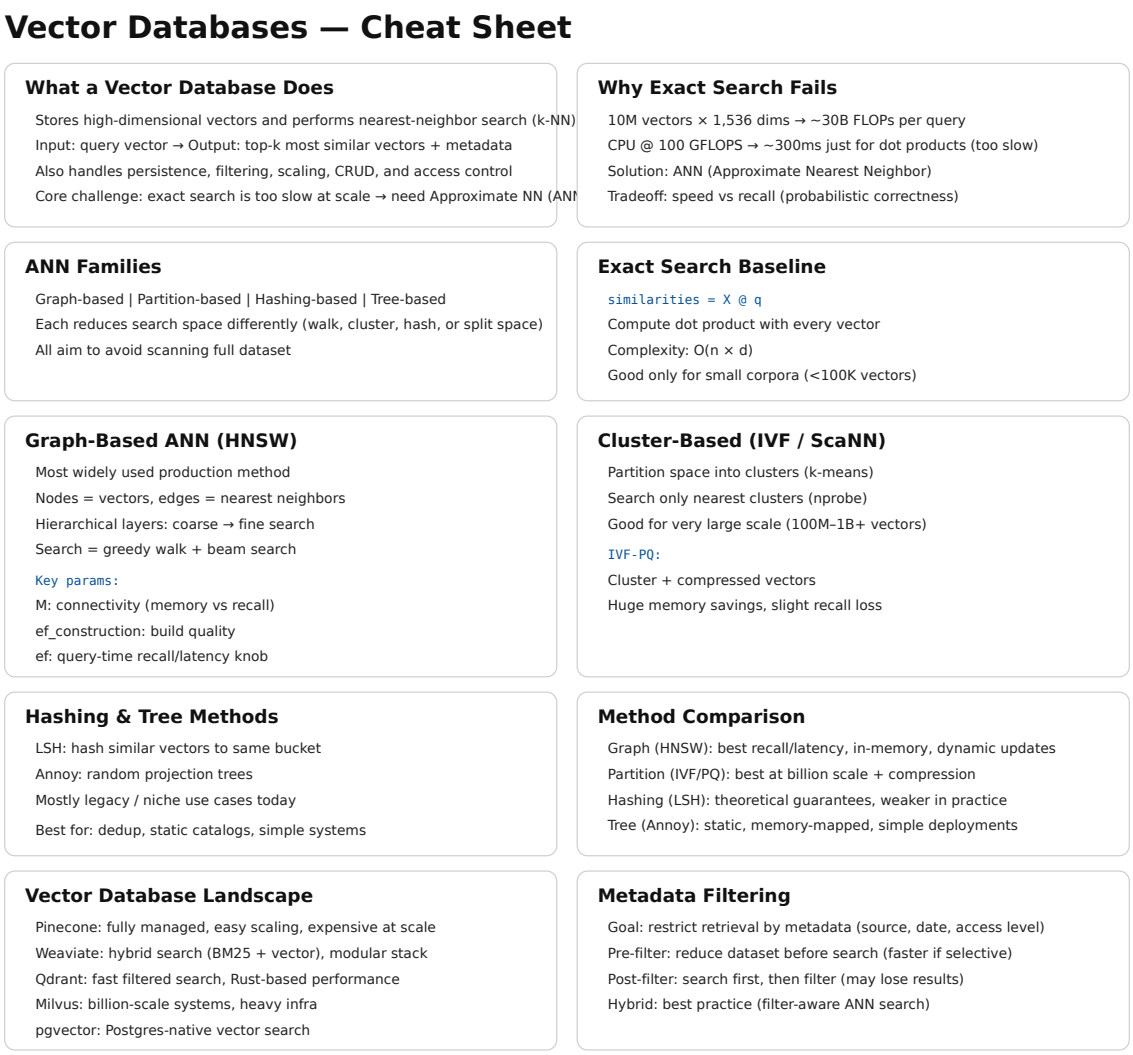


Figure 14.2: Cheat sheet.

14.5 Key takeaways

- Exact nearest-neighbor search is O(n × d) per query — too slow for interactive systems at scale; ANN algorithms (HNSW, IVF) trade a small recall loss for orders-of-magnitude speedup
- HNSW is the dominant ANN algorithm in production: layered graph structure gives O(log n) search; the `ef` parameter adjusts the recall/latency tradeoff at query time without index rebuild
- Product quantization compresses vectors by 4–64× at the cost of approximate distances; use when memory is the constraint and some recall loss is acceptable

- Choosing between vector databases depends on corpus size, operational model (managed vs. self-hosted), filtering complexity, and existing infrastructure
- pgvector is the right choice when you already run Postgres and your corpus is under ~5M vectors; dedicated systems (Qdrant, Weaviate, Pinecone) are better at scale
- Metadata filtering restricts search to relevant subsets; pre-filtering is accurate, post-filtering is fast, hybrid filtering combines the benefits; index the fields you filter on
- Multi-tenancy isolation is safest with namespaces or collections per tenant; metadata-based isolation relies on application-layer enforcement and is risky for strict data isolation requirements
- Store raw vectors durably in object storage as the canonical source of truth; the vector index is a derived artifact that can be rebuilt
- Hybrid retrieval (vector + BM25) consistently outperforms pure vector search for production RAG; prefer systems with native hybrid search support

---

14.6 Further reading

- Malkov & Yashunin (2018). *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.* — The original HNSW paper; the foundational algorithm in most production vector databases.
- Johnson et al. (2019). *Billion-Scale Similarity Search with GPUs.* — The FAISS paper; foundational for large-scale ANN and IVF.
- Jégou et al. (2011). *Product Quantization for Nearest Neighbor Search.* — The PQ paper; essential for understanding compressed vector search.
- Aguerrebere et al. (2023). *Similarity Search in the Blink of an Eye with Compressed Indices.* — ScaNN; Google's production ANN system with strong benchmark performance.
- Douze et al. (2024). *The FAISS Library.* — Comprehensive FAISS documentation and benchmarks across index types.

Chapter 15

Retrieval Algorithms

The canonical question for this chapter: *Given a user query and a corpus of embedded chunks, what algorithms actually find the right chunks, and why is vector similarity alone not enough?*

Where are we?

Figure 15.1: The journey through the Model Mind.

The vector database is now in ready, and the queries are coming to be processed. This chapter explores the algorithms that transform a natural language query into a ranked list of relevant chunks, and explains why combining multiple retrieval methods consistently delivers better results than relying on any single approach. In a Retrieval-Augmented Generation (RAG) system, the retrieval stage has the greatest impact on answer quality, often more than the choice of language model or prompt design.

15.1 The retrieval problem

Retrieval is deceptively simple to describe: given a query, return the most relevant chunks. In practice, no single algorithm reliably finds the right chunks across all query types, corpus structures, and domain vocabularies.

Vector similarity search finds semantically related content but fails on exact string matches, rare terms, and out-of-distribution vocabulary. Keyword search (BM25) finds exact matches but fails when the query and document use different words for the same concept. Re-ranking models produce better relevance signals than either but are too slow to run over the full corpus. Each algorithm has failure modes the others compensate for.

Production RAG systems that work reliably across diverse query types combine multiple retrieval algorithms, using each where it performs best. This chapter covers each algorithm in depth, then covers how to combine them.

15.2 Dense retrieval: vector similarity search

Dense retrieval embeds the query and retrieves chunks whose embeddings are nearest to the query embedding in vector space.

15.2.1 What dense retrieval is good at

Semantic matching. The query "how long before my session expires" retrieves the chunk "the default token expiration is 3600 seconds" because the embeddings are nearby even though no words overlap.

Paraphrase matching. The query "terminate an employee" retrieves documents about "offboarding" and "separation from the company", different words, same concept.

Cross-lingual retrieval. With a multilingual embedding model, an English query retrieves a French document about the same topic.

Conceptual queries. "Explain the difference between authentication and authorization" retrieves relevant conceptual explanations even when the query phrasing does not match any document passage verbatim.

15.2.2 What dense retrieval fails at

Rare and out-of-vocabulary terms. The query "error code E1042" may produce a poor embedding if E1042 is rare in the embedding model's training data. The model has no strong statistical association to learn from, so E1042 gets embedded near generic "error code" content rather than near the specific documentation for E1042. The correct chunk may rank 500th.

Exact string matching. Product names, model numbers, person names, and technical identifiers must match exactly. "GPT-4o" and "GPT-4" should not be treated as semantically equivalent, they are different products and embedding models can easily conflate them.

High-specificity queries. "What is the difference between TCP and UDP?" embeds similarly to "compare two network protocols." Dense retrieval may surface documents about other protocol comparisons that are not specifically about TCP vs. UDP.

Numerical and structured queries. "Find all incidents from Q3 2024" requires exact date matching. Embedding similarity cannot handle structured numerical constraints.

15.3 Sparse retrieval: BM25

BM25 (Best Match 25) is the dominant keyword retrieval algorithm. It extends TF-IDF with term frequency saturation and length normalization, producing relevance scores that behave well across a wide range of corpus and query types.

15.3.1 The BM25 formula

For a query Q and document D:

$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \times \frac{\text{TF}(t, D) \times (k_1 + 1)}{\text{TF}(t, D) + k_1 \left(1 - b + b \times \frac{|D|}{\text{avgdl}}\right)}$$

Where `t` iterates over query terms, `TF(t,D)` is term frequency in document D, `IDF(t)` is $\log\left(\frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} + 1\right)$, N is total documents, df(t) is documents containing term t, |D| is document length in tokens, avgdl is average document length, k1 is the term frequency saturation parameter (typically 1.2-2.0), and b is the length normalization parameter (typically 0.75).

IDF penalizes terms that appear in many documents ("the" and "is" get near- zero IDF and a rare technical term gets high IDF).

TF saturation (k1 parameter) limits the advantage of high repetition. A term appearing 100 times scores only marginally higher than one appearing 10 times.

Length normalization (b parameter) favors shorter documents. A document that mentions a query term once in 100 words scores higher than one that mentions it once in 10,000 words.

15.3.2 What BM25 is good at

Exact term matching. "Error code E1042" retrieves documents containing "E1042" precisely, regardless of whether the embedding model has seen E1042 in training.

Named entities. Person names, organization names, product names, and proper nouns are retrieved reliably by exact match.

Technical and domain-specific terminology. Medical codes, legal citations, scientific terms, and other domain vocabulary with precise meaning are handled by exact matching.

15.3.3 What BM25 fails at

Vocabulary mismatch. "Car" and "automobile" are the same concept but BM25 treats them as different terms. The query "automobile maintenance" does not retrieve documents about "car repair."

Conceptual queries. "Explain how transformers work" has no single discriminating term. Every technical document might contain "work"; many contain "transformer." BM25 cannot reason about the conceptual relationship.

Short queries. BM25 degrades for one or two word queries because there are few terms to score. "Timeout" as a query retrieves all documents mentioning timeout regardless of whether they are about network timeouts, session timeouts, or database timeouts.

15.4 Hybrid retrieval

Combining dense and sparse retrieval consistently outperforms either alone on real world RAG benchmarks. Their failure modes are complementary: BM25 catches exact matches that dense retrieval misses; dense retrieval catches semantic matches that BM25 misses.

15.4.1 Reciprocal Rank Fusion (RRF)

RRF is the simplest and most robust fusion method that relies only on rank positions rather than score magnitudes, eliminating the need for score normalization.

For a set of retrieval systems, each producing a ranked list, RRF score for a chunk `c` is defined as:

$$\mathrm{RRF}(c) = \sum_{s \in S} \frac{1}{k + \mathrm{rank}_s(c)}$$

where `S` represents the set of retrieval systems, and $\mathrm{rank}_s(c)$ denotes the 1-indexed rank of chunk `c` in the result list returned by system `s`. The parameter `k` is a smoothing constant, typically set to 60, that controls the influence of lower-ranked results. If a chunk does not appear in a system's result list, that system contributes a score of 0 for the chunk.

Why RRF is robust. RRF relies on rank positions rather than raw retrieval scores. For example, a chunk with a dense retrieval score of 0.99 receives the same contribution as a chunk with a score of 0.51 if both are ranked first.

This avoids the need to normalize or align score scales across different retrieval systems. Additionally, RRF naturally handles varying result sets by rewarding chunks that consistently appear across multiple systems

RRF in practice. Retrieve a larger candidate pool from each system (e.g., top-50 or top-100 results) before applying fusion. RRF can promote chunks that are ranked moderately across multiple systems, for example, a chunk ranked 30th by one system and 5th by another may outperform a chunk appearing only in the top-10 of a single system.

15.4.2 Weighted score combination

This is an alternative approach to normalize scores from each retrieval system to the same range, typically ([0,1]), and combine them using system-specific weights:

`hybrid_score =  × normalized_dense_score + (1 −  ) × normalized_bm25_score`

Where `= 1.0` is pure dense and `= 0.0` is pure sparse. Weaviate uses this with its `alpha` parameter. The limitation: BM25 scores are unbounded and skewed; cosine similarity is bounded to [-1, 1] and normalization artifacts can cause instability.

RRF is generally more robust than weighted combination in practice because it does not require score normalization. Use weighted combination only when you have domain-specific knowledge about the relative importance of semantic vs. keyword matching for your query distribution.

15.5 Query expansion and reformulation

Retrieval quality depends heavily on query phrasing. Users phrase queries differently from how documents phrase answers. Query expansion and reformulation bridge this gap.

Synonym expansion Expand queries with synonyms and related terms before performing BM25 retrieval. Simple synonym expansion can improve recall by matching additional relevant terms, but it may also introduce noise by adding less relevant matches. Careful tuning of the BM25 `k_1` parameter can help reduce the impact of these added terms on the final scoring.

Multi-query retrieval. Generate multiple semantically equivalent or complementary reformulations of the original query and retrieve results for each. The retrieved results are then merged and deduplicated to produce a more comprehensive candidate set. This approach consistently improves recall, particularly for ambiguous, underspecified, or poorly phrased queries. Although generating multiple queries with an LLM typically adds 200–500 ms of latency, the improvement in retrieval quality often justifies the additional cost when answer accuracy is the primary objective.

Step-back prompting. For complex analytical queries, first generate or retrieve information for a broader, more general "step-back" question and use it as additional context for answering the original query. This provides foundational knowledge that may be missing from the specific question.

For example, the query *"What causes the token refresh to fail?"* can benefit from retrieving background information on *"How does token refresh work?"* first. The broader context helps explain the underlying mechanism and makes it easier to identify and answer the specific failure causes.

15.6 Contextual and conversational retrieval

In a multi-turn conversation, follow-up messages are often not self-contained. "What about for enterprise customers?" makes no sense as a standalone retrieval query without the previous turn's context.

Standalone query generation Rewrite the user's message into a self-contained query that incorporates the necessary context from previous conversation turns before performing retrieval.

Without standalone query generation, retrieval for follow-up questions often fails because the query lacks the context needed to identify the intended topic. For example, *"What about for enterprise customers?"* may retrieve general information about enterprise customers rather than information related to the specific subject discussed earlier. Generating standalone queries is one of the most common and effective improvements for conversational RAG systems.

Conversation-aware embedding, A faster alternative to standalone query generation is to concatenate recent conversation context with the current user query before creating the embedding.

This approach avoids the additional latency of an LLM-based rewriting step, but it is generally less reliable because the combined text may not accurately represent the user's true information need. The embedding can be influenced by irrelevant conversational details rather than focusing on the core query intent. Use standalone query generation when retrieval quality is the priority, and conversation-aware embedding when lower latency is more important.

15.7 Knowledge graph retrieval

For some domains, structured knowledge in graph form enables retrieval that pure text search cannot support.

15.7.1 When graphs help

Consider a question about a complex regulatory requirement that depends on: the regulation text (in documents), which entities are subject to it (structured data), how it relates to other regulations (graph relationships), and which exceptions apply (structured rules). Text retrieval finds the regulation document. It cannot navigate the relationships between regulatory entities, exceptions, and cross-references.

GraphRAG (Edge et al., 2024) extracts entities and relationships from documents and builds a knowledge graph. Retrieval traverses the graph to gather related entities before generating a response.

15.7.2 Entity extraction for graph building

```
def extract_entities_and_relations(text: str, llm) -> dict:
    prompt = f"""Extract entities and relationships from the following text.
Return a JSON object with:
- "entities": list of {{name, type, description}} objects
- "relations": list of {{subject, predicate, object}} triples

Text: {text}"""

    response = llm.complete(prompt)
    return json.loads(response)

# Build graph incrementally from corpus
import networkx as nx

graph = nx.DiGraph()
for chunk in corpus_chunks:
    extracted = extract_entities_and_relations(chunk.text, llm)

    for entity in extracted["entities"]:
        graph.add_node(entity["name"], **entity)

    for relation in extracted["relations"]:
        graph.add_edge(
            relation["subject"], relation["object"],
```

```
            label=relation["predicate"],
            source_chunk=chunk.id
        )
```

15.7.3 Graph retrieval at query time

```
def graph_retrieve(
    query: str, graph, text_retriever, k: int = 5
) -> list:
    query_entities = extract_entities(query)

    # Expand to related entities via graph traversal
    expanded = set(query_entities)
    for entity in query_entities:
        if entity in graph:
            expanded.update(nx.neighbors(graph, entity))

    # Gather chunks associated with expanded entities
    entity_chunks = []
    for entity in expanded:
        entity_chunks.extend(graph.nodes[entity].get("source_chunks", []))

    # Merge with standard text retrieval
    text_results = text_retriever.retrieve(query, top_k=k)
    return list(set(entity_chunks) | set(r[0] for r in text_results))
```

GraphRAG is powerful for corpora with rich entity relationships (medical knowledge bases, legal systems, financial networks) but adds significant ingestion complexity (entity extraction per chunk) and retrieval complexity (graph traversal). —

15.8 Retrieval evaluation

15.8.1 Metrics

Recall@k: of all truly relevant chunks, what fraction appear in the top-k retrieved chunks?

$$\text{Recall@k} = \frac{|\{\text{relevant}\} \cap \{\text{top-k retrieved}\}|}{|\{\text{relevant}\}|}$$

This is the primary metric for RAG. If the relevant chunk is not in the top-k, the language model cannot use it and no downstream component can compensate.

Precision@k: of the top-k retrieved chunks, what fraction are truly relevant?

$$\text{Precision@k} = \frac{|\{\text{relevant}\} \cap \{\text{top-k retrieved}\}|}{k}$$

Mean Reciprocal Rank (MRR): average reciprocal rank of the first relevant result across queries.

$$\text{MRR} = \frac{1}{|\text{queries}|} \sum_{q \in \text{queries}} \frac{1}{\text{rank\_of\_first\_relevant}(q)}$$

NDCG@k: accounts for graded relevance and position. A relevant chunk at rank 1 contributes more than the same chunk at rank 5.

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

$$\text{DCG@k} = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

$$\text{IDCG@k} = \sum_{i=1}^{k} \frac{2^{rel_i^*} - 1}{\log_2(i+1)}$$

15.9 The full retrieval pipeline

A production pipeline combining the techniques from this chapter:

```python
class HybridRetriever:
    def __init__(self, vector_db, bm25_index, embedding_model, llm):
        self.vector_db     = vector_db
        self.bm25          = bm25_index
        self.embedding_model = embedding_model
        self.llm           = llm

    def retrieve(
        self,
        query: str,
        conversation_history: list[dict] | None = None,
        filters: dict | None = None,
        top_k: int = 10,
    ) -> list[dict]:

        # Step 1: Resolve conversational context
        retrieval_query = query
        if conversation_history:
            retrieval_query = generate_standalone_query(
                conversation_history, query, self.llm
            )

        # Step 2: Dense retrieval
        query_embedding = self.embedding_model.encode(
            f"query: {retrieval_query}", normalize_embeddings=True
        )
        dense_results = self.vector_db.search(
            query_embedding, top_k=50, filters=filters
        )

        # Step 3: Sparse retrieval
        sparse_results = self.bm25.search(retrieval_query, top_k=50)

        # Step 4: Fuse with RRF
        fused = reciprocal_rank_fusion([dense_results, sparse_results], k=60)
        candidates = fused[:100]  # top-100 for re-ranking
```

```
        # Step 5: Re-rank (chapter 23)
        # reranked = reranker.rerank(retrieval_query, candidates, top_k)

        # Step 6: Return top results
        return [self.get_chunk(chunk_id) for chunk_id, _ in candidates[:top_k]]
```

15.10 Key takeaways

- No single retrieval algorithm works well for all query types; dense retrieval excels at semantic matching, BM25 excels at exact term matching, and their failure modes are complementary — hybrid retrieval consistently outperforms either alone
- BM25 scores by term frequency (saturated), inverse document frequency, and length normalization; it is essential for exact matching of rare terms, product codes, and named entities that embedding models handle poorly
- Reciprocal Rank Fusion is the most robust fusion method because it depends only on rank positions, not score magnitudes — no normalization required; retrieve top-50 or top-100 from each system before fusing
- Multi-query retrieval generates paraphrase variants and fuses results, improving recall for ambiguous queries at the cost of one LLM call per query
- Standalone query generation is mandatory for multi-turn conversations; direct retrieval on follow-up questions without context rewriting consistently fails and is one of the most common fixable RAG failures
- GraphRAG adds entity-relationship traversal for domains with rich structured relationships; adds significant complexity and is only worth it when text retrieval quality is provably insufficient for relationship-dependent queries
- Recall@k on a held-out evaluation set is the only reliable way to measure retrieval quality; build this before tuning anything else in the RAG pipeline

---

15.11 Further reading

- Robertson & Zaragoza (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* — The authoritative BM25 reference; explains the mathematical motivation for saturation and length normalization.
- Karpukhin et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering.* — established dense retrieval for question answering.
- Cormack et al. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.* — The RRF paper.
- Ma et al. (2023). *Fine-Tuning LLaMA for Multi-Stage Text Retrieval.* — LLM- based query reformulation for improved retrieval.
- Edge et al. (2024). *From Local to Global: A Graph RAG Approach to Query- Focused Summarization.* — GraphRAG; entity-based retrieval for complex multi-document reasoning.
- Es et al. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation.* — End-to-end RAG evaluation framework including retrieval metrics.
- Thakur et al. (2021). *BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models.* — The standard benchmark for comparing retrieval algorithms across diverse domains.

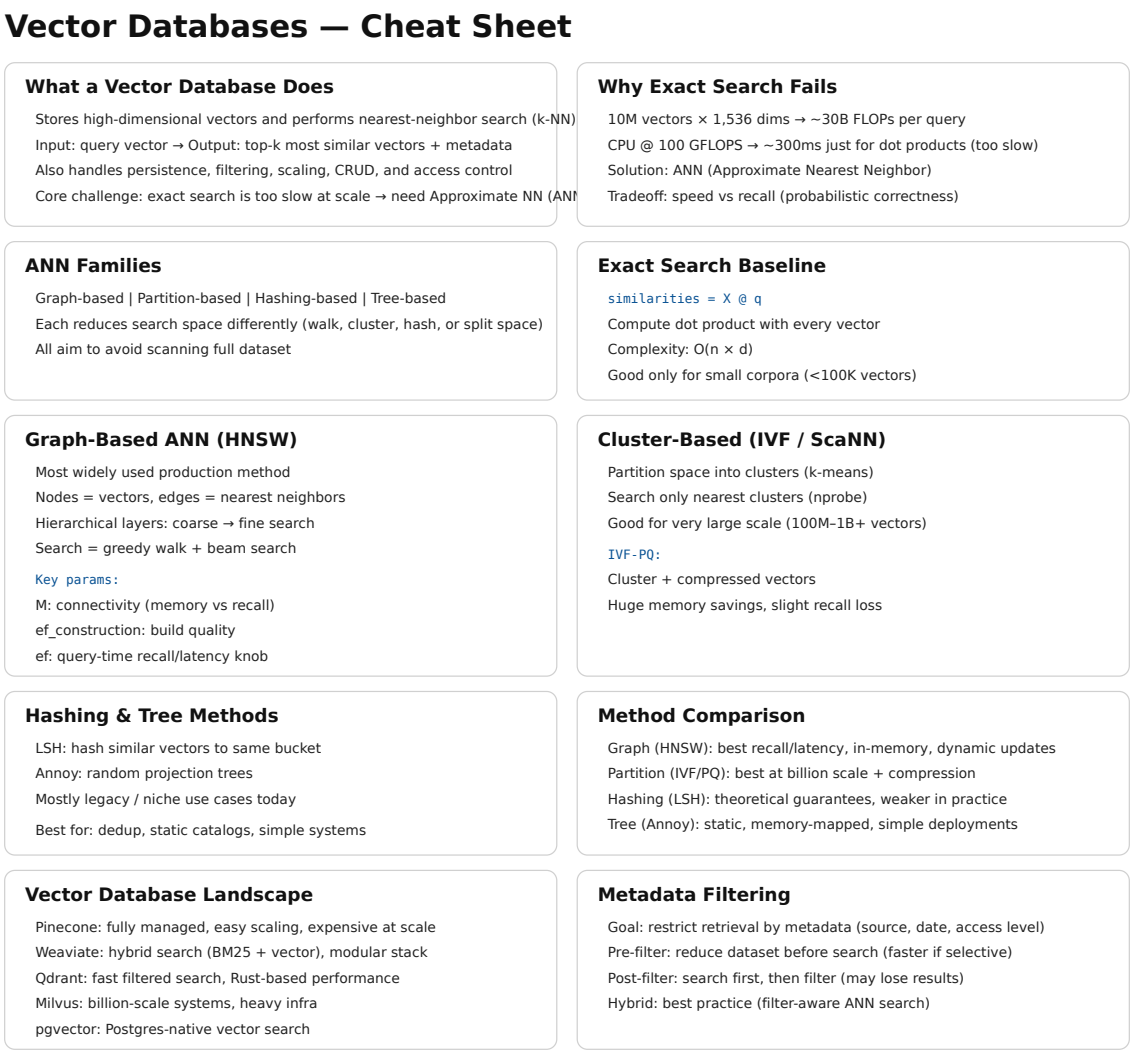


Figure 15.2: Cheat sheet.

Chapter 16

Reranking

The canonical question for this chapter: *Initial retrieval returns 50 candidates. The language model can use 10. How do you pick the right 10 and why is the answer more subtle than just sorting by embedding similarity?*

Where are we?

Figure 16.1: The journey through the Model Mind.

Initial retrieval maximized recallthat helps to find everything that might be relevant. Now the problem inverts: among 50 candidates, which 10 should the language model actually see? Passing all 50 wastes context budget, dilutes signal with noise, and increases the probability of lost-in-the-middle failures. Reranking is the precision stage, the step that decides which evidence the model gets to work with.

16.1 Why reranking exists as a separate stage

Recall and precision pull in opposite directions. A retriever that returns 50 candidates to maximize recall will include: chunks that directly answer the query, chunks that are topically related but do not answer the specific question, chunks that share surface features with the query but are semantically unrelated, and chunks that answer part of a multi-part query but not the whole thing.

The language model receives whatever context you pass it. If you pass 50 chunks (many irrelevant) several failure modes emerge. The model spends context window budget on noise. Relevant information is diluted among irrelevant information amd finally long contexts increase the probability of the lost-in-the-middle phenomenon, where the model fails to use information in the middle of a long context.

Reranking is the precision stage: given 50 candidates from initial retrieval, identify the 10 most likely to help the model answer the query correctly.

16.2 The spectrum of reranking approaches

Reranking methods form a spectrum from fast-and-approximate to slow-and-accurate:

```
Fast                                                          Accurate


Score-based    Bi-encoder     Late interaction    Cross-encoder    LLM judge
(retrieval     (embedding     (ColBERT           (BERT-based      (GPT-4 as
 scores)        similarity)    MaxSim)            reranker)        evaluator)
     ↑              ↑              ↑                  ↑                ↑
Already done   Initial          Medium             Best practical    Expensive;
in retrieval   retrieval        cost/quality       quality/cost      use for
               produces this    tradeoff           tradeoff          offline tasks
```

"Reranking" in common usage refers to applying a cross-encoder or late- interaction model as a second stage after initial retrieval. The earlier parts of the spectrum are usually considered part of retrieval itself.

16.3 Score-based reranking

The simplest approach: use the scores from initial retrieval to filter candidates before passing them to the language model.

16.3.1 Threshold filtering

```python
def threshold_filter(
    candidates: list[tuple[str, float]],
    min_score: float = 0.7,
) -> list[tuple[str, float]]:
    return [
        (chunk_id, score) for chunk_id, score in candidates
        if score >= min_score
    ]
```

This works for cosine similarity scores but requires careful calibration. The right threshold depends on the embedding model (different models have different score distributions), the query type, and the corpus. A threshold tuned on one distribution fails on another.

Thresholds on BM25 scores are even less reliable these scores are unbounded and vary with corpus size and document length distribution.

16.3.2 Why score-based filtering alone is insufficient

Embedding similarity is not the same as relevance. A chunk about a different product from the same company might have a cosine similarity of 0.85 with the query (above any reasonable threshold) while being completely irrelevant to the specific question. Score-based filtering works only when similarity correlates well with relevance, which holds for queries within the embedding model's training distribution and breaks for out-of-distribution queries, domain-specific content, or multi-hop reasoning.

16.4 Cross-encoder reranking

Cross-encoders are the most widely used practical reranking approach, they consistently outperform bi-encoders for relevance scoring and fit within interactive latency budgets when deployed on GPU.

16.4.1 Why cross-encoders are more accurate than bi-encoders

A bi-encoder encodes query and document independently:

```
Query  → Encoder → q_embedding (fixed vector)
Chunk  → Encoder → c_embedding (fixed vector)
Score  = cosine_similarity(q_embedding, c_embedding)
```

The query embedding does not "see" the chunk during encoding. Every query-chunk interaction happens at score computation which is a single dot product.

A cross-encoder takes both as input simultaneously:

```
[CLS] query tokens [SEP] chunk tokens [SEP]
                  → Encoder
                  → [CLS] representation
                  → scalar relevance score
```

Query and chunk tokens attend to each other through all attention layers. The model can detect that a chunk discusses the right concept in the wrong context, that a surface-level match is semantically irrelevant, or that a specific phrase in the query requires a specific phrase in the chunk that is absent.

This expressiveness is why cross-encoders consistently outperform bi-encoders for relevance scoring by 10–20 NDCG points on standard benchmarks. The tradeoff: cross-encoders cannot be pre-computed. Every query-chunk pair requires a fresh forward pass, making them too slow for full-corpus search but fast enough for a 50-candidate reranking step.

16.4.2 Cross-encoder inference

```python
from sentence_transformers import CrossEncoder
import numpy as np

class CrossEncoderReranker:
    def __init__(
        self,
        model_name: str = "cross-encoder/ms-marco-MiniLM-L-6-v2"
    ):
        self.model = CrossEncoder(model_name, max_length=512)

    def rerank(
        self,
        query: str,
        candidates: list[tuple[str, str]],  # (chunk_id, chunk_text)
        top_k: int = 10,
        batch_size: int = 32,
    ) -> list[tuple[str, float]]:
        if not candidates:
            return []

        pairs = [(query, chunk_text) for _, chunk_text in candidates]

        # Score in batches to avoid OOM on GPU
        all_scores = []
```

```
        for i in range(0, len(pairs), batch_size):
            batch = pairs[i:i + batch_size]
            scores = self.model.predict(batch, show_progress_bar=False)
            all_scores.extend(scores.tolist())

        scored = sorted(
            zip([chunk_id for chunk_id, _ in candidates], all_scores),
            key=lambda x: x[1],
            reverse=True,
        )
        return scored[:top_k]
```

16.4.3 Input length and truncation

Cross-encoders have a maximum input length (typically 512 tokens for BERT-based models). The query plus chunk must fit within this limit. If the chunk is too long, it must be truncated which may result in cuting the relevant portion.

Strategies for long chunks:

Sliding window: split the chunk into overlapping windows, score each, take the maximum as the chunk score. Most reliable but multiplies inference calls.

First-N tokens: relevant facts often appear near the start of chunks; simple and fast. Fails when the key passage is in the second half.

```
def rerank_long_chunks(
    query: str,
    candidates: list[tuple[str, str]],
    reranker: CrossEncoder,
    max_chunk_tokens: int = 400,
    stride: int = 200,
) -> list[tuple[str, float]]:
    chunk_scores = {}

    for chunk_id, chunk_text in candidates:
        tokens = chunk_text.split()  # simplified; use real tokenizer in production

        if len(tokens) <= max_chunk_tokens:
            score = reranker.model.predict([(query, chunk_text)])[0]
        else:
            window_scores = []
            for start in range(0, len(tokens), stride):
                window = " ".join(tokens[start:start + max_chunk_tokens])
                window_scores.append(reranker.model.predict([(query, window)])[0])
            score = max(window_scores)

        chunk_scores[chunk_id] = score

    return sorted(chunk_scores.items(), key=lambda x: x[1], reverse=True)
```

16.4.4 Cross-encoder model selection

The choice of cross-encoder involves quality, latency, and cost tradeoffs:

ms-marco-MiniLM-L-6-v2 (22M parameters): fastest option, good for low- latency requirements. ~39 MRR@10 on MS MARCO dev. The right default for most production applications.

ms-marco-MiniLM-L-12-v2 (33M parameters): better quality than L-6, roughly 2× slower. ~42 MRR@10. Worth the latency cost when quality is critical.

BGE-Reranker-v2-m3 (568M parameters): state-of-the-art open-source cross- encoder. Multilingual, strong across BEIR benchmarks. Requires GPU for acceptable latency.

Cohere Rerank 3 (commercial API): among the best retrieval quality available for English. Priced per API call. No GPU infrastructure required.

Voyage Rerank 2 (commercial API): strong competitor to Cohere, particularly for code and technical content.

16.5 LLM-based reranking

Using a large language model as the relevance judge. More accurate than cross- encoders for complex relevance judgments but significantly more expensive.

16.5.1 Pointwise LLM scoring

Score each candidate individually by asking the LLM whether it is relevant:

```
def llm_rerank_pointwise(
    query: str,
    candidates: list[tuple[str, str]],
    llm,
    top_k: int = 10,
) -> list[tuple[str, float]]:
    scored = []

    for chunk_id, chunk_text in candidates:
        prompt = f"""On a scale of 0 to 10, how relevant is the following passage
to answering this query? Respond with only a number.

Query: {query}

Passage: {chunk_text}

Relevance score (0-10):"""

        response = llm.complete(prompt).strip()
        try:
            score = float(response)
        except ValueError:
            score = 0.0
        scored.append((chunk_id, score))

    return sorted(scored, key=lambda x: x[1], reverse=True)[:top_k]
```

This requires one LLM call per candidate. For 50 candidates, that is 50 LLM calls it is expensive and slow (5–20 seconds depending on model and parallelism).

16.5.2 Listwise LLM reranking

Score all candidates together in a single prompt:

```
def llm_rerank_listwise(
    query: str,
    candidates: list[tuple[str, str]],
    llm,
    top_k: int = 10,
) -> list[tuple[str, float]]:
    candidates_text = "\n\n".join([
        f"[{i+1}] {chunk_text[:300]}..."
        for i, (_, chunk_text) in enumerate(candidates)
    ])

    prompt = f"""Rank the following passages by relevance to the query.
Return a JSON array of passage numbers in order of relevance (most relevant first).
Numbers only, no explanation.

Query: {query}

Passages:
{candidates_text}

Ranking (JSON array):"""

    response = llm.complete(prompt).strip()

    try:
        ranking = json.loads(response)
    except json.JSONDecodeError:
        ranking = list(range(1, len(candidates) + 1))

    result = []
    for rank, position in enumerate(ranking[:top_k]):
        idx = position - 1
        if 0 <= idx < len(candidates):
            result.append((candidates[idx][0], 1.0 / (rank + 1)))

    return result
```

One LLM call regardless of candidate count, but candidate text must fit in the prompt. For 50 candidates at 300-character truncation: approximately 15,000 characters per query is feasible but expensive.

16.5.3 Sliding window listwise reranking (RankGPT)

For long candidate lists that do not fit in a single prompt:

```
def rankgpt_rerank(
    query: str,
    candidates: list[tuple[str, str]],
    llm,
    window_size: int = 20,
    step_size: int = 10,
    top_k: int = 10,
) -> list[tuple[str, float]]:
    ranked = list(candidates)
```

```
    # Slide window from bottom to top
    for start in range(len(ranked) - window_size, -1, -step_size):
        end = min(start + window_size, len(ranked))
        window = ranked[start:end]
        window_ranking = llm_rerank_listwise(query, window, llm, top_k=len(window))
        id_to_rank = {cid: r for r, (cid, _) in enumerate(window_ranking)}
        window.sort(key=lambda x: id_to_rank.get(x[0], len(window)))
        ranked[start:end] = window

    return [
        (chunk_id, 1.0 / (rank + 1))
        for rank, (chunk_id, _) in enumerate(ranked[:top_k])
    ]
```

RankGPT achieves high-quality rankings (comparable to or better than cross- encoders on several benchmarks) but requires many LLM calls for long candidate lists. It is impractical for real-time applications.

16.5.4 When to use LLM reranking

LLM reranking is appropriate for offline processing (generating training data for a smaller reranker), high-value queries where stakes justify cost and latency (legal document review, medical information retrieval), complex relevance judgments requiring multi-hop reasoning, and evaluation (grading retrieval quality at scale). For most interactive applications, cross-encoder reranking provides the best quality/cost/latency tradeoff.

16.6 Diversity-aware reranking

Relevance alone is not sufficient. If the top 10 most relevant chunks all say the same thing (different phrasings of the same fact) passing all 10 wastes context window budget and adds no new information.

Diversity-aware reranking selects a set that is both relevant and diverse, covering different aspects of the query.

16.6.1 Maximum Marginal Relevance (MMR)

Maximum Marginal Relevance (MMR) iteratively selects chunks that maximize a combination of relevance to the query and distance from already-selected chunks. It balances retrieving highly relevant chunks while avoiding selecting multiple chunks that contain the same information.

$$\text{MMR}(D_i) = \lambda \,\text{Sim}(D_i, Q) - (1 - \lambda) \max_{D_j \in S} \text{Sim}(D_i, D_j)$$

where: D_i is a candidate chunk being evaluated, Q is the query, S is the set of chunks already selected, $\text{Sim}(D_i, Q)$ measures relevance between the chunk and query, $\text{Sim}(D_i, D_j)$ measures redundancy between the candidate chunk and previously selected chunks, and λ controls the trade-off between relevance and diversity.

At each iteration, MMR chooses:

$$D^* = \arg \max_{D_i \in R \setminus S} \left[\lambda \,\text{Sim}(D_i, Q) - (1 - \lambda) \max_{D_j \in S} \text{Sim}(D_i, D_j) \right]$$

where R is the set of all retrieved candidate chunks.

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
```

```python
def mmr(
    query_embedding: np.ndarray,
    candidate_embeddings: np.ndarray,
    candidate_ids: list[str],
    top_k: int = 10,
    lambda_param: float = 0.5,  # 0 = pure diversity, 1 = pure relevance
) -> list[str]:
    query_similarities = (candidate_embeddings @ query_embedding).flatten()

    selected_indices = []
    remaining_indices = list(range(len(candidate_ids)))

    for _ in range(min(top_k, len(remaining_indices))):
        if not selected_indices:
            best_idx = remaining_indices[
                np.argmax(query_similarities[remaining_indices])
            ]
        else:
            selected_embeddings = candidate_embeddings[selected_indices]
            mmr_scores = []
            for idx in remaining_indices:
                relevance = query_similarities[idx]

                max_sim_to_selected = cosine_similarity(
                    [candidate_embeddings[idx]], selected_embeddings
                )[0].max()

                mmr_score = (
                    lambda_param * relevance
                    - (1 - lambda_param) * max_sim_to_selected
                )

                mmr_scores.append((idx, mmr_score))

            best_idx = max(mmr_scores, key=lambda x: x[1])[0]

        selected_indices.append(best_idx)
        remaining_indices.remove(best_idx)

    return [candidate_ids[i] for i in selected_indices]
```

lambda_param = 1.0 is pure relevance, meaning MMR becomes equivalent to sorting chunks by query similarity.

lambda_param = 0.0 is pure diversity, selecting chunks that are maximally different from previously selected chunks.

lambda_param = 0.5 provides an equal balance between relevance and diversity and is a common default value.

MMR is computationally cheap because it operates on pre-computed embedding vectors and cosine similarities. It is particularly effective for retrieval-augmented generation (RAG) systems and document retrieval pipelines where highly relevant chunks often contain overlapping or redundant information.

16.6.2 When diversity matters

For factual queries with a single answer, diversity is counterproductive, similar chunks provide useful redundancy if the most relevant one is imprecise.

For multi-part queries, complex questions, or queries requiring synthesis from multiple sources, diversity is essential. "What are the pros and cons of OAuth 2.0?" benefits from chunks discussing both advantages and disadvantages, not five chunks about the same advantage. Use diversity-aware selection for summarization tasks, "compare/contrast" queries, "list" queries, and corpora with heavy content duplication.

16.7 Contextual compression

Rather than selecting which chunks to include, contextual compression extracts only the relevant portion of each retrieved chunk which results in freeing context budget for more chunks or longer reasoning.

16.7.1 LLM extraction

```
def compress_chunk(query: str, chunk: str, llm) -> str:
    prompt = f"""Extract the portion of the following passage that is relevant
to answering the query. If nothing is relevant, return "IRRELEVANT".
Return only the extracted text, no explanation.

Query: {query}

Passage: {chunk}

Relevant excerpt:"""

    result = llm.complete(prompt).strip()
    return "" if result.upper() == "IRRELEVANT" else result
```

A 500-token chunk might compress to a 50-token relevant excerpt. The cost: one LLM call per retrieved chunk. For 10 chunks, that is 10 LLM calls before the final generation call.

16.7.2 Sentence-level extraction

A cheaper alternative: extract only sentences similar to the query using embedding similarity.

```
def extract_relevant_sentences(
    query: str,
    chunk: str,
    embedding_model,
    top_k_sentences: int = 3,
    min_similarity: float = 0.4,
) -> str:
    sentences = split_into_sentences(chunk)
    if not sentences:
        return chunk

    q_emb = embedding_model.encode(query, normalize_embeddings=True)
    s_embs = embedding_model.encode(sentences, normalize_embeddings=True)
    similarities = s_embs @ q_emb

    relevant = [
        i for i, sim in enumerate(similarities) if sim >= min_similarity
    ]
    relevant = sorted(relevant, key=lambda i: similarities[i], reverse=True)[:top_k_sentences]
```

```
    relevant_sorted = sorted(relevant)  # restore original order for coherence

    return " ".join(sentences[i] for i in relevant_sorted)
```

Sentence extraction is faster than LLM extraction but less precise. It cannot detect that a sentence is relevant only given the preceding sentence, or that a tangentially related sentence is the key to answering the query.

16.8 The full reranking pipeline

```
User query


[Optional] Query reformulation


Initial retrieval: dense + sparse, top-50 to top-100


Cross-encoder reranking: top-50 → top-15


[Optional] MMR diversity selection: top-15 → top-10


[Optional] Contextual compression: extract relevant sentences per chunk


Context assembly → Language model
```

Not every application needs every stage. The decision depends on latency budget, quality requirements, and corpus characteristics.

16.8.1 Latency budget allocation

For a 500ms TTFT target:

```
Query embedding:                    10ms
Dense retrieval (HNSW):             20ms
Sparse retrieval (BM25):            15ms
RRF fusion:                          5ms
Cross-encoder reranking
  (MiniLM-L-6, 50 candidates, GPU):30ms
Context assembly:                    5ms
LLM TTFT (first token):           ~200ms

Total:                              285ms  ← within 500ms budget

With LLM reranking (50 candidates): +2,000ms  ← blows the budget
With MMR (cosine similarities):        +10ms  ← acceptable
```

The reranking budget is typically 20–100ms in interactive applications. Cross- encoder reranking on GPU with small models (MiniLM) fits. Cross-encoder on CPU does not for most TTFT targets. LLM reranking does not.

16.9 Training custom rerankers

Off-the-shelf cross-encoders are trained on general retrieval datasets (MS MARCO, Natural Questions) but for specialized domains, fine-tuning can improve NDCG@10 by 5–15%.

16.9.1 Training data sources

User click data: queries users submitted plus results they clicked (positive) and results they saw but did not click (negative). High quality, requires sufficient production traffic.

Synthetic generation: use an LLM to generate queries for known-relevant chunks and sample hard negatives from the retrieval system.

LLM-judged pairs: retrieve candidates, judge relevance with an LLM, train the cross-encoder on those judgments.

16.9.2 Fine-tuning a cross-encoder

```
from sentence_transformers import CrossEncoder, InputExample
from torch.utils.data import DataLoader

train_samples = [
    InputExample(
        texts=["how long before session timeout?",
               "The default token expiration is 3600 seconds."],
        label=1.0,  # relevant
    ),
    InputExample(
        texts=["how long before session timeout?",
               "API rate limits apply to all endpoints."],
        label=0.0,  # not relevant
    ),
    # ... 1,000-5,000 more examples
]

model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2", num_labels=1)
train_dataloader = DataLoader(train_samples, shuffle=True, batch_size=32)

model.fit(
    train_dataloader=train_dataloader,
    epochs=3,
    warmup_steps=100,
    output_path="./domain-reranker",
)
```

Even 1,000–5,000 domain-specific examples produce measurable improvements for specialized corpora. The investment is typically worthwhile for high-value RAG applications where a general model underperforms.

16.10 Measuring reranking quality

16.10.1 Pipeline stage comparison

Measure how precision improves at each stage:

```python
def evaluate_pipeline_stages(
    eval_queries: list[dict],  # {query, relevant_chunk_ids}
    retriever,
    reranker,
    k_retrieve: int = 50,
    k_rerank: int = 10,
) -> dict:
    retrieval_recall    = []
    retrieval_precision = []
    reranking_precision = []

    for item in eval_queries:
        query        = item["query"]
        relevant_ids = set(item["relevant_chunk_ids"])

        retrieved    = retriever.retrieve(query, top_k=k_retrieve)
        retrieved_ids = {cid for cid, _ in retrieved}

        retrieval_recall.append(
            len(relevant_ids & retrieved_ids) / len(relevant_ids)
        )
        retrieval_precision.append(
            len(relevant_ids & {cid for cid, _ in retrieved[:k_rerank]}) / k_rerank
        )

        reranked     = reranker.rerank(query, retrieved, top_k=k_rerank)
        reranked_ids = {cid for cid, _ in reranked}

        reranking_precision.append(
            len(relevant_ids & reranked_ids) / k_rerank
        )

    return {
        f"retrieval_recall@{k_retrieve}":   np.mean(retrieval_recall),
        f"retrieval_precision@{k_rerank}":  np.mean(retrieval_precision),
        f"reranking_precision@{k_rerank}":  np.mean(reranking_precision),
        "precision_lift":
            np.mean(reranking_precision) / np.mean(retrieval_precision),
    }
```

A well-functioning reranker should lift Precision@10 by 20–50% relative to initial retrieval while maintaining the Recall@50 of the retrieval stage.

16.11 Key takeaways

- Reranking is the precision stage of a two-stage retrieve-then-rank pipeline; initial retrieval maximizes recall, reranking maximizes precision over the candidate set
- Cross-encoders jointly encode query and document through full attention, outperforming bi-encoders by 10–20 NDCG points; the tradeoff is that they cannot be pre-computed and are applied only to the candidate set (50–100 chunks), not the full corpus
- For chunks longer than the cross-encoder's context limit, use a sliding window approach and take the maximum window score as the chunk score

- LLM-based reranking achieves the highest quality but is impractical for interactive applications — use it for offline tasks, training data generation, and evaluation
- MMR selects a set of chunks that is both relevant and diverse; the lambda parameter controls the tradeoff; use diversity selection for multi-part queries, summarization, and corpora with heavy content duplication
- Contextual compression extracts only the relevant portion of each chunk, freeing context budget — powerful when combined with a large top-k, at the cost of additional LLM calls per chunk
- Cross-encoder reranking on GPU with MiniLM adds ~30ms — fits within a 500ms TTFT budget; LLM reranking adds ~2,000ms and does not
- Fine-tuning a cross-encoder on 1,000–5,000 domain-specific examples improves NDCG@10 by 5–15% for specialized corpora; synthetic query generation from chunks is a scalable way to create training data
- Measure both Precision@k improvement and end-to-end answer quality; retrieval precision gains do not always translate linearly to answer quality gains

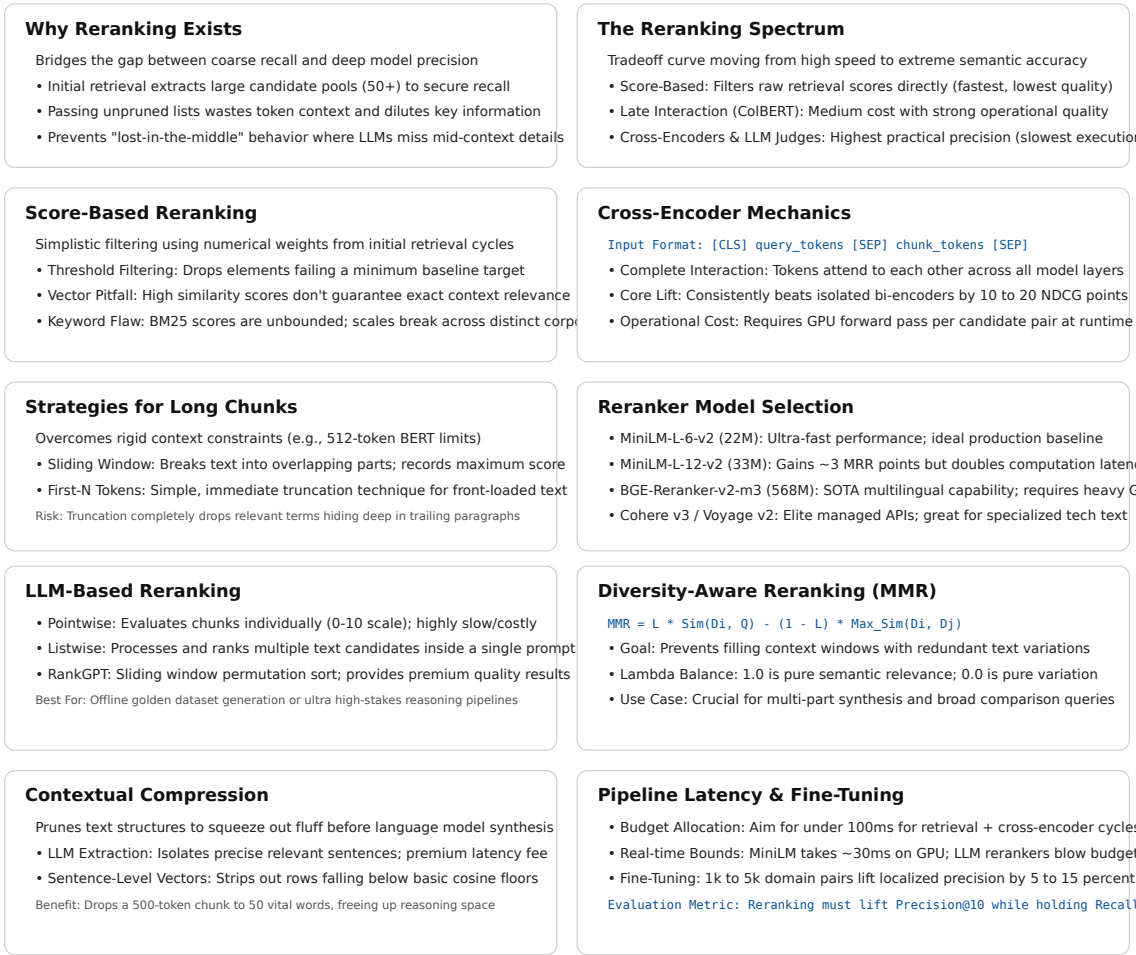


Figure 16.2: Cheat sheet.

16.12 Further reading

- Nogueira & Cho (2019). *Passage Re-ranking with BERT.* — Established cross- encoder reranking as the standard second-stage approach.
- Sun et al. (2023). *Is ChatGPT Good at Search? Investigating Large Language Models as Re-ranking Agents.* — RankGPT; listwise LLM reranking with sliding window.
- Pradeep et al. (2023). *RankZephyr: Effective and Robust Zero-Shot Listwise Reranking is a Breeze!* — Open-source LLM-based reranking.
- Liu et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts.* — Why reranking and context compression matter for LLM context utilization.
- Cohere (2024). *Rerank 3 Technical Report.* — State-of-the-art commercial reranker; useful benchmark reference.

Part V

The Forging of Intelligence

Part VI

The Trial of the Answer

Part VII

The Keepers of the Waking Engine

[1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {transformer-based} generative models," in *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, 2022, pp. 521–538.